

Feature: The booking component

From build-plan: feature 9

Branch: `feature/09-the-booking-component`

Status: verified. Steps 9.1 to 9.7 (and 9.1b) built, tested and reviewed step by step (audit, independent review, re-reviews), 2026-10-08 to 2026-10-09. Design review passed on the live preview (Frank, 2026-10-09). No P0 or P1 open or fixed. The checkpoint for the final review.

Goal

A customer on a business's own site presses "Book now", a modal in that site's colours opens, they pick a service (unless the button already named one), a day and a time, answer the standard and the business's own questions, press Book once, and see that they are booked. The booking, the contact and the lead land in that business through the public routes built in features 5 to 7, now rate limited. The modal never names its provider.

This feature builds the component and proves it on a booking preview page inside this repo. Wiring it into the agency's own site is feature 10.

Decisions

1. **The component calls the API straight from the visitor's browser** (Frank, 2026-10-08; answers project-plan open question 15). The public routes and their CORS from `WIDGET_ORIGINS` already exist (step 2.4); the rate limits see each visitor. Rejected: proxying through each host site's server action. Booking holds no secret (free times are public), and the proxy would need a key per site plus trusting the visitor address the host passes on, or every visitor would share one limit. The Sep 22 lean towards the proxy came from the clinic's store, where there was a secret. 2. **The code lives in a new workspace, `packages/booking-component` (`@scheduleads-app/booking-component`)**, compiled by `tsc` to `dist/` like `packages/shared`, with its own CSS file. One source for every tenant. Rejected: building it inside `frontend/` (feature 10 would move it out of the CRM app), or a copy in each host site (four copies that drift). How a host site in another repo gets it (GitHub Packages, a git dependency) is feature 10's question; this feature keeps the package buildable on its own. 3. **Who does the job is a per-service setting** (decision 30, answering the build plan's "open until this item's spec"): `booking_link.personChoice`, `customer_picks` (a salon: the customer sees the people who do that service, and "Any available") or `business_assigns` (Primo's estimates: no choice shown, any available). No database default, so every new service states it; existing rows are backfilled `business_assigns`, which is exactly how every booking behaves today. The server enforces it: a person sent for a `business_assigns` service is refused, never ignored. 4. **The layout is a stored value:** `booking_link.layout`, only `month` allowed by its check for now (project-plan decision 11). The modal

switches on it exhaustively, so the week strip in `prototypes/modal-primo.html` returns as one more value and one more branch. No database default; existing rows backfilled `month`. 5. **The business's own questions are a table, the answers a snapshot on the lead.** `booking_question` (per business, ordered, a label and whether it is required). The lead keeps each answer with the question's words as asked, so editing a question in feature 12 never rewrites what a customer was asked. Every answer is text for now; other kinds (a choice, yes or no) come with feature 12's ready-made sets for each trade, as a column whose existing rows read as text. Until feature 12 the questions are set by the seed and, for a real business, at its setup by hand. 6. **The modal opens on the service list when no service is named.** The header's "Book now" names none; a service page's button names its `bookingId` (face-and-body's contract: `Service.bookingId` is an opaque handle, here a booking link id; pages ask to book a service and do not know who answers). 7. **Rate limits are counted in the API's memory**, in fixed windows, written by hand (no package). The backend is one Railway instance; a restart forgets the counts, which lets at most one more burst through. Rejected: a Postgres counter table (a write on every public read and a cleanup job, for protection the one instance does not need yet) and `hono-rate-limiter` (a dependency for about forty lines). A second instance moves the counts to Redis (Frank, Oct 8: memory now, Redis then), written in the code beside the counter; routes reach the counter through one function, so the store can change under them. 8. **The limits** (agreed by Frank, Oct 8, at step 9.3's plan):

- per visitor, every `/public/*` read: 60 a minute;
- per visitor, every `/public/*` write (book, cancel, move): 10 in 10 minutes;
- per contact, bookings: 4 in 10 minutes for the same email, and separately for the same phone, in one business. A parent booking two or three children at once is never refused (5d, Oct 2); a script trying dozens is.

A refusal is `429` with `Retry-After` and the usual shape: `too_many_tries`, "Too many tries. Please wait a minute and try again." A booking is counted just before it is saved, after the form's own booking is looked up, so a form sent again after a lost answer gets its booking and never "too many tries", while many sent at once cannot all pass. Copies of one form (its request key) are booked one after the other, in the API's memory like the counts: a copy sent while the first is still being saved waits, then gets that booking from the look-up. The count is handed back when the booking is refused (a taken time, a missing answer), crashes, or is the form's own booking answered again: only bookings made count against the contact (9.3). A count handed back after its window ended frees nothing in the next one. 9. **The Book button locks after one press and the form owns one key** (5d decision 7). The key is made when the details screen opens. A retry after a lost answer sends the same key, so the server answers with the booking that won. A new key is made only for a new form: after a booking, or when the modal is opened again. 10. **The modal is a native `<dialog>`** opened with `showModal()`: the browser gives the focus trap, Escape and the inert page behind it. Focus returns to the button that opened it. 11. **Times show in the business's time zone, named on the screen**, the same rule as the customer's page in 7b. 12. **The Sep 23 mock, checked against everything decided since** (Frank, Oct 9, before step 9.5). The shape stands (`prototypes/modal-month.html`, `modal-month-details.html`); eight details did not fit: 1. Who does the job: the rail gets "Who would you like?", "Any available" first and chosen, then each person; only for `customer_picks` (Frank's yes). 2. No time-zone switcher: every time in the business's zone, named under the calendar

(Frank's yes). Note for feature 10: the agency's video calls may want the visitor's own clock, as a per-service setting. 3. No struck-through busy times: the times route sends free times only, and showing busy ones would show the business's calendar. 4. No location or footer line in the rail ("At your home...", "No charge, no obligation"): a service has no fields for them; the rail shows the service's description. Fields for them, if a business wants them, are feature 12's settings. 5. The service list, absent from the mock, is the one built in 9.4. 6. Screen two marks email and phone as "at least one is required", not all three required (the rule until feature 12). 7. No "+ Add another contact": in no plan. 8. The screens for when something goes wrong are 9.4's three, plus 9.6's "time taken". Frank asked to decide 3 to 8 without asking (Oct 9). The look itself gets a design review on the live preview page before `/complete`, with the `email-design-eng` skill read from the blueprint for the polish. 13. **The customer's yes to text messages is a yes to later texts only** (Frank, Oct 9, answering the open question before step 9.6). Texts about the booking she made (8b's confirmation and reminder) go as now, ticked or not: she asked for that booking. The box is her yes to later texts (offers, "time to book again"), saved on her contact with its date; an unticked box never takes back an earlier yes. Whether screen two asks at all, and what a business later sends with the yes, are the business's settings (decision 30). Check the consent rules with someone who knows them before a real client relies on it.

In scope

- `booking_link.layout` and `booking_link.personChoice`, migration and seed.
- The public routes grow: the business's face (name, logo, phone) and its questions with the service list; layout and personChoice with one service; the people with the times only when the customer picks; a person refused for a `business_assigns` service on the times route and the booking route.
- `booking_question`, `lead.answers`, answers sent with a booking, checked and saved, and shown in the business's booking notification email.
- Rate limits on every `/public/*` route.
- `PublicAppType`: the backend's type of the public routes only.
- The package: `BookingProvider`, `useBooking()`, `BookNowTrigger`, the modal, its CSS on `--sa-*` tokens, the month layout's three screens and the done screen, with every state below.
- A booking preview page for the platform admin, `/admin/booking-preview/[businessSlug]`, hosting the component in the agency's and Primo's tokens from `prototypes/`.
- The plans: open question 15 moved to decided, the build plan's "open until this item's spec" line answered.

Out of scope

- Any host site, its `siteConfig`, `WIDGET_ORIGINS` values: feature 10.
- How another repo installs the package: feature 10.
- The week strip layout (parked, decision 4).

- The owner editing questions, layout, who-picks or the email or phone rule: feature 12. Until then "an email or a phone, at least one" stays the rule.
- Question kinds other than text (decision 5).
- The hosted page `/book/<slug>` (24), an embed script (named, not planned), deposits (open question 16).
- The customer's own booking page (7a, 7b) stays as built, except that a service the business assigns offers no pick of person there either (step 9.1b, from F-256, Frank 2026-10-08).

Build loop

`workflow.stepReview` is `every` and `checkpointCommits` is `enabled`.

After the green light, nothing stops until the review. Each step's plan gets Frank's yes just before it is built. From that yes the step runs straight through: build, tests and checks, tick the box, the build log entry, commit and push to `feature/09-the-booking-component`, `/audit` scoped to the step, the independent review. The planned stop is after the review, where the findings are talked through. Earlier stops only for: the agreed plan proving wrong, a line only Frank crosses (a package install, Railway or real data, `main`, a merge, a force push, deleting anything), or blocking findings left unfixed.

Build steps

- ☒ **9.1 The service's two settings and the business's face.** Shared: `booking_link.layout` (text, not null, check `in ('month')`) and `booking_link.personChoice` (text, not null, check `in ('customer_picks', 'business_assigns')`), no database defaults, one migration that adds them, backfills existing rows (`month`, `business_assigns`) and then sets not null. The seed states both on every service: Summit Painting (dev) `business_assigns`, Riverbend Clinic (dev) `customer_picks`. Backend: `GET /public/:slug/booking-links` answers `{ business: { name, logo, phone }, bookingLinks }`. `GET /public/:slug/booking-links/:id` adds `layout` and `personChoice` to `bookingLink`. The people (active persons who do that service, nobody ticked meaning everyone, by name) are the times route's own `people`, which it already listed since 5c, now empty when the business assigns (amended at 9.1: one list, not a second copy on this route). The times route and `POST /public/:slug/bookings` refuse a person for a `business_assigns` service: 400, "This service does not take a pick of person." The owner's own path (`bookTime` with `source: manual`) is untouched. Plans: project-plan open question 15 moved into a decided entry (browser to API, 2026-10-08, decision 1's reason), the overview's open-questions line and fingerprint refreshed, the build plan's "open until this item's spec" sentence replaced by decision 3. **Done when:** route tests show the business face, both settings, people only for `customer_picks` (inactive and unticked people excluded as the rules say), and both refusals; the seed runs; backend and shared suites pass.
- ☒ **9.1b The customer's own booking page follows who picks** (F-256, Frank 2026-10-08). Decision 3 reaches the 7b page: for a service the business assigns, "Change the time" lists no people and offers

no pick. Backend: the booking page's answer carries the service's `personChoice`. The move times route (`GET /public/bookings/:token/times`) answers `people: []` for a service the business assigns and refuses a person asked for, 400 with the same "This service does not take a pick of person."; the move (`POST /public/bookings/:token/move`) refuses a person the same way, after its own "pressed twice answers the same" check so a repeat still answers as before. A service the customer picks for is unchanged (decision 14: the panel opens on her own person). Amended after the step's review (Frank, 2026-10-08, F-263 option A): a move with nobody picked, under either setting, keeps the booking's own person while they are free; someone else only when they are busy, and "try again" when their calendar cannot be read (F-265). Frontend: `change-time-panel.tsx` shows no person choice for a service the business assigns, asks the times with nobody picked (any available), and its last question names the time only. The booked person's name on the page stays: the customer is told who is coming. **Done when:** route tests show the empty people list, both 400s with nothing moved, a move with nobody picked moving, and the clinic's picker answers unchanged; the frontend builds; with the two Scheduleads dev servers running (3400, 3401), a painting-dev booking's page shows no painter list and a clinic-dev booking's page still shows its picker, screenshots taken; suites pass.

- ☒ **9.2 The business's own questions, answered and saved on the lead.** Shared: `booking_question` (`id`, `organizationId`, `position` integer, `label` 1 to 200 characters, `required` boolean, timestamps; organization-scoped, cascade with the business) and `lead.answers` (jsonb, null when the business asks nothing: `[{ questionId, question, answer }]`, `question` being the label as asked). The create-booking schema takes `answers: [{ questionId, answer }]`, at most 20, each answer trimmed, at most 500 characters; the route's body limit grows from 16 KB to 64 KB to fit 20 full answers in multi-byte text. The seed gives each dev business two or three questions, one required. Backend: the service list's `business` carries `questions: [{ id, label, required }]` in position order. A booking is refused (400) with an answer to a question the business does not have, two answers to one question, or a required question unanswered or blank ("Answer: {label}"). `bookTime` saves the snapshot on the lead in the same transaction as the rest. The business's booking notification email shows each question and answer under the customer's own words, as text, never HTML. **Done when:** route tests cover saved answers with their words as asked, each refusal, a business with no questions (answers absent, `lead.answers` null); the email test shows the answers escaped; suites pass.
- ☒ **9.3 Rate limits on the public routes.** Backend: one limiter (decision 7) in `backend/lib/rate-limit/`, a middleware on `/public/` counting reads and writes per visitor, and the per-contact count inside the booking route after the body is valid (email lowercased and trimmed; phone through 8b's `textable-phone-number` helper, so `+1 403...` and `403...` are one phone, and any other number reduced to its digits), both keyed by business. The numbers and the refusal are decision 8. The clock is injectable for tests. The visitor's address: in production the address Railway's proxy puts in its header (the step confirms the header name from Railway's documentation and writes it in the code and the deploy notes), otherwise the socket's address. Each counter's window entry is dropped once it is over, so memory stays bounded by recent visitors. `/texts/` (Twilio, signed) and the dashboard routes are not limited here. **Done when:** tests show the 61st read in a minute refused with `Retry-After` and

allowed again in the next window, the 11th write refused, the 5th booking for one email refused while a second email from the same visitor still books, the 5th booking for one phone written different ways (+1 403... , 403-...) refused (added with Frank, Oct 8), two businesses counted apart, and a parent booking two children at the same time both booked; suites pass.

- ☒ **9.4 The package, the provider and the service list.** Backend: `PublicAppType` , the type of a Hono app holding only the `/public` routes, exported beside `AppType` through `build:types` , so the package never sees admin or dashboard route shapes. Package `packages/booking-component : package.json` (React and React DOM as peer dependencies; `hono` and `@scheduleads-app/shared` as dependencies, both already in this repo at the same versions, so nothing new is downloaded; Vitest as a dev dependency, likewise already here; the step's yes covers these declarations), `tsconfig` , `dist/` output keeping `"use client"` , and `booking-component.css` ported from `prototypes/widget.css` and `month.css` : every colour, font and radius reads a `--sa-*` token with a neutral fallback, no CRM token. Exports, documented in the package's `index.ts` :
`BookingProvider({ apiUrl, slug, children })` wraps the host's children so server components stay server components; `useBooking()` returns `{ open(bookingId?: string) }` ;
`BookNowTrigger({ bookingId?, className?, ariaLabel?, children })` is one unstyled `<button type="button">` . Calls go through `hc<PublicAppType>` with credentials omitted, each with a 10-second ceiling, so a stuck spinner is impossible. The modal (decision 10): opens on the service list when no `bookingId` is given (name, duration, description as text), or straight on that service. States: loading; no services ("Nothing can be booked online right now.", and the business's phone as a call link when it has one); a service id that is not offered (the same words); cannot load or too many tries (the message, Try again, and the phone). Frontend: the workspace dependency, and `/admin/booking-preview/[businessSlug]` for the platform admin only (anyone else gets the same not-found as a wrong address), a plain host page with a Book now button, one button per service, and a switch between the agency's and Primo's tokens. **Done when:** the package builds and its tests pass; the frontend builds; on the booking preview page with clinic-dev, Book now opens the service list, Escape closes it and focus is back on the button; a service button opens that service; a slug with no services shows the empty words; nothing on screen names the provider; the look switch moves the window between the agency's look and Primo's (added with Frank at the plan, Oct 9). Amended while building (Oct 9): the package also declares `typescript` , `@types/react` and `backend` (for `PublicAppType` , build time only) as dev dependencies, all already in the repo (Frank's yes, Oct 9). The phone shows wherever the business was read; when even the business could not be read (the first call failed) there is no phone to show. "Nothing to book" has no Try again. The preview page's not-found is Next's own 404 page, reached after the session check in the browser, so its HTTP status is 200. The week strip's classes in `widget.css` were not ported (decision 4 parks that layout).
- ☒ **9.5 Screen one: pick a time.** The left rail: the business's logo (or its name), the service, its duration and description. When the service is `customer_picks` : "Who would you like?", "Any available" first and chosen, then the people by name (decision 12). A month calendar from today to the horizon, times asked one month at a time (the route's 31-date limit), days with a time marked and pickable, the chosen day's times listed, the time zone named. Previous month stops at the current month, next stops at the horizon. The first day with a time is chosen when a month loads, so its times show at once.

A chosen time splits into the time and Next; Next opens screen two, an empty shell until 9.6. States: loading; no times this month ("No times left in {month}." and Next month while the horizon allows); cannot read times (503: the route's own words, "Times cannot be read right now. Try again shortly.", Try again, the phone); too many tries (9.4's words). Changing the person reloads the month. The day and time picks are keyboard reachable buttons with clear names ("Thursday, October 15", "9:30 AM"). Logic, in plain functions with tests: the month's grid, grouping times by day in the business's zone, the month bounds from today and the horizon. **Done when:** those tests pass, and on the booking preview page with clinic-dev (customer picks) and painting-dev (business assigns) a time can be picked, each state is shown by a real case where one exists (a closed day, a month past the horizon), and screenshots of both themes are taken. Built (Oct 9): the window now keeps the one-service answer's `availability` (the business's zone and horizon) for the calendar; screen two is an empty shell whose rail keeps the picked time, the person and the zone, and Back returns to the same month and person. The picked time is set in the times' own font, not the mock's monospace, which wrapped "10:00 a.m.". Not seen live, because the seed has no such case: a month with no times left, and a calendar that cannot be read (503); both are covered by tests of what the window is handed.

- ☒ **9.6 Screen two, Book, and done.** The rail adds the chosen time. The standard questions: name, email, phone, address, what they want done; "At least one is required" under email and phone; then the business's own questions, required ones marked. Checked in the browser with the shared create-booking schema before sending; each error under its field, tied with `aria-describedby`, `aria-invalid` set, focus on the first wrong field, cleared when that field is edited. Book locks after one press, shows it is sending, and sends the form's key (decision 9). Answers: time taken (back to screen one with the route's words and the times reloaded), form already used, cannot read, too many tries, a lost connection (the choice kept, Try again with the same key). Done: "You're booked", the service, the time with its zone, the person, and "A confirmation is on its way to {email}" when an email was given. Closing and opening again starts a new form. User text (the customer's own and the business's labels) is rendered as text only. **Done when:** on the booking preview page a real booking is made in clinic-dev with answers, and the lead, its answers and the booking are in the database; pressing Book twice quickly makes one booking; a taken time (booked from a second tab) shows the taken message with fresh times; each field error shows and clears; suites pass.
- ☒ **9.7 The yes to later texts** (decision 13; plan gone through with Frank and approved Oct 9). Screen two gains one optional box, never ticked at first, only for a business whose setting says to ask (set per business at client setup until feature 12; off until set). The booking texts (8b) are untouched, ticked or not; owner-made bookings are untouched. 9.7.1 Data: `text_settings.askLaterTextsYes` (boolean, false until the business sets it; a business with no text settings never asks); `contact.laterTextsYesAt` (nullable, the date of her latest yes); a new timeline type `later_texts_yes`; one migration generated from the schema. The seed sets painting-dev to ask; clinic-dev has no text settings (it is the seed's business that sends no texts), so it never asks, and the preview shows both. 9.7.2 API: the public business route sends the box's sentence, written by the API with the business's name, or nothing when it does not ask. The shared create-booking schema takes an optional `laterTextsYes`. A tick with no phone is refused under Phone ("Enter a phone for texts."), since a yes with no number proves nothing; a tick sent to a business that does not ask is ignored. A tick

stamps the contact's date and adds one timeline entry with the booking, holding the phone and the same sentence the API wrote, so what was shown and what was saved cannot drift. An unticked box changes nothing, so an earlier yes stays. 9.7.3 The box: under Phone, unticked, the API's sentence as its label, kept with the form while she moves between screens and sent with Book. **Done when:** a booking with the box ticked stamps the contact and its timeline, one without it leaves an earlier yes in place, a tick with no phone is refused under Phone, a business that does not ask shows no box and ignores a tick, and the booking texts go the same either way; suites pass; the box is seen on the painting-dev preview and absent on clinic-dev. Built (Oct 9): each yes is saved with its number, one row per number in a `later_texts_yes` table (F-291, Frank: one yes per number), because a contact is matched by email and its phone is never changed by a public form: a yes belongs to the number it was given for, and a tick with another number adds a yes, never moves one. A tick needs a phone that can get texts ("Enter a phone that can get texts."), checked by the shared schema in the browser and the route alike. The migration is `0024_later_texts_yes`. Seen live: a ticked booking on painting-dev saved the date, +14035550148 and the sentence, with its confirmation text as usual; clinic-dev's form has no box.

Files / areas

- `packages/shared/db/booking-tables/booking-link-table.ts`, a new `packages/shared/db/crm-tables/booking-question-table.ts`, `packages/shared/db/crm-tables/lead-table.ts`, `packages/shared/db/index.ts`, `packages/shared/migrations/`, `packages/shared/scripts/seed-dev.ts`.
- `packages/shared/zod-validation/booking-links-validation-schemas/create-booking-validation-schema.ts`.
- `backend/routes/public-booking-links-routes.ts`, `backend/routes/public-bookings-routes.ts`, their tests; `backend/lib/booking/book-time.ts`; `backend/lib/scheduling/find-service-resources.ts` (who does a service); `backend/emails/booking-notification.tsx`; a new `backend/lib/rate-limit/` and `backend/middleware/public-middleware/`; `backend/app.ts`, `backend/tsconfig.types.json`, `backend/package.json` (the `PublicAppType` export).
- New `packages/booking-component/`.
- `frontend/package.json`, a new `frontend/app/admin/booking-preview/[businessSlug]/`.
- `blueprint/project-plan.md`, `blueprint/build-plan.md`, `blueprint/context/project-overview.md` (step 9.1's plan lines).

Data / contracts

- `booking_link.layout`: `'month'`. `booking_link.personChoice`: `'customer_picks' | 'business_assigns'`. Both not null, no default.
- `booking_question`: `id` text (the repo's id generator), `organizationId`, `position` integer (order; not unique), `label` 1 to 200, `required` boolean not null, `createdAt`, `updatedAt`. Unique on

(`organizationId`, `id`) for scoped lookups.

- `lead.answers`: jsonb, null or `[{ questionId: string, question: string, answer: string }]` in the business's question order; an optional question left blank is left out.
- `GET /public/:slug/booking-links` 200: `{ business: { name, logo: string | null, phone: string | null, questions: [{ id, label, required }] }, bookingLinks: [...] }`.
- `GET /public/:slug/booking-links/:id` 200: `{ bookingLink: { ...as now, layout, personChoice }, availability }`.
- `GET /public/:slug/booking-links/:id/times` 200: as since 5c, `{ timezone, people: [{ id, name }], startTimes }`, `people` empty when the business assigns (amended at 9.1), plus `localStartTimes: [{ startsAt, date, time }]`, each time's business date and clock label worked out by the API, which the window shows as sent (F-279, 9.5 review); each also carries `when`, the whole moment as the emails say it (F-281).
- `GET /public/bookings/:token` 200: the booking gains `when`, and `GET /public/bookings/:token/times` 200 gains the same `localStartTimes`; the customer's page shows those as sent (F-281, 9.5 review).
- `POST /public/:slug/bookings` body adds `answers?: [{ questionId, answer }]`. New refusals: 400 person not taken, 400 answers; 429 `too_many_tries` with `Retry-After` (seconds). The 201 answer's booking gains `when`, the moment as the emails say it, which the done screen shows as sent (amended at 9.6, as F-279 did for the times).
- Every public refusal keeps `{ error: { code, message } }`.
- Theme tokens the host defines (from `prototypes/theme.css`): the `--sa-*` set used by `widget.css` (`--sa-bg`, `--sa-surface`, `--sa-surface-2`, `--sa-text`, `--sa-muted`, `--sa-border`, `--sa-identity`, `--sa-identity-ink`, `--sa-font`, `--sa-font-mono`, `--sa-radius`, and the rest the port finds), each with a fallback.

Testing

- Backend (Vitest, local `scheduleads_dev`): route tests for 9.1 to 9.3, `bookTime` answers, the notification email, the limiter with a pinned clock. Each new rule's test is shown able to fail.
- Shared: the create-booking schema's answers.
- Package (Vitest, new in 9.4): the month grid, grouping by day in a zone, month bounds, the request key's lifetime.
- No browser test harness exists (`Browser tests` not declared), so the screens are checked by hand on the booking preview page with screenshots, named as such in each report, not claimed as automated.
- No `Verify` command exists; each step runs the shared and backend suites, the package's tests, and the frontend build.

Deploy notes

- The per-visitor limits read the visitor's address from `X-Real-IP` , the header Railway's proxy adds (Railway's networking docs, Specs & Limits), only when `NODE_ENV=production` ; the API must run with it set. Railway's docs do not say whether the proxy replaces an `X-Real-IP` a visitor sends themselves: check it at the first deploy (send one with curl and see which address the API counts), with F-176 and F-179.
- The counts live in the API's memory: one API copy (one replica). A second copy moves them to Redis first (decision 7).
- The API goes out before the frontend (9.5 review): the window, the customer's page and its panel read `localStartTimes` and `when` , which an older API does not send; the frontend's two calls trust the answer's shape.

Notes for the AI

- Never the word `scheduleads`, or any name of this product, anywhere a customer can see it: text, titles, `aria-*` , alt text.
- Every class in the package's CSS is `sa-` prefixed so it cannot collide with a host's; it reads only `--sa-*` tokens (`prototypes/widget.css` 's own rule: a CRM token in it is a bug).
- The public routes stay one identical "not here" for an unknown business, service or inactive service; the new people list never reveals a person's login, email or phone.
- The package does not import from `frontend/` ; the booking preview page imports the package, never the other way.
- 8b's texts, 6's emails and 7's links are unchanged by 9.1 to 9.6.
- Only a note for feature 10: no `agents-web` folder exists on this laptop; tenant zero's site is probably `agency-site-app` .

Design review (Frank, Oct 9, on the live preview, clinic-dev and painting-dev): the layout, the forms and the colours are approved as the start. Each business making its own services, questions and colours is Settings, feature 12; nothing changes in feature 9.

Open questions

1. Answered Oct 9, decision 13: a yes to later texts only. What does the customer's "yes to text messages" do? The build plan (Oct 2) puts an optional yes on screen two. Feature 8b (Oct 7) since sends the confirmation and reminder texts to any textable phone a booking carries, as messages about a booking the customer asked for. Either the yes decides whether this customer gets those texts (an unticked box means no booking texts, and owner-made bookings need the same choice), or it is only saved on the contact for later messages and booking texts go as now. Decided before step 9.6.

Note for feature 12 (Frank, Oct 9, at step 9.6): whether the form asks for an address becomes the business's setting; until then every booking requires one, as the API has since 5d. Written into item 12 of the build plan.

Note for later (Oct 8, not this feature): the limits stop a script from one address, not one that keeps changing addresses, emails and phones. The usual next layer is an invisible bot check on the Book button; no plan holds it. Added only if fake bookings show up.

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

shared: the service's settings, the questions and the yes (9.1, 9.2, 9.7)

- `db/booking-tables/booking-link-table.ts` (migration 0022): `layout` (only `month` passes its check) and `personChoice` (`customer_picks` or `business_assigns`), both not null with no database default, so every new service states them. The migration backfilled existing rows as `month` and `business_assigns`, which is how every booking behaved before.
- `db/crm-tables/booking-question-table.ts` and `lead.answers` (migration 0023): a business's own questions in its order, and on the lead a snapshot of each answer with the question's words as asked, so editing a question in feature 12 never rewrites what a customer was asked. Null when the business asks nothing.
- `db/crm-tables/later-texts-yes-table.ts` and `text_settings.askLaterTextsYes` (migration 0024): the plan said a date on the contact; it became one row per (business, contact, phone) with the date of the latest yes, because a contact is matched by email and a public form never changes its phone. A yes belongs to the number it was given for; a tick with another number adds a yes and never moves one. The timeline gains `later_texts_yes`.
- The create-booking schema takes `answers` (at most 20, each trimmed, at most 500 characters) and an optional `laterTextsYes`; a tick needs a phone that can get texts, checked by the same schema in the browser and the route.
- The seed states both settings on every service (Summit assigns, Riverbend's clinic lets the customer pick), gives each business its questions, and sets Summit to ask for the yes. Where a database was seeded before a setting existed, the seed brings it up only while the business's rows are untouched, so a choice made by hand survives a reseed.

backend: the public routes grow (9.1, 9.1b, 9.2, 9.5, 9.7)

- `routes/public-routes.ts` gathers every `/public` route into one app whose type, `PublicAppType`, is all the booking component's typed client can see: no admin or dashboard route shape reaches the package.
- The service list answers the business's face (name, logo, phone), its questions and the box's sentence (`find-later-texts-yes-words.ts`, written only here, so what she saw is what is saved). One service adds `layout` and `personChoice`. The times route's `people` is empty when the business

assigns, and a person sent for such a service is refused 400 there, on the booking route, the move times route and the move (`find-person-choice.ts` , `errors/person-not-taken.ts`).

- `lib/scheduling/local-start-times.ts` : every free time also comes with the business's date, clock label and the whole moment as the emails say it, worked out with the API's own time-zone rules. The window and the customer's page show these as sent, because an older browser's rules put every Alberta time after Nov 1 2026 an hour early. The booking's 201 answer and the customer's page carry `when` the same way.
- 9.1b, the customer's own page: a move with nobody picked keeps the booking's own person while they are free, under either setting; someone else only when that person is busy, and "try again" when their calendar cannot be read (`find-booking-choices.ts` , `find-booking-move-times.ts`). The change time panel offers no pick of person for a service the business assigns.
- `lib/booking/book-time.ts` : in order, the form's own booking is looked up by its key first (so a form sent again gets its booking, whatever changed since), then the contact limit, then the person choice, then the answers (`check-answers.ts` : unknown question, two answers to one, a required one blank, each refused 400 "Answer: {label}"), then the save. The answers' snapshot and the yes to later texts are written in the same transaction as the booking. The business's notification email lists the answers as text.

backend: rate limits (9.3)

- `lib/rate-limit/create-rate-limiter.ts` : a counter per key in fixed windows, in the API's memory, reached only through `take` and `giveBack` , so a second API copy can move the store to Redis without touching a route. A window's entry is dropped once it is over; a count handed back after its window frees nothing in the next one. The clock is injectable for tests.
- `public-rate-limiters.ts` holds the three limits (60 looks a minute and 10 actions in 10 minutes per visitor, 4 bookings in 10 minutes per email and per phone in one business). `public-rate-limit-middleware.ts` is mounted after CORS, so a preflight is never counted and a refusal still carries the CORS headers. `visitor-address.ts` reads Railway's `X-Real-IP` only in production.
- `booking-contact-keys.ts` : the email lowercased and trimmed, the phone through 8b's textable-phone helper, so `+1 403...` and `403...` are one phone; each key inside one business.
- The contact count is taken just before the save and handed back when the booking is refused, crashes, or turns out to be the form's own booking answered again: only bookings made count. `one-copy-of-a-form-at-a-time.ts` runs copies of one form (its key) one after the other, so a copy sent while the first is saving waits and is then answered that booking instead of taking a second count. Three re-reviews shaped this (copies sharing a count let a fifth booking through; the turn is cleared from memory once done).

packages/booking-component: the window (9.4 to 9.7)

- A new workspace, compiled by `tsc` to `dist/` with `"use client"` kept. `index.ts` exports `BookingProvider` (wraps the host's children, so server components stay server components),

`useBooking().open(bookingId?)` and `BookNowTrigger`, an unstyled button. `booking-component.css` is ported from the prototypes: every class `sa-` prefixed, every colour, font and radius a `--sa-*` token with a neutral fallback.

- `api-client/` : `hc<PublicAppType>` with credentials omitted and a 10-second limit on every call (`fetch-with-time-limit.ts`), so a stuck spinner is impossible. `problem-from-api-answer.ts` turns any answer into one of four problems; `worst-booking-problem.ts` picks which to say when two calls fail at once. `send-booking.ts` never throws, and tells "no answer at all" apart from a refusal.
- `booking-window.tsx` : a native `<dialog>` (decision 10), mounted only while open, which owns the form and its key for as long as it is open, so Back and a taken time keep her words, and a new opening makes a new key.
- Screen one (`month-layout/`): the rail with the logo or name, the service and, for `customer_picks`, "Who would you like?" with Any available first. `use-month-times.ts` asks one month per person and drops an answer for a month or person no longer shown; the first day with a time is chosen when a month loads. The calendar logic (`month-calendar/`) is plain functions with tests: the grid, the bookable months from today to the horizon, the dates to ask, grouping the API's labels by day.
- Screen two (`booking-form-view.tsx`): checked with the shared schema before sending, each error under its field and the focus on the first. The form freezes (fields disabled, no Back to the times) from the press until an answer settles it: no answer, a server fault the route did not write, or a limit reached while unsure keep it frozen, because the booking may exist and another time with the same key would answer "already used" while she is in fact booked. Try again sends the same booking with the same key. A status line holds the focus while a send is out, and the words take it when a send ends with words, so focus never falls to the page. This was the longest thread of the feature: five re-reviews of 9.6 and the final review each closed a smaller gap in it.
- The done screen shows the service, the time as the API wrote it, the person, and where the confirmation went. The box for the yes to later texts shows only where the business asks, never ticked at first.

frontend: the preview page and the renames

- `/admin/booking-preview/[businessSlug]` : a stand-in client site for the platform admin (anyone else gets the plain not-found), with Book now, one button per service and a switch between the agency's look and Primo's. This is where the design review passed (Frank, 2026-10-09).
- Between 9.2 and 9.3 the frontend was renamed to say what each piece is: `lib/api-client.ts` split into one file per call under `lib/api-client/` (the dashboard client sends the login cookie, the public one never does); the customer's page moved to `/customer-booking/[bookingPageToken]`, still reached at `/b/<token>` through a rewrite so every link already sent keeps working; the sign-in, client setup and dashboard screens split into named components on a shared `centred-card`.

Settled after the steps

- The final review found that Back stayed live during the first send, so a lost answer after Back would lose the freeze (F-293); the form now freezes from the press. It also brought the coding standards up to decision 1 (browser to API), cleared the last history from comments, and made the seed bring an older database's yes setting up.
- Deploy notes, carried: `X-Real-IP` replacement by Railway's proxy and `NODE_ENV=production` are checked at the first deploy with F-176 and F-179; the counts need one API copy; the API goes out before the frontend.
- For feature 10: the package's built types import `backend/app-type`, which only resolves inside this repo, so a host site in another repo needs that settled with how it installs the package.

Findings

9/F-58 [P3] closed - The seed never gives an existing Chemical Peel its 15-minute step, so a migrated (not rebuilt) dev database keeps it empty

File: packages/shared/scripts/seed-dev.ts:485 **Found:** 2026-10-02 by independent review of step 5c.1 (scope: bf53ee6..d5a5878; lenses: all) **Why it matters:** The seed's find-or-make inserts a service only when its slug is missing (`if (!existingLink)`), so `slotIntervalMinutes: 15` reaches the database only on a fresh build. Checked read-only on the local `scheduleads_dev` after this step: migration 0012 is applied, yet every clinic service, Chemical Peel included, has `slotIntervalMinutes` null. The build log says so, and dev databases are disposable, so nothing is wrong in 5c.1 itself. The risk is 5c.4 and 5c.5, whose tests run "on the seeded clinic": any test that leans on the peel's seeded 15 passes on a rebuilt database and fails on this one, or on any machine that only ran `db:migrate` and `db:seed`. **Suggested fix:** Nothing to change in 5c.1. Before 5c.4, rebuild the local `scheduleads_dev` (drop, migrate, seed), and have the 5c.4/5c.5 tests that need a step set it on a service they create themselves rather than read the seed's value. **Resolution:** Settled 2026-10-08 on chore/cleanup-before-9 with no code change, as the suggested fix asked: the local `scheduleads_dev` has been rebuilt since (chemical-peel reads `slotIntervalMinutes 15`), and every test that needs a step sets it on a service it makes itself (find-free-times.test.ts:86, booking-link-slot-interval-rules.test.ts:27-33, apply-free-times-rules.test.ts:63 and 302, the two move route tests); git grep finds no test reading the seeded peel. The seed keeps making rows only while none exist, on purpose, so settings changed by hand survive a reseed; a migrated-only database still needs the documented rebuild (drop, migrate, seed). Closed 2026-10-08 by independent review of chore/cleanup-before-9 (392cc7d..394cca2; lenses: quality, security, performance, tests): rechecked read-only, clinic-dev's chemical-peel reads `slotIntervalMinutes 15` and every other clinic service null. git grep finds no test naming the seeded peel: the only "Chemical Peel" in a test is booking-link-slot-interval-rules.test.ts:30, a service that test makes under its own slug, and every `slotIntervalMinutes` in a test is on a service or object the test builds itself. The risk it named (a test leaning on the seeded 15) does not exist, so the suggested fix is met; the seed keeping hand-made settings is its intended behaviour.

9/F-62 [P3] accepted - Each start costs four Intl calls, repeated for every person, which grows "any available" on a public route

File: backend/lib/scheduling/apply-free-times-rules.ts:71 **Found:** 2026-10-02 by independent review of step 5c.2 (scope: cb29f51..b5bf8c4; lenses: all) **Why it matters:** `localTimeToMoment` runs `formatToParts` four times per candidate start (about 12 microseconds each call here). Measured on this laptop: 31 days of all-day windows with a 15-minute step is about 40 ms for one person; ordinary office hours (8 to 18, weekdays) about 10 ms. 5c.4 applies the rules per person, so "any available" over six practitioners repeats identical date and minute conversions six times, roughly 60 ms typical and 250 ms worst case of synchronous work per public request, on a route with no rate limit yet. Unverified because no real request has been timed; 5c.4 does not exist yet. **Suggested fix:** When 5c.4 is built, time an "any available" request on the seeded clinic. If it matters, convert each (date, minute) once per request and share it across people, or work out each date's offset once and only fall back to `localTimeToMoment` on clock-change days. **Resolution:** Confirmed (unverified to open) by independent review of step 5c.4 (2026-10-02). The cost is real and is synchronous work, so reading people side by side (Promise.all) shortens the database and Google waits but not this: each person's `applyFreeTimesRules` still runs one after another on the event loop. Measured in a scratch copy, the rules alone for 7 people over 31 dates (30 minutes, 15 after, one room): weekdays 9 to 17 every 15 minutes, about 73 ms; every day all day every 15 minutes, about 416 ms; every day all day every 5 minutes, about 1.15 s. The build log's 75 ms for the dev clinic matches the first case, so that request is almost all this work, not the reads. Fine for the four tenants' daytime hours; it grows with long windows and a small step (the database allows any step above 0), on a public route with no rate limit yet. Stays a P3, for feature 9 (first public traffic) or feature 12 (where an owner sets the step): work out each date's offset once per request and share it across people, or put a floor on the step. Measured 2026-10-08 on the seeded clinic-dev (six practitioners, no calendars connected), `findFreeTimes` with any available, whole call including the database, five runs each: a week ahead 12 to 28 ms median for every service except Chemical Peel (15-minute step, 192 starts) at 63 ms; 31 days ahead 12 to 32 ms, Chemical Peel 105 ms. The customer page asks a week at a time. Left for Frank to decide. Accepted by Frank 2026-10-08: the measured times are under what a customer can notice, and a 15-minute step is the business's own choice (more start times, more to loop over), so it stays; worth revisiting only if traffic grows far beyond this.

9/F-94 [P3] closed - `chooseAnyAvailable` has no caller outside its own test, while the spec still says the booking's order comes from it

File: backend/lib/scheduling/choose-any-available.ts:11 (spec: blueprint/context/current-feature.md:255 and :434) **Found:** 2026-10-02 by the second final independent review of feature 5d (scope: 12a21d6..3cec4ae; lenses: quality, security, performance, tests) **Why it matters:** `bookTime` takes the whole try order from `orderAnyAvailable` (book-time.ts:276), so `chooseAnyAvailable`, built in 5c for this booking, is now called only by `choose-any-available.test.ts`. Its header still says it decides who gets an "any available" booking, and the spec's 5d.3 bullet and Notes for the AI still name it as the function the booking reuses. A reader following the spec opens a function nothing in the app runs, and the

two files keep one rule behind two entry points that can drift. **Suggested fix:** Either delete choose-any-available.ts and move its useful cases into order-any-available.test.ts, or keep it and say why; and change the two spec lines to name `orderAnyAvailable`. **Resolution:** Fixed 2026-10-08 on chore/cleanup-before-9: choose-any-available.ts deleted; its nine tests moved, under the same names, into order-any-available.test.ts, which checks the first choice through a local firstChoice helper (orderAnyAvailable(...)[0] ?? null). 11/11 pass. The 05d archive keeps its lines as written; it already records F-94. Closed 2026-10-08 by independent review of chore/cleanup-before-9 (392cc7d..394cca2; lenses: quality, security, performance, tests): choose-any-available.ts and its test are gone and git grep finds no chooseAnyAvailable outside the ledger and the archives; the nine cases sit under the same names in order-any-available.test.ts through firstChoice, which is the deleted function's whole body (orderAnyAvailable(...)[0] ?? null), and current-feature.md holds no spec naming it. order-any-available.test.ts 11/11 in all four backend runs.

9/F-95 [P3] closed - When the no-wait test fails, its cleanup hangs on the held Google answer and leaves its business in the dev database

File: backend/lib/calendar/write-booking-event.test.ts:191-219 (cleanup: :166-170) **Found:** 2026-10-02 by the second final independent review of feature 5d (scope: 12a21d6..3cec4ae; lenses: quality, security, performance, tests) **Why it matters:** The test releases Google's held answer only after its first assertion. If bookTime ever waits for Google again (the regression it guards), the test times out before `answerGoogle()` runs, the background write never settles, and `afterAll` hangs on `bookingEventWrites.settled()` until the hook times out, so the delete never runs. Reproduced in this review by putting the wait back: the run reported the failure, then the throwaway business `test-event-no-wait-<tag>` stayed in `scheduleads_dev` with its booking. The file's header promises every business is removed after, and the seed and the other files rely on that. **Suggested fix:** Release Google in a `finally` around the test body (or in `afterEach`), so a failure still lets the write settle and the cleanup run. **Resolution:** Fixed 2026-10-08 on chore/cleanup-before-9: the held Google answer is file-level and every afterEach lets it go, so a test that fails or times out while Google is held (including inside bookTime, the regression it guards) still lets the write end. Note: since 8a the write is a job and afterAll no longer waits on it; a probe failing the test before Google answered left no business behind with or without the change, so the hang described is no longer reachable that way; the release keeps a failed test from leaving a pending Google answer. Closed 2026-10-08 by independent review of chore/cleanup-before-9 (392cc7d..394cca2; lenses: quality, security, performance, tests): answerGoogle is file level and afterEach calls it before unstubbing. Probe: the no-wait test made to fail after Google was asked and before it answered (its second toBeNull changed to expect "PROBE-FAIL"), once with the fix and once with the afterEach release removed; both runs reported only that failure and ended in about 10 s, and a read-only query afterwards found no test-event-% business in scheduleads_dev. So the hang is no longer reachable (since 8a afterAll waits on no write), as the Resolution says, and the release keeps the held write from dangling. File restored, sha256 64e0c61c... unchanged. Closed rather than invalid: the finding was right when raised and was reproduced then.

9/F-134 [P3] closed - The cancel test that checks log lines for the customer's details reads them before any are written

File: backend/lib/booking/cancel-booking.test.ts:299-311 **Found:** 2026-10-03 by /audit independent (scope: current, 32114fc..14772a1; lens: tests) **Why it matters:** The test "nothing in the timeline entry or a log line carries the customer's details" collects console lines, awaits `cancelBooking`, and builds `everything` from `lines.join` straight away (:307). Every line a cancel can write comes from the work it starts without waiting (the Google removal and the cancellation emails, which need a database round trip first), so none has been written when the lines are read, and the log half of the test passes whatever those lines say. The same lines are checked in their own files (remove-booking-event.test.ts:236-239, send-cancellation-emails.test.ts:320-326), so nothing is unguarded today; the risk is a test whose name promises more than it proves. **Suggested fix:** Await `bookingEventRemovals.settled()` and `bookingCancellationEmails.settled()` before reading `lines`, or drop "a log line" from the test's name and leave the log checks to the two files that already make them. **Resolution:** Fixed 2026-10-08 on chore/cleanup-before-9: since 8a the cancel's emails and event removal run as jobs, so the test now works the due jobs before reading the lines and asserts some were written. Proved: without the `workDueJobs` line the test fails (no lines read). Closed 2026-10-08 by independent review of chore/cleanup-before-9 (392cc7d..394cca2; lenses: quality, security, performance, tests): the test works the due jobs before reading. Probe: printing what it reads shows five lines, all written by the jobs (two "[text] ... not sent, the booking was cancelled", two "[email] ... nothing sent", one "[text] ... worker_removed not sent"), none with Jane's details; with the `workDueJobs` line removed it fails on `expect(lines.length).toBeGreaterThan(0)` (expected 0 to be greater than 0). File restored, sha256 14c75e71... unchanged. The cancellation email's and event removal's failure lines stay pinned in their own files, as the finding noted.

9/F-145 [P3] closed - The move's check is a copy of bookTime's: the free check, the room rule, the day's counts and `namesOf`

File: backend/lib/booking/move-booking.ts:109-176,252-259 (original: backend/lib/booking/book-time.ts:119-126,196-277) **Found:** 2026-10-03 by independent review of step 7b.2 (scope: 7059d79..1731845; lenses: quality, security, performance, tests) **Why it matters:** About 70 lines repeat bookTime's customer path with two `bookingId` filters added, and `namesOf` is copied verbatim. A later change to how a start is checked or ordered (F-62's cost, the room rule, a new any-available rule) has to be made in both files, and one being missed is exactly the drift the spec's "changing the time is booking again" (decision 10) forbids. **Suggested fix:** Extract one function in `lib/booking/` that, given the service, the start and an optional booking to leave out, answers the ordered choices or the refusal; bookTime's customer path and moveBooking both call it. Can wait for the owner's move (features 11 and 12b), which will be a third caller. **Resolution:** Partly fixed 2026-10-03: the name lookup is one shared helper (find-resource-names.ts) used by bookTime and moveBooking. The free check's copy stays, carried for when the owner's move (features 11 and 12b) gives a third caller; noted in the spec. Fixed 2026-10-08 on chore/cleanup-before-9: the rest of the copy is now one function, backend/lib/booking/find-booking-

choices.ts (findBookingChoices): the free check per candidate (the free times for a customer, real busy time and Google for the owner), the room rule over the span, and the order to try, answering the choices or "time_taken"/"unavailable". bookTime calls it with byOwner for a manual booking; moveBooking with movingBooking, whose own held rows and day count it leaves out. Both files lost their copies (about 150 lines). Typecheck, both builds, format check, and backend 789 three runs in a row pass. The owner's move (features 11 and 12b) is the third caller it was waiting for. Closed 2026-10-08 by independent review of chore/cleanup-before-9 (392cc7d..394cca2; lenses: quality, security, performance, tests): find-booking-choices.ts is the one check. Read line by line against main's two copies, both callers pass what they used before: byOwner is source === "manual" for bookTime and false for the move; movingBooking is undefined for bookTime, so notTheMovingBooking keeps every row (a commitment's bookingId is string or null, never undefined); the move's ignoreBooking.id is its bookingId, which its old room and day-count filters used; the standby read, the side-by-side calendar reads and the refusal reasons are unchanged, and bookTime still maps "unavailable" and "time_taken" to the same refusals. Seven mutants of the new file, each restored byte for byte (sha256 61c5ffa1... after each), each failing named tests: the room's own-row filter dropped ("a move in a room takes the room again at the new time"); the day count's filter dropped ("any available does not count the booking being moved"); standby read for the owner too ("a room on standby that date is not chosen for a customer, but the owner may use it"); the owner path skipped (three owner-made booking tests); ignoreBooking not passed (four move tests); a picked unreadable calendar answered time_taken (three tests, in bookTime, the booking route and the move route); any available with every calendar unreadable answered time_taken (two). The owner path does not yet honour movingBooking, which no caller reaches today: F-253.

9/F-146 [P3] closed - Two move tests promise more than they check: "the log" spies only console.log, and "another business's booking" only sends a foreign person id

File: backend/routes/public-booking-move-routes.test.ts:306-330 **Found:** 2026-10-03 by independent review of step 7b.2 (scope: 7059d79..1731845; lenses: quality, security, performance, tests) **Why it matters:** The move path writes no `console.log`; its only log lines are `console.warn` (calendar), which `beforeAll` silences and no test reads, so the log half of the privacy test passes whatever those lines say (the same shape as F-134). "another business's booking is never touched" is refused by the person check at move-booking.ts:93 before any write, so it does not show that the writes stay inside the booking's business. Both behaviours are right by reading; the names claim coverage the tests do not give.

Suggested fix: Collect `console.warn` and `console.error` too (with a connected calendar that fails, so a warn line is actually written), and either rename the tenant test to what it checks or add a case that moves one business's booking and asserts the other business's rows unchanged. **Resolution:** Fixed 2026-10-03: the privacy test also reads console.warn and a refusal's body; the tenant test also makes a real move of ours to a time the other business has booked and checks their booking and rows are untouched. Not closed by independent review of step 7b.3 (2026-10-04): the tenant half holds; the log half still reads a `console.warn` that nothing writes (no calendar is connected in that test, and it reads before `bookingEventMoves.settled()`). A real follow failure's line is now checked for the name and address in move-booking-event.test.ts "a Google error keeps the move and logs one line". Not closed by independent

review of feature 7b (scope: a55c8ee..5db122e): the route test is unchanged and still reads `console.warn` and `console.log` before `bookingMoveEmails.settled()` (public-booking-move-routes.test.ts:369-388), so its log half checks only what happens to have run. The move's real log lines are now pinned elsewhere (move-booking-event.test.ts:365, send-move-emails.test.ts:328 and :355), so the remaining gap is the test's name, not the coverage. Not closed by independent review of feature 7b (scope: a55c8ee..6c1fa5d): public-booking-move-routes.test.ts:416-435 is unchanged since that pass. Not closed by independent review of feature 7b (scope: a55c8ee..851fadb): the test file is unchanged since 6c1fa5d. Not closed by independent review of chore/cleanup-before-9 (392cc7d..394cca2; lenses: quality, security, performance, tests): the file changed only in two imports. A probe printing what the privacy test reads found `log.mock.calls` and `warn.mock.calls` both empty (`[[[]], [[]]]`): the move's emails and Google follow are jobs, and the test reads before any `workDueJobs`. File restored, sha256 e9d9ac19... unchanged. The same repair as F-134 (work the due jobs, then assert some lines were read) would close it. Fixed again 2026-10-08 on chore/cleanup-before-9, as F-134 was: the privacy test works the due jobs (the move's emails and event, which write the lines) before reading `console.log` and `console.warn`, and asserts some lines were read. Proved: without the `workDueJobs` line it fails "expected 0 to be greater than 0" (file restored, same sha256). Closed 2026-10-08 by independent review of chore/cleanup-before-9 (9610596..177f12e; lenses: quality, security, performance, tests): public-booking-move-routes.test.ts:418 works the due jobs before the log is read and :420 asserts some lines were read. Three probes with only this test selected (-t, 17 skipped), each restored byte for byte (test sha256 f1e23814... and send-move-emails.ts sha256 70aeda26... after each): the `workDueJobs` line removed fails "expected 0 to be greater than 0"; the customer's name added to the move email job's nothing-sent line (send-move-emails.ts:37) fails at :431 on `not.toContain`; the same leak with both new lines removed (the old shape) passes, so the repair is what makes the log half bite.

9/F-161 [P3] closed - The week bar spills out of the card at 320px

File: frontend/components/booking-page/change-time-panel.tsx:243-267 **Found:** 2026-10-04 by independent review of step 7b.5 (scope: 79acbd8..48d743c; lenses: quality, security, performance, tests, accessibility) **Why it matters:** Three `whitespace-nowrap` items in a 232px content box. Measured at 320px wide: the "Later >" button ends at x=301 while the card ends at x=289, so it crosses the card's border (no page scroll). At 375px it fits, which is the width the step was checked at; small phones still exist. **Suggested fix:** Let the range label wrap or shrink (`min-w-0`, `text-center`), or shorten the buttons to icons with `aria-label`s below a breakpoint. **Resolution:** Fixed 2026-10-04: the week reads "Oct 4 to 10" (both months only across two) with narrower buttons; at 320px Later ends at x=260 inside the card at 289. Not closed by independent review of feature 7b (scope: a55c8ee..5db122e): this reviewer started no dev server and could not measure; the code matches the repair (weekName at change-time-panel.tsx:49-56, `px-2` buttons), but the measurement above was for a same-month week, and a week across two months ("Oct 25 to Nov 1") is about four characters wider. Look at one such week at 320px before closing. Not closed by independent review of feature 7b (scope: a55c8ee..6c1fa5d): no dev server was started, so still unmeasured. Estimated from the code only: the buttons went from `px-3` to `px-2` (16px narrower in all) and a cross-month label is about one character shorter than the "Oct 11 to Oct 17" first measured 12px past the card's border, so Later likely ends inside the border but in the card's padding. A

measurement is still needed to close it. Not closed by independent review of feature 7b (scope: a55c8ee..851fadb): no dev server was started; the week bar's markup (change-time-panel.tsx:285-315) is unchanged by the last repair, so still unmeasured across two months at 320px. Not closed by independent review of chore/cleanup-before-9 (392cc7d..394cca2; lenses: quality, security, performance, tests): no dev server was started (outside this review's limits). The panel changed only in its date helper imports; the week bar's markup is untouched, so a week across two months at 320px is still unmeasured. Measured 2026-10-08 in the browser at 320px on a real booking page (clinic-dev, a throwaway booking), the week across two months "Oct 29 to Nov 4": before this change Later ended at x=289, inside the card border (x=304) but 14px past the content edge (275), the bar 244 wide in 230, so Later stood out past the time grid. Fixed the same day on chore/cleanup-before-9 with the suggested fix: the week label is min-w-0 and text-center and may wrap. After: the bar fits exactly (scrollWidth 230 = clientWidth 230), Later ends at 275, the label wraps to two lines; at 375px it stays on one line (285 = 285). No console errors. Frontend build and lint pass. Closed 2026-10-08 by independent re-review of chore/cleanup-before-9 (e041ce1; lenses: quality, accessibility), measured in the built-in browser on the same throwaway clinic-dev booking: at 320x700 the week across two months "Oct 29 to Nov 4" has the bar at scrollWidth 230 = clientWidth 230 (x=45 to 275), and Later ends at x=275, the bar's own right edge, 28px inside the card's right border (inner 303, outer 304); page scrollWidth 320. The label wraps to two lines as "Oct 29 to / Nov 4", each line centred, its centre at x=163.87 against the middle of the gap between the buttons at 163.87. The same-month week "Oct 8 to 14" also fits (230 = 230, Later at 275) on one line. At 375x700 the cross-month label stays on one line (bar 285 = 285, Later at 330, border at 358). Keyboard: Shift+Tab from Later lands on Earlier, Enter moves a week and focus stays on the pressed button, Tab goes straight to Later (the label takes no tab stop), and the focus ring sits inside the card. No console errors. Only a note: the two-line label makes the bar 40px tall against 36px for one line, so the times below move down 4px when a week crosses two months; harmless. The repair commit had also dropped the blank line between this entry and F-170; restored here.

9/F-170 [P3] closed - The move-times privacy test reads a 503 refusal, not the times, when its file runs in order

File: backend/routes/public-booking-move-times-routes.test.ts:339-347 (connections saved at :244,253-254,264; fetch reset at :178-182) **Found:** 2026-10-04 by independent review of feature 7b (scope: a55c8ee..6c1fa5d; lenses: quality, security, performance, tests) **Why it matters:** The file shares one clinic, and the Google tests save calendar connections for Ana and Mei that are never removed. `afterEach` resets `fetch` to throw, and "nothing in the answer carries the customer's details" fakes no Google answer, so with both people connected and both calendars unreadable, any available throws `CalendarUnavailableError` and the route answers 503 `unavailable`. The test then checks that a refusal's body has no customer details and never asserts the status, so the 200 answer it is named for (`people` , `startTimes` , `lastDate`) is only checked when the test runs alone. The same order dependence F-135 removed from the Google tests, in the one test that does not set Google up. **Suggested fix:** Call `fakeGoogleBusy([])` at the start of the test and assert `response.status` is 200 before reading the body. **Resolution:** Fixed 2026-10-08 on chore/cleanup-before-9: the test fakes Google with no busy

times and asserts status 200 before reading the body. Proved: without fakeGoogleBusy([]) the file in order answers 503 and the test now fails. Closed 2026-10-08 by independent review of chore/cleanup-before-9 (392cc7d..394cca2; lenses: quality, security, performance, tests): the test fakes Google with no busy times and asserts 200 before reading the body. Probe: with the fakeGoogleBusy([]) line removed, the file run in order fails that test with "expected 503 to be 200". File restored, sha256 df6284f6... unchanged.

9/F-171 [P3] closed - AGENTS.md's branch example still has no build-plan number, which the skills now require

File: AGENTS.md:211-212 (skills: .claude/skills/feature/SKILL.md:141-147, .claude/skills/implement/SKILL.md:50-52) **Found:** 2026-10-04 by independent review of feature 7b (scope: a55c8ee..6c1fa5d; lenses: quality, security, performance, tests) **Why it matters:** The delta changes `/feature`, `/implement`, `/autopilot`, `/continuous` and the spec template to name feature branches `feature/NN-<name>` (this branch is `feature/07b-reschedule`), matching the workspace rules. AGENTS.md, the file every session loads, still gives `feature/booking-links-resources-and-availability-rules` as the example of a feature branch, so the project's two instruction sources now show different shapes for the same name. **Suggested fix:** Change the example to the numbered form, for example `feature/07b-reschedule`. **Resolution:** Fixed 2026-10-08 on chore/cleanup-before-9: the example is now `feature/08c-the-worker-s-text`, with "its build-plan number first". Closed 2026-10-08 by independent review of chore/cleanup-before-9 (392cc7d..394cca2; lenses: quality, security, performance, tests): AGENTS.md:228 gives `feature/08c-the-worker-s-text`, the numbered shape the skills require and the name of feature 8c's archive (blueprint/history/features/08c-the-worker-s-text.md); the one other branch name in AGENTS.md (:247) is feature 1's real, kept branch.

9/F-172 [P3] closed - The change-time panel keeps its own untested copies of the backend's date helpers

File: frontend/components/booking-page/change-time-panel.tsx:26-38 (backend/lib/local-time/local-date.ts:6, backend/lib/local-time/add-days.ts:3) **Found:** 2026-10-04 by independent review of feature 7b (scope: a55c8ee..851fadb; lenses: quality, security, performance, tests) **Why it matters:** `dateIn` (a moment's YYYY-MM-DD in the business's zone) and `addDays` (calendar arithmetic on YYYY-MM-DD) do the same jobs as the backend's `localDate` and `addDays`, which are tested; the panel's copies are not, because the frontend has no test runner. Every week the panel asks for, the day groups, "today" and the last-week check rest on them. This delta already met the cost of a hand copy once: F-164, a third `telHref` that lost one backslash and dialled nothing, repaired by moving the one helper into `packages/shared/helpers/` with its test. The coding standards put a helper both apps need in `packages/shared`. The copies agree today (both work in UTC on YYYY-MM-DD); nothing keeps them agreeing. **Suggested fix:** Move `addDays` and a `localDate` that builds from `formatToParts` into `packages/shared/helpers/` with one test each, and import them in both the backend and the panel; or leave it for the dashboard (features 11 and 12b), which will need the same dates, and note it there. **Resolution:** Fixed 2026-10-08 on chore/cleanup-before-9: `localDate` and `addDays` now live once in `packages/shared/helpers` (exported as `@scheduleads-app/shared/local-date` and `/add-days`), each with

its tests (localDate: late evening, a month and year turning at midnight, the two 1:30s of a clock change; addDays: months, a year back, a leap day). The backend's copies in lib/local-time are deleted and its 15 files import the shared ones; the change-time panel drops dateIn and its addDays for the same imports. localDate is now built from the date's parts (en-CA formatToParts, one formatter per zone) instead of clockAsUtc, which stays in the backend for localTimeToMoment. Shared 159, backend 789 three runs in a row (two tests moved to shared), both builds and the format check pass. Not checked in a browser. Closed 2026-10-08 by independent review of chore/cleanup-before-9 (392cc7d..394cca2; lenses: quality, security, performance, tests): packages/shared/helpers/local-date.ts and add-days.ts are exported as ./local-date and ./add-days; the backend's lib/local-time copies are gone (git grep finds no import of them) and the 15 backend files and the change-time panel import the shared ones. Equivalence probe against main's clockAsUtc localDate: every 15 minutes from 2025-01-01 to 2029-01-01 in all 418 zones Node 26.7.0 knows (ICU 78.3, tz 2026c), plus 1.4 million random moments in seven odd-offset zones (Chatham, Lord Howe, Kolkata, St Johns, Kiritimati, Edmonton, New York): 60,027,008 moments, no difference. addDays against both old versions (the backend's Date.UTC one and the panel's Date.parse one), every day 2024 to 2030 with eleven offsets from -400 to +366: no difference. The panel's old dateIn was the same formatToParts code, so the page's dates do not change. Mutants: localDate formatted in UTC fails two of its three tests; addDays a day off in February fails the leap day test. Both restored, sha256 unchanged.

9/F-193 [P3] closed - A deploy's stop exits without waiting for the requests in flight, so a booking being made at that moment is cut

File: backend/server.ts:39-48 **Found:** 2026-10-05 by /audit independent current (scope: 779512a..dd65fe3; lenses: quality, security, performance, tests) **Why it matters:** On SIGTERM, `stop()` calls `server.close()` without waiting for it, awaits only `runner.stop()`, then calls `process.exit(0)`. With no job in hand the runner stops in milliseconds, and `process.exit` ends the process whatever sockets are still open, so a request the old API is still answering is dropped: a booking mid-transaction rolls back and Jane sees an error instead of her booking, though the deploy notes (F-176) are about to give the old API 30 seconds it could use to finish it. `server.close()` is the half that already does this (it refuses new connections and calls back once the open ones end). Not a regression: before this range the API had no SIGTERM handler and died at once. The comment above it says "no new requests", which is true; it is the requests already accepted that nothing waits for. **Suggested fix:** Wait for both before exiting, for example `await Promise.all([new Promise((resolve) => server.close(resolve)), runner?.stop(signal)])`, optionally bounded a little under the draining time so the API exits itself before Railway's SIGKILL. **Resolution:** Fixed 2026-10-08 on chore/cleanup-before-9: the stop is now backend/lib/server/stop-gracefully.ts (stopGracefully), which waits for both server.close (refuses new connections, calls back once the open requests end) and runner.stop, bounded; server.ts calls it with a 25 second limit, under the 30 seconds F-176 asks Railway to allow, logs when the limit is hit, then exits 0. Tests (lib/server/stop-gracefully.test.ts, a real HTTP server with a held request): a request in flight finishes before the stop resolves; a new request is refused once stopping; the runner gets the signal and is waited for; a request that never ends lets it stop at the limit. Proved: not waiting for server.close fails two of them

(file restored, cmp identical). The built API starts and answers /health 200. Not exercised: a real SIGTERM, which Windows cannot send; Railway's draining is still F-176. Re-reviewed 2026-10-08 by independent review of chore/cleanup-before-9 (daa79f7..4fe2e61; lenses: quality, security, performance, tests); stays fixed, for F-254. What holds: @hono/node-server 2.1.1's serve (dist/index.mjs:1285-1311) returns a plain node:http Server, so server.close is Node's own; on Node 26.7.0 a scratch probe showed close() drops an idle keep-alive connection at once (2 open connections to 1) and calls back only after the open request ends, so the original cut is gone and nothing hangs on an idle connection. The limit's timer is cleared in finally on both paths; a stop that times out leaves server.close and runner.stop pending, which process.exit(0) ends. exitWhenRunnerStops still holds: stopping is set before runner.stop, so the runner's promise settling during the stop returns without exit(1). Exit 0 at the limit is right given F-179: the stop was asked for, and a non-zero exit is what Railway's On Failure policy restarts and counts. Each test bites (stop-gracefully.ts sha256 14691a8ab487d4fa... before and after, cmp identical): close not awaited fails tests 1 and 4; close never called fails 1, 2 and 4; runner.stop() without the signal fails 3; runner.stop not awaited fails 3; no race with the limit fails 4. A temporary probe test, deleted after, showed that a runner.stop that rejects (graphile-worker 0.18.0 runner.js:112, "Runner is already stopped", only while the runner is already stopping itself) makes stopGracefully reject at once with the request still open, so server.ts's stop becomes an unhandled rejection and exits 1; not recorded, since that same window ends in exitWhenRunnerStops' exit(1) anyway. What does not hold is F-254: a connection busy when the stop begins stays open after its answer, and the API keeps answering new requests on it. Also, server.ts:40-41 says Railway "is set to allow" 30 seconds; F-176 is still open, so until that setting lands Railway's 0 to 3 second default is the real bound. Backend tests 793 passed twice, build and format:check pass, the built API answered /health 200 and 3401 was free after. Closed 2026-10-08 by independent review of chore/cleanup-before-9 (b0806c2..c5cf49b; lenses: quality, security, performance, tests): what held it, F-254's kept connection, is gone in the code (the evidence is in F-254, which stays fixed only for its test, F-255). stop-gracefully.ts still waits for server.close and runner.stop raced against the limit, with the timer and the new sweep cleared in finally, and server.ts still exits 0 after it. F-193's own tests still bite (stop-gracefully.ts sha256 22436fdf8c9c8d6f... before and after): a sweep that calls closeAllConnections, cutting what is in flight, fails tests 1 and 4. Backend 794 passed twice, build and format:check pass, the built API answered /health 200 and 3401 was free after. server.ts:43-44 still says Railway "is set to allow" 30 seconds while F-176 is open, as the last review noted.

9/F-194 [P3] closed - installJobTables ships in the API but only one test calls it, and its comment says the tests use it before their first job, which they do not

File: backend/lib/jobs/install-job-tables.ts:1-12 (its one caller: backend/lib/jobs/job-runner.test.ts:55; the setup's own copy: backend/vitest.setup.ts:49-52) **Found:** 2026-10-05 by /audit independent current (scope: 779512a..dd65fe3; lenses: quality, security, performance, tests) **Why it matters:** The comment says the API's runner installs the tables itself and that "tests call it before adding their first job". The second half is not what happens: `vitest.setup.ts` runs graphile-worker's `runMigrations` on the worker's schema itself before every file, and every `run` and `runOnce` migrates again on its own (graphile-worker 0.18.0 dist/lib.js:323-326, reached from dist/runner.js:24-26 and 43-45). The one

caller, job-runner.test.ts:55, migrates a schema the setup already migrated. So the file is compiled into the API's dist/ with no production caller, and a reader looking for where the tables come from is pointed at the wrong place. The spec's 8a.1 step says the tables are installed by the API's runner and by the test helper; both already do it through the library. **Suggested fix:** Delete install-job-tables.ts and its one call in job-runner.test.ts (the setup and the library already install the tables), and leave the setup's comment as the place that says the tests' schema is migrated there. **Resolution:** Fixed 2026-10-08 on chore/cleanup-before-9: install-job-tables.ts deleted with its one call in job-runner.test.ts; vitest.setup.ts says beside its runMigrations that it builds each worker schema's runner tables, which db:migrate never does. job-runner.test.ts 8/8 and the backend build pass. Closed 2026-10-08 by independent review of chore/cleanup-before-9 (392cc7d..394cca2; lenses: quality, security, performance, tests): install-job-tables.ts and its call are gone and git grep finds no reference. vitest.setup.ts runs runMigrations on the worker's own schema (graphile_worker_test_<pool>, the one job-schema.ts reads through JOBS_SCHEMA) before every file, and the API's runner migrates through run() in start-job-runner.ts; job-runner.test.ts passes in all four backend runs.

9/F-239 [P3] closed - The record says identical replies are never taken for one and both ways are claimed, which the built check does not promise

File: blueprint/context/current-feature.md:241-243; packages/shared/db/text-tables/text-reply-table.ts:2-4; backend/routes/public-text-routes.test.ts:359 **Found:** 2026-10-07 by the independent review of feature 8b (scope: current, 3b47c1c..43d308e; lens: quality) **Why it matters:** The spec's 8b.4 says the lost-answer check counts "from when the reply was recorded, so two replies in the same words are never taken for one". That holds when the identical reply was passed on before this one was recorded (the code comment at pass-on-reply.ts:144-146 says exactly that). It does not hold after: Jane texts "Yes" (reply A) and "Yes" again (reply B) a few seconds later; A's send times out without reaching Twilio, B is passed on, and A's retry a minute later finds B's text, same words, after A's record, and sends nothing, so the business's phone hears one "Yes" (the email still carries both). Twilio takes no idempotency key, so this residue is inherent, but /complete archives the spec as the record. The table's header also says "A run claims each way (the text, the email)": only the text is claimed; the email relies on Resend's idempotency key. And the test named "a retry counts only from its own claim" now pins a check that counts from the reply's record (F-237). **Suggested fix:** Say "an identical reply passed on before this one was recorded is never taken for it" in the spec, say in the table comment that the text is claimed and the email keyed, and rename the test to "counts from the reply's record". No code change. **Resolution:** Fixed 2026-10-08 on chore/cleanup-before-9 (wording only): text-reply-table.ts says the text is claimed and the email kept to one by Resend's idempotency key, and that a reply in the same words passed on before this one was recorded is never taken for it; the test is renamed "a retry counts from the reply's record: ..."; the 8b archive line now says the same, marked as corrected by F-239. Closed 2026-10-08 by independent review of chore/cleanup-before-9 (392cc7d..394cca2; lenses: quality, security, performance, tests): the table comment says the text is claimed and the email kept to one by Resend's key, which pass-on-reply.ts does (claimText at :35 claims only the text; the email goes with idempotencyKey text-reply/<sid> at :107), and that only a same-words reply passed on before this one was recorded is never taken for it, which is

what findSentText since record.createdAt (:147-149) gives. The test name and the 8b archive line say the same. Wording only, no behaviour changed.

9/F-252 [P3] accepted - A "new booking" or "moved" still in doubt keeps counting after the person was told the booking is off, so a booking back and away again before its texts run sends a second "off your day"

File: backend/lib/text/send-worker-text.ts:159-164; backend/lib/jobs/has-worker-text-in-doubt.ts:13-25
Found: 2026-10-08 by independent review (scope: current, ddca0e1..2ad0463; lenses: quality, security, performance, tests) **Why it matters:** F-243's repair says nobody gets a second "off your day" for a booking they already think is gone: `believes` needs an added or moved entry newer than the latest off entry. But the in-doubt branch is ORed in with no such comparison, and `hasWorkerTextInDoubt` counts any try of an added or moved job for that booking and person, including one that gave up long ago (F-248, accepted, so it stays in the runner's table for good). Path, read from the code: Pedro's "new booking" gives up after its tenth try (for example a day with the production keys missing, which `send-text.ts` retries); later the booking moves to Maria and Pedro's "off your day" goes (in doubt, so he believes), recorded; then it moves back to Pedro and away again before those jobs run. The move back's added finds the booking another person's and sends nothing; the second taken off finds no newer added entry but the same given-up job, so `believes` is true and Pedro gets a second "off your day". The same happens inside the seconds a lost answer waits for its retry. The code matches decision 5's amendment as written (an OR of the two conditions); it is the amendment's F-243 promise that does not hold. Rare and harmless in effect (a repeated "it is off", never a missed one), so P3. **Suggested fix:** Decide it in decision 5. Either list it under "Accepted as is", or let a doubtful try count only when it came after the person's latest "off your day" entry (compare the job's last try time, `_private_jobs updated_at`, with that entry's `occurredAt`; comparing move numbers is not enough, since an added job can try after an off recorded at a higher number). Add a test of the same name if it is fixed. **Resolution:** Accepted by Frank 2026-10-08 as a known limit: it needs a lost Twilio answer (or a text that gave up) and the booking back on the person and off again before its texts run; the worst case is a repeated "off your day", never a missed one, which keeps his rule "when unsure, tell him". No code change.

9/F-253 [P3] closed - findBookingChoices takes a moving booking on the owner path, but never leaves it out of the person's busy time or Google there

File: backend/lib/booking/find-booking-choices.ts:74-77 (its promise: :4-5; the customer path's handling: backend/lib/scheduling/find-free-times.ts:117,138-143) **Found:** 2026-10-08 by independent review of chore/cleanup-before-9 (scope: 392cc7d..394cca2; lenses: quality, security, performance, tests) **Why it matters:** The header promises "A booking being moved never stands in its own way", and the input type accepts `byOwner: true` with `movingBooking`, the combination F-145's Resolution names as the third caller (the owner's move, features 11 and 12b). The customer path honours it through `findFreeTimes'` `ignoreBooking`, and the rooms and the day count filter it. The owner path does not: it answers busy on `findCommitments(organizationId, [id], from, to)` and on the person's Google busy times with neither filter, so an owner moving Pedro's 9:00 to 9:30 with Pedro would find Pedro's own

held rows (and his own Google event) in the span and be refused `time_taken`. No caller passes both today (`bookTime`: `byOwner` without `movingBooking`; `moveBooking`: `movingBooking` with `byOwner` false), so nothing behaves wrongly now; the risk is the next caller trusting the header. **Suggested fix:** When the owner's move is built, filter the owner path's commitments with `notTheMovingBooking` and cross the moving booking's own time off that person's Google busy, as `findFreeTimes` does, with a test of the owner moving into a time overlapping the old one; or until then, type the input so `movingBooking` only goes with `byOwner` false. **Resolution:** Fixed 2026-10-08 on chore/cleanup-before-9 (the second suggestion, until the owner's move): the input type is a union, so `movingBooking` goes only with `byOwner` false; the header says the owner's check does not take a moving booking yet (features 11 and 12b). Proved: a temporary call with `byOwner` true and `movingBooking` fails tsc (TS2345), removed after. Typecheck, build, format and backend 789 three runs pass. Closed 2026-10-08 by independent review of chore/cleanup-before-9 (9610596..177f12e; lenses: quality, security, performance, tests): `find-booking-choices.ts:36-39` is a union on `byOwner`, and the header (:4-5) says the owner's check takes no moving booking yet. A temporary `backend/lib/booking/zz-probe-f253.ts`, deleted after, run through tsc --noEmit: `byOwner` true with `movingBooking` fails TS2345 (`movingBooking` not assignable to undefined), and `byOwner` typed boolean with `movingBooking` fails TS2345 (boolean not assignable to false), while `byOwner` false with `movingBooking` and `byOwner` boolean without it both pass. Both callers typecheck in the backend build: `book-time.ts:189` passes `byOwner`: `source === "manual"` with no `movingBooking`, `move-booking.ts:120-121` `byOwner` false with `movingBooking`; no test file passes `movingBooking` (tests sit outside tsc). The owner's move (features 11 and 12b) will have to widen the union and honour the moving booking on the owner path.

9/F-254 [P3] closed - A connection busy when the stop begins is kept alive after its answer, so the API keeps taking new requests on it and a client that reuses it holds the stop to the 25 second limit

File: `backend/lib/server/stop-gracefully.ts:1-2,20` (the claim's test: `backend/lib/server/stop-gracefully.test.ts:50`) **Found:** 2026-10-08 by independent review of chore/cleanup-before-9 (scope: `daa79f7..4fe2e61`; lenses: quality, security, performance, tests) **Why it matters:** The header says new requests are refused at once. Node's `server.close` (Node 26.7.0, on the `http Server @hono/node-server 2.1.1` returns) refuses new connections and drops the idle ones when it is called, but a connection that is busy then is not marked to close: its answer goes out with `Connection: keep-alive` and the socket stays open. A scratch probe (an `http.Agent` with `keepAlive`, one held request, `close()`, the held answer, then a request a second on the same agent) had the closing server answer 30 more requests on that socket over 30 seconds, and its close callback fired only 5 seconds (`keepAliveTimeout`) after the last one. So the stop always waits out the keep-alive linger after the last answer (tests 1 and 2 take about 3 seconds each because `fetch`'s client hangs up its idle socket then, not because the API finished), and a client that keeps reusing the connection (whether Railway's proxy does so with the old deployment after the switch is not known) feeds it requests until the 25 second limit, when `process.exit` cuts whichever one is running: F-193's cut booking, for a request accepted after the stop began. Test 2 proves only that a new connection is refused, which its name does not say. **Suggested fix:** While stopping, end each kept connection once its

answer is out, for example `server.closeIdleConnections()` on a short unref'd interval, cleared when close calls back or at the limit. The same probe with a 100 ms sweep had close call back the moment the held answer went out, and the next request was refused (ECONNREFUSED). Add a test that sends a second request on the held request's connection (an `http.Agent` with `keepAlive`) and expects it refused and the stop finished promptly, and make the header and test 2 say what they prove. **Resolution:** Fixed 2026-10-08 on chore/cleanup-before-9: while stopping, `stopGracefully` sweeps `server.closeIdleConnections()` every 100 ms (unref, cleared on both endings), so a connection busy when the stop began is closed once its answer goes out; the header says so. Test "a connection kept open is closed after its answer, so nothing more comes in on it" (one keep-alive connection: the held answer arrives, the stop finishes within a second, a second request on the same agent fails); test 2 renamed "a new connection is refused once stopping has begun". Proved: a sweep that does nothing fails the new test (5.1 s; file restored, cmp identical). The file now runs in 1.3 s, not the clients' idle timeouts. `server.ts` types `serve`'s result as `node:http Server` (it makes HTTP/2 or HTTPS only when asked). Build, format, backend 794 twice pass; the built API starts and `/health` answers 200. Re-reviewed 2026-10-08 by independent review of chore/cleanup-before-9 (b0806c2..c5cf49b; lenses: quality, security, performance, tests); stays fixed, for F-255. What holds is the code: `@hono/node-server 2.1.1` (one copy, in the root `node_modules`) builds its server with `(options.createServer || createServer)` from `node:http` (dist/index.mjs:1292) and `server.ts` passes no `createServer`, so the `as Server` is true; the cast is needed because `serve`'s `ServerType` also covers `Http2Server`, which has no `closeIdleConnections`. On Node 26.7.0 `closeIdleConnections` destroys only connections the parser lists idle and skips any whose response is not finished, and scratch probes against the built dist showed it cuts nothing in flight: a 64 MB answer to a slow reader arrived whole with the sweep running (12.6 s), and tests 1 and 4 hold their requests across several sweeps. A client reusing one connection back to back (handlers of 5, 20 and 200 ms, no gap, 11 runs) was caught idle and the stop finished within 0 to 420 ms with every answer whole; F-254's probe shape (one request a second) ends at the first sweep. The sweep is unref'd (`hasRef` false) and cleared in finally on all three endings: after finished, `timed_out` (4 sweeps in 350 ms) and a rejecting `runner.stop`, `process.getActiveResourcesInfo()` listed no `Timeout` and no sweep ran in the next 400 ms. A 100 ms sweep for at most 25 seconds costs nothing measurable; nothing here touches input or secrets. Each probe restored `stop-gracefully.ts` (sha256 22436fdf8c9c8d6f... before and after): a sweep that does nothing fails the new test (5.1 s); a sweep that closes every connection fails tests 1 and 4. What does not hold is the new test: a sweep that runs once (`setTimeout`) passes all five, F-255. Closed 2026-10-08 by independent review of chore/cleanup-before-9 (c5cf49b..d6e8bd1; lenses: quality, security, performance, tests): what held it, F-255, is closed. `stop-gracefully.ts` and `server.ts` are unchanged since c5cf49b (git diff empty), so the evidence above for the code still stands, and the sweep is now proved by its own test: with `stop-gracefully.ts` restored byte for byte after each (sha256 22436fdf8c9c8d6f... before and after), one sweep via `setTimeout` fails the keep-alive test (5.1 s), a sweep that does nothing fails it (5.1 s), and a sweep calling `closeAllConnections` fails it and tests 1 and 5. The file passed 10 runs in a row (1.4 to 2.0 s each), backend 794 passed and `format:check` passes.

9/F-255 [P3] closed - The keep-alive test answers its held request before the first sweep, so a sweep that runs only once, or one that cuts busy connections, still passes it

File: backend/lib/server/stop-gracefully.test.ts:60-90 (the code it guards: backend/lib/server/stop-gracefully.ts:25-27) **Found:** 2026-10-08 by independent review of chore/cleanup-before-9 (scope: b0806c2..c5cf49b; lenses: quality, security, performance, tests) **Why it matters:** F-254's repair sweeps every 100 ms because a booking's save can still be in flight well after the stop begins. The new test calls finishFirst() straight after stopGracefully (:81-82), so its answer is out within a millisecond and the first sweep at 100 ms already finds the connection idle. Probes, each restored (stop-gracefully.ts sha256 22436fdf8c9c8d6f... before and after): setTimeout in place of setInterval, one sweep only, passes all five tests; a sweep that calls closeAllConnections passes this test too, caught only by tests 1 and 4. With the one-shot sweep a request longer than 100 ms is back to F-254. Also, send("/second") (:88) goes out only after the stop has finished, when server.close's callback has already confirmed no connection is left, so it opens a fresh connection to a closed port and is refused whatever happened to the kept one; the one second timing line (:87) is what carries the proof. **Suggested fix:** Hold the first answer across a few sweeps before finishing it (wait 250 ms after stopGracefully, then finishFirst()). A probe of exactly that, restored after (test file sha256 f76f71fc891be549... before and after), passed on the code as it is, failed with one sweep (5.1 s) and failed with closeAllConnections. Drop the /second line, or say beside it that the timing is the proof. **Resolution:** Fixed 2026-10-08 on chore/cleanup-before-9: the test (renamed "a connection kept open is closed once its answer goes out, however long that takes") waits 250 ms after the stop begins before the answer goes out, so it is past the first sweeps; the /second request is gone and a comment names the timing as the proof. Proved: one sweep (setTimeout) and a sweep that does nothing each fail it (5.1 s); file restored, cmp identical. 5/5 pass. Closed 2026-10-08 by independent review of chore/cleanup-before-9 (c5cf49b..d6e8bd1; lenses: quality, security, performance, tests): the test now holds its answer 250 ms after the stop begins, so the connection is still busy through the sweeps at 100 and 200 ms and is closed only by a later one. Each probe restored stop-gracefully.ts byte for byte (sha256 22436fdf8c9c8d6f... before and after): one sweep (setTimeout at IDLE_SWEEP_MS) fails it (5.1 s, the server's keep-alive timeout); a sweep that does nothing fails it (5.1 s); closeAllConnections now fails it too (the held answer is cut), with tests 1 and 5. On the real code it passed 10 file runs in a row (1.4 to 2.0 s each) and the whole backend suite (794). The /second request is gone and the comment names the one second bound as the proof, which is what it measures. What it cannot catch, like any fixed hold: a single sweep timed after 250 ms (a probe at 400 ms passed all five, restored after); that needs a changed constant, not a slip of setInterval, so not recorded. The test still waits a fixed 100 ms for its request to arrive rather than the arrival signal the helper uses; a late arrival makes it fail or time out, never pass falsely, so not recorded. Test file sha256 c71154f29ab7da45... unchanged by this review.

9/F-256 [P2] closed - The customer's own booking page still lists the people and takes a pick for a service the business assigns, so decision 3's "the server refuses" has a second public way round it

File: backend/routes/public-booking-page-routes.ts:52-80,88-115 (the picker it feeds: frontend/components/booking-page/change-time-panel.tsx:254-264; the stated rule: blueprint/project-plan.md decision 32, backend/lib/errors/person-not-taken.ts:1-2) **Found:** 2026-10-08 by independent step review of 9.1 (scope: ece1aa2..b2926cf; lenses: quality, security, performance, tests) **Why it**

matters: 9.1 enforces `business_assigns` on the times route and on `POST /public/:slug/bookings` only. The 7b routes on the customer's private link, `GET /public/bookings/:token/times` and `POST .../move`, call `findBookingMoveTimes` and `moveBooking`, which read no `personChoice`: the times answer carries `findFreeTimes`' `people` for every service (`find-free-times.ts:78`), and `moveBooking` accepts any person who offers the service (`move-booking.ts:94`). The shipped change-time panel renders that list as a picker. So a Primo customer (every real row was backfilled `business_assigns` by 0022) sees Primo's painters by name on their booking page and can move the estimate onto the one they choose, which is what decision 3 says no front end can do. Decision 32 in the project plan and the person-not-taken header state the refusal without limiting it to the component's routes. Not a regression (7b behaved this way before 9.1) and the spec's out-of-scope line keeps 7a/7b "as built", so this is a gap between the decision as written and the code, not a fault in 9.1's own routes. **Suggested fix:** Frank's call, before feature 9 closes. Either extend the rule to 7b (`findBookingMoveTimes` answers `people: []` and `moveBooking` refuses a person other than the booking's own when the service is `business_assigns`, with route tests like 9.1's), or narrow decision 32 and the person-not-taken header to say the move page keeps its picker on purpose. **Resolution:** Frank decided 2026-10-08: extend the rule to the 7b page, built as step 9.1b (added to the spec). Fixed 2026-10-08 in step 9.1b: the booking page's answer carries `personChoice`; `findBookingMoveTimes` answers `people: []` for a service the business assigns and refuses a person asked for (state `person_not_taken`); `moveBooking` refuses a person the same way after its "already there" check; both routes answer the same 400 as 9.1. `change-time-panel.tsx` shows no Who for such a service and asks the time only. Tests in `public-booking-move-times-routes.test.ts` and `public-booking-move-routes.test.ts`; proved: removing the refusals and the empty list fails three of them. Hand check on the running dev servers: painting-dev's page has no Who, clinic-dev's keeps Any available, Ana, Mei, Sofia. Re-reviewed 2026-10-08 by independent review of 9.1b (scope: `bd63ab6..d21a3e9`; lenses: quality, security, performance, tests); stays fixed, for F-263. What holds: the way round decision 3 is gone on every public path that takes a person. The four are the form's times (`public-booking-links-routes.ts:127-129`) and booking (`book-time.ts:177`, after the replay), both from 9.1, and the 7b times and move (`find-booking-move-times.ts:53-62`, `move-booking.ts:101-105`), each reading the booking's own service without `active`, so a switched-off service (7b decision 13) holds too: a probe on a switched-off `business_assigns` service answered the pick 400, the times 200 with `people: []` and times, a move with nobody 200, and the page `personChoice: "business_assigns"`, `canMove: true`. Each guard bites: removing `moveBooking`'s refusal, `findBookingMoveTimes`' refusal, or the emptied list each fails exactly its own test (every file restored by git checkout, sha256 identical). The 400 is reached only through a verified link, after the same 404/409s as before, so the identical-404 rule is untouched, and the page's new field says only what its panel already shows. What does not hold: the repair's default for a service the business assigns is any available with a confirm that names nobody, which brings back 7b/F-168's silent hand-over to another person (F-263). Two of the step's claims are pinned by no test (F-264). This entry can close with F-263. Closed 2026-10-08 by re-review of 9.1b's fixes (scope: `d21a3e9..0ec47db`): the only things holding it were F-263 and F-264, both closed below on this pass's evidence. The delta touches none of the four refusals or the emptied list (git diff `d21a3e9 0ec47db` changes only `find-booking-choices.ts` among product files), and the move route's refusal now also has its order pinned (F-264). Backend 821 passed three times.

9/F-257 [P3] closed - The seed sets who picks only on services it creates, so a database seeded before 0022 keeps the clinic as `business_assigns` and two new route tests fail on it

File: packages/shared/scripts/seed-dev.ts:565-575 (the tests that read it: backend/routes/public-booking-links-routes.test.ts:532,543) **Found:** 2026-10-08 by independent step review of 9.1 (scope: ece1aa2..b2926cf; lenses: quality, security, performance, tests) **Why it matters:** 0022 backfills every existing service `business_assigns`. The seed writes `personChoice` only inside `if (!existingLink)`, so on any database seeded before this step (Frank's other machine after pull, migrate, seed) Riverbend Clinic (dev)'s ten services stay `business_assigns`. Shown on the local `scheduleads_dev`: set clinic-dev's services to `business_assigns` (what the migration leaves), ran `db:seed` (it reported everything "already there"), and all ten were still `business_assigns`; restored to `customer_picks` after, and the backend suite passed again (808). With that state "a service the customer picks for says so" and "Mei alone does laser" fail, and the try page in 9.5/9.6 would show the clinic with no people. The seed's own pattern elsewhere is to upgrade rows seeded before a column existed ("so no machine needs a rebuild", seed-dev.ts:425-431, 473-485), and the spec's 9.1 line says "the seed states both on every service".

Suggested fix: After the insert-or-find, bring an existing seeded service to its business's value, the way the holidays are brought up: update `personChoice` where it is still the migration's backfill and differs (or, since dev databases are disposable, say in the step report that 0022 needs a reseed from scratch on other machines). A rerun then leaves clinic-dev `customer_picks`. **Resolution:** Fixed 2026-10-08 in 9.1's review fixes, as suggested: after finding an existing seeded service, the seed brings its `personChoice` to its business's value where it differs (seed-dev.ts, the else branch beside the insert), reported as "who picks set on N services". Proved on the local `scheduleads_dev`: clinic-dev's ten services set to `business_assigns`, `db:seed` run, all ten back to `customer_picks`. Nothing sets the column by hand until feature 12, so no owner's choice is overwritten. Re-reviewed 2026-10-08 by re-review of 9.1's fixes (b2926cf..5725ad2; lenses: quality, security, performance, tests); stays fixed, for F-260 and F-261. What holds: the update runs on `tx` inside the seed's one db.transaction (seed-dev.ts:340), and its `bookingLinkId` comes from the select scoped to the business and the seeded slug, so it touches only that business's own seeded services. Shown again on the local `scheduleads_dev`: clinic-dev's ten services set to `business_assigns`, `db:seed` run, all ten back to `customer_picks`, painting-dev's three untouched. What does not hold: the repair brings any differing value back, not only the migration's backfill, so a hand-set choice is reverted (F-260), and its report line reads "(created who picks set on 10 services)" (F-261). Re-reviewed 2026-10-08 by re-review of 9.1's second fixes (5725ad2..84198ad; lenses: quality, security, performance, tests); stays fixed, for F-260. What holds: F-257's own case. On the local `scheduleads_dev`, clinic-dev's ten services set to `business_assigns` (what 0022 leaves) and `db:seed` run: all ten back to `customer_picks`, printed as "who picks the person set on 10 existing services". F-261 is closed. What does not hold: F-260, the repair still reverts a choice made by hand in one direction (see there). Closed 2026-10-08 by independent review of 9.1b (scope: bd63ab6..d21a3e9): re-read the per-business rule from bd63ab6 (seed-dev.ts:554-594): `bringChoicesUp` only when the business is seeded `customer_picks` and every one of its services still holds `business_assigns`, then each existing seeded service is updated on `tx` by an id from the business-scoped select. On the local `scheduleads_dev`, clinic-dev's ten set to

business_assigns (what 0022 leaves) and painting-dev's three to customer_picks, `db:seed` : the clinic's ten back to customer_picks, printed "who picks the person set on 10 existing services", painting's three left customer_picks. F-260, the one thing keeping this open, is closed below. Data restored: clinic-dev 10 customer_picks, painting-dev 3 business_assigns.

9/F-258 [P3] closed - The new person-choice check runs before bookTime's request-key lookup, so a retried booking form whose service changed in between gets a 404 or 400 instead of its booking

File: backend/routes/public-bookings-routes.ts:58-60 (the rule it overtakes: backend/lib/booking/book-time.ts:134-149) **Found:** 2026-10-08 by independent step review of 9.1 (scope: ece1aa2..b2926cf; lenses: quality, security, performance, tests) **Why it matters:** bookTime asks for a booking already made with the form's key first ("Asked first", decision 7/9: a retry after a lost answer gets the booking that won), before it reads the service, so before 9.1 a retry succeeded even if the service had been switched off meanwhile. The route now calls findPersonChoice before bookTime: a switched-off service answers 404, and a service switched from `customer_picks` to `business_assigns` answers 400 to a retry that names its person, though the booking exists. The customer is told it failed and may book again. Narrow today: nothing but a hand edit at client setup changes either column until feature 12's settings screen, and the window is one lost answer long. **Suggested fix:** Keep the replay first: either move the two checks into bookTime after `earlier` (returning a new reason the route maps to the 400), or have the route skip them when the request key already has a booking. A route test: book, switch the service off (or to `business_assigns`), resend the same form, expect 200 with `alreadyBooked` and the same booking. **Resolution:** Fixed 2026-10-08 in 9.1's review fixes, the first suggested way: the route's pre-check is gone; bookTime refuses a customer's pick for a business_assigns service after the form's own booking is looked up, with a new reason `person_not_taken` the route maps to the same 400 (book-time.ts, after `if (!service)`). The owner's path may still pick. A switched-off service is again answered by the replay first. Tests: the route's "still gets its booking, after the business assigns and after the service is switched off" and bookTime's "a booked form sent again after its service changed still gets its booking" and "a customer's pick for a service the business assigns answers person_not_taken and writes nothing; the owner may pick"; proved: restoring the route pre-check and removing bookTime's check fail the route retry test and the bookTime refusal test. Closed 2026-10-08 by re-review of 9.1's fixes: the route no longer reads the service before bookTime (findPersonChoice is now used only by the times route), and bookTime's check sits after `earlier` and after the service read (book-time.ts:155-178), so a resent form gets its booking whatever changed. `person_not_taken` has one production caller to map, the public bookings route, and its switch is exhaustive by type: removing the new case fails `tsc` (TS2339 on result.booking). Each test bites (every file restored by git checkout, status clean after): asking the replay after the service checks fails the route retry test and bookTime's retry test; dropping the refusal fails the route's 400 test and bookTime's refusal test; applying it to source manual too fails bookTime's "the owner may pick". Left as is: like NOT_FOUND before it, the new refusal is not re-asked through `refuseUnlessBooked`, so only a service switched to business_assigns inside one double-submit's own

window could refuse the second copy; not worth an entry. One query fewer per booking. Backend 811 passed three times, build and format:check pass.

9/F-259 [P3] closed - workerTextEntriesOf reads the activity rows with no order, and its callers compare them as an ordered list

File: backend/lib/jobs/worker-text-job.test.ts:168-176 (compared in order at :950-953 and elsewhere)

Found: 2026-10-08 by independent step review of 9.1 (scope: ece1aa2..b2926cf; lenses: quality, security, performance, tests) **Why it matters:** The builder saw "a lost new booking found on its retry after a cancel still texts the person off your day" fail once with `[worker_removed, worker_added]`. The helper's select has no ORDER BY, so Postgres may return the two rows in either order (other test files insert and delete activity rows in parallel, so free space is reused), while the expectation is an ordered array. Pre-existing: the helper dates from 8c.2 (6912cfa); 9.1 only added the two new columns to this file's service insert, which touches neither the jobs nor the activity table. Not reproduced in five full backend runs here (808 passed each time). **Suggested fix:** Order the helper's select by

`activity.createdAt`, then `activity.id`, or compare the kinds as a set where order is not the claim.

Resolution: Fixed 2026-10-08 in 9.1's review fixes: the helper orders by `activity.occurredAt`, then `activity.createdAt` (worker-text-job.test.ts:168-177). In the flaky test the retry records worker_added before the cancel's worker_removed runs, so that order is the claim. Not reproducible on demand (it failed once in this session), so the fix is shown by reading, not by a failing run; three full backend runs after it passed (811 each). Closed 2026-10-08 by re-review of 9.1's fixes: the order is the one the tests claim. Both worker texts of a booking run in one lane (enqueue-worker-text.ts:16, queueName per booking), so one after the other, and the retry's worker_added is recorded before the cancel's worker_removed job reads it; send-worker-text.ts records each through recordActivity with no transaction, so occurredAt is the JS clock at that moment (record-activity.ts:29) and createdAt is a separate statement's now(), microseconds apart, which settles a same-millisecond occurredAt; no practical tie is left (id, a random UUID, would not have ordered them). The helper does read in order: reversing it (both columns descending) fails "a cancel texts the person off your day" and "a lost new booking found on its retry after a cancel still texts the person off your day" (file restored by git checkout). Backend 811 passed three times; the once-seen flake did not recur.

9/F-260 [P3] closed - The seed's new person-choice upgrade reverts any differing value, so a choice set by hand on a seeded service is undone by the next reseed

File: packages/shared/scripts/seed-dev.ts:576-591 **Found:** 2026-10-08 by re-review of 9.1's fixes (scope: b2926cf..5725ad2; lenses: quality, security, performance, tests) **Why it matters:** The seed's other upgrades touch only rows still in the state from before their column existed, so hand edits survive a reseed: the first person's login link only where userId is null (:430), the holidays only where none are picked, the stages only when there are none ("so stages renamed by hand survive a reseed", :434), ticks "added by hand stays" (:600). F-257's suggested fix asked for the same: update "where it is still the migration's backfill and differs". The repair's where is only `ne(personChoice, business.personChoice)`, so it reverts any value. Shown on the local scheduleads_dev: painting-dev's

three services set to customer_picks (as when trying the picker on Summit's estimate in 9.5/9.6), `db:seed` run, all three back to business_assigns, reported "who picks set on 3 services". Harmless today (dev only, nothing but a hand edit sets the column), but from feature 12 a choice made on the Settings screen in dev is silently undone by every reseed. F-257's Resolution says "no owner's choice is overwritten", which holds only until then. **Suggested fix:** Add `eq(bookingLink.personChoice, "business_assigns")` (the backfill) to the update's where, beside the `ne`, so only a service still as 0022 left it is brought up; the clinic case F-257 needed still works, and a painting service set to customer_picks by hand stays. Adjust the comment to say so. **Resolution:** Fixed 2026-10-08 in 9.1's second review fixes, as suggested: the update also requires the value still to be `business_assigns`, the migration's leftover, so a choice made by hand survives. Proved on the local scheduleads_dev: clinic-dev's ten services set to business_assigns and painting-dev's three to customer_picks, `db:seed` run: the clinic's ten came back to customer_picks, painting's three stayed customer_picks; painting restored to business_assigns by hand after. Reopened 2026-10-08 by re-review of 9.1's second fixes (5725ad2..84198ad; lenses: quality, security, performance, tests): the painting case holds, the clinic case does not. A row-level `eq(personChoice, "business_assigns")` cannot tell 0022's backfill from a business_assigns chosen by hand, so on a customer_picks business a service set to business_assigns by hand is still reverted, and the new comment ("a choice made by hand survives a reseed", seed-dev.ts:577-579) says the opposite. Shown on the local scheduleads_dev: the probe above repeated (clinic ten business_assigns, painting three customer_picks, `db:seed`: clinic ten customer_picks, painting three customer_picks, so that half holds); then painting restored to business_assigns and only clinic-dev's laser-hair-removal set to business_assigns (as when trying the assigned path on the clinic in 9.5/9.6, or from feature 12 on its Settings screen), `db:seed` run: it came back to customer_picks, printed "who picks the person set on 1 existing services". So this finding's "Why it matters" still holds for the clinic. Data left as found: clinic-dev's ten customer_picks, painting-dev's three business_assigns. The suggested row check was this finding's own and was applied faithfully; it was not enough. Further suggested fix: decide per business, the way the stages are ("only when there are none"): bring a business's existing seeded services up only when every one of them is still business_assigns and the business is customer_picks, the state 0022 leaves and no hand edit of one service produces; or, if the row rule is kept, reword the comment to name the one case it cannot tell apart. Either way the `ne` beside the new `eq` is now redundant (on a business_assigns business the where can never match), and "1 existing services" reads oddly, like the file's other counts. Fixed again 2026-10-08 after the re-review, its new suggested fix: the seed decides per business, like the stages. A business's existing services are brought to its seeded choice only while every one of them still holds business_assigns, so one choice made by hand on any service keeps them all; the redundant `ne` is gone and the comment says so. Proved on the local scheduleads_dev: (a) clinic-dev all business_assigns, painting-dev all customer_picks: reseed brings the clinic's ten to customer_picks, painting stays customer_picks; (b) one clinic service set back to business_assigns by hand among nine customer_picks: reseed leaves it. Data restored after (clinic 10 customer_picks, painting 3 business_assigns). Closed 2026-10-08 by independent review of 9.1b (scope: bd63ab6..d21a3e9): both of the reopening's cases now hold, shown on the local scheduleads_dev. (a) clinic-dev's ten set to business_assigns and painting-dev's three to customer_picks, `db:seed`: clinic ten customer_picks, painting three still customer_picks (a choice by hand on a business seeded business_assigns is never touched, since `bringChoicesUp` needs

the business to be customer_picks). (b) painting back to business_assigns, only clinic-dev's laser-hair-removal set to business_assigns among nine customer_picks, `db:seed` : no "who picks" line printed, laser still business_assigns, the nine unchanged. The redundant `ne` is gone (the update's where is the id alone, seed-dev.ts:589-592), and the comment (:556-559) now says what the rule does. What it still cannot tell apart, by design and like the stages: every service of a customer_picks business set to business_assigns by hand reads as 0022's leftover and is brought back; the comment's "while every service it has still holds the business_assigns migration 0022 gave them" names that condition, so not recorded. Data restored: clinic-dev 10 customer_picks, painting-dev 3 business_assigns.

9/F-261 [P3] closed - The seed's report line reads "(created who picks set on N services)"

File: packages/shared/scripts/seed-dev.ts:622 (the sentence it lands in: :629) **Found:** 2026-10-08 by re-review of 9.1's fixes (scope: b2926cf..5725ad2; lenses: quality, security, performance, tests) **Why it matters:** Every entry of `made` is printed after "(created ", and the new one is not a thing created but a change. Shown on the local scheduleads_dev after setting clinic-dev to business_assigns and reseeding: `owner@example.com ordinary owner, owns "Riverbend Clinic (dev)" (created who picks set on 10 services)` . The seed's report is what tells Frank on a new machine what a reseed did, and this line reads as a typo. **Suggested fix:** Word the entry so it reads after "created", or print upgrades apart from creations, e.g. "(created ...; brought up: who picks on 10 services)". The existing upgrade entries ("first person's login link", "holiday picks") read as nouns, so a noun phrase also fits. **Resolution:** Fixed 2026-10-08 in 9.1's second review fixes: the count is no longer in the "(created ...)" list; it prints on its own line, "who picks the person set on N existing services", seen in the probe's output. Closed 2026-10-08 by re-review of 9.1's second fixes: `choicesSet` is gone from `made` (seed-dev.ts:610-626) and prints on its own indented line after the owner's line, only when non-zero (:631). On the local scheduleads_dev after setting clinic-dev to business_assigns and reseeding, the report read `owner@example.com ordinary owner, owns "Riverbend Clinic (dev)" (already there) then who picks the person set on 10 existing services` ; painting-dev printed no extra line. "1 existing services" for a count of one is the file's own habit ("N services"), noted under F-260, not a reason to keep this open.

9/F-262 [P3] closed - "the move's answer does not wait for Google" gives Google's PATCH only vi.waitFor's default second, so a slow full run fails it

File: backend/lib/calendar/move-booking-event.test.ts:489 (the same wait at :737; remove-booking-event.test.ts:267, write-booking-event.test.ts:226) **Found:** 2026-10-08 by re-review of 9.1's second fixes (scope: 5725ad2..84198ad; lenses: quality, security, performance, tests; seen in the gate run, not in the delta) **Why it matters:** The test holds Google's answer, works the event jobs, and waits for the PATCH with `vi.waitFor` and no options, so 1000 ms. Before the PATCH the job is claimed and its rows are read, all against the shared local Postgres while 70 other files run beside it. In this pass's full backend run (89 s, the file alone 70.6 s) it failed after 2189 ms: `expected false to be true` at :489, 1 failed, 810 passed. The next full run (56.5 s) passed 811. Nothing in the delta reaches it (the seed is not run by the tests; the test makes its own clinic). A gate that fails on load alone teaches the next reader to rerun rather than read, which is how a real failure gets waved through. The test dates from 7b.3, reworked in 8a.3

(40f598e). **Suggested fix:** Give the four `vi.waitFor` calls that wait on a job's Google call a timeout sized for a loaded run, e.g. `{ timeout: 10_000 }`; the claim each test makes (the answer came before Google) is unchanged, since that is asserted before the wait. **Resolution:** Fixed 2026-10-08 in 9.1's third review fixes, as suggested: the four `vi.waitFor` calls (move-booking-event.test.ts two, remove-booking-event.test.ts, write-booking-event.test.ts) wait up to 10 seconds; each test's claim, that the answer came before Google, is asserted before the wait and unchanged. Shown by reading: the 1-second default failed once in a slow full run, and is not reproducible on demand. Closed 2026-10-08 by independent review of 9.1b (scope: bd63ab6..d21a3e9): all four waits on a job's Google call now pass `{ timeout: 10_000 }` (move-booking-event.test.ts:489-491 and :739-741, remove-booking-event.test.ts:267, write-booking-event.test.ts:226), and they are the only `vi.waitFor` calls in the backend's tests. The three "does not wait for Google" tests still assert "answered before Google is asked" before their wait (the fourth, "two of one booking's jobs never run at the same time", only waits for the PATCH to begin) (move-booking-event.test.ts:486-487, remove-booking-event.test.ts:264, write-booking-event.test.ts:223), so the claim is unchanged; the 10 seconds sit inside the backend's own `testTimeout: 30_000` (vitest.config.ts), so the wait, not the test, is what gives up. Three full backend runs in this review passed 816 each (31 to 32 s); a timing flake cannot be shown fixed by passing runs, only by the wider bound, which is in place.

9/F-263 [P2] closed - On a service the business assigns, "Change the time" always moves with any available, so the customer's booked person can be swapped for another while free, and the confirm names nobody

File: frontend/components/booking-page/change-time-panel.tsx:80,175-181 (the order it inherits: backend/lib/booking/find-booking-choices.ts:120-135, backend/lib/scheduling/order-any-available.ts; the rule it re-opens: blueprint/history/features/07b-reschedule.md decision 14, 7b/F-168) **Found:** 2026-10-08 by independent review of 9.1b (scope: bd63ab6..d21a3e9; lenses: quality, security, performance, tests) **Why it matters:** Before 9.1b every Primo booking's panel opened on her own person (7b decision 14), so keeping the default kept the painter. Now, for a business_assigns service, the panel can only ask with nobody picked, and a move with nobody picked orders the free people by fewest bookings that day, then name, with no preference for the person who holds the booking. Decision 14 rejected exactly this: "opening on Any available, which could hand her to another person on a tie without saying so". Shown with a scratch test in the move routes (removed after): Jane's facial with Ana at 9:00, Ana also booked at 13:00, the service set to business_assigns, a move to 11:00 with nobody picked while Ana is free at 11:00: the booking went to Mei. The confirm reads "Move to <time>?" with no name, while the page still says who is coming (the spec keeps the name: "the customer is told who is coming"). For Primo, where every real service is business_assigns, each time-only move can quietly change the estimator, and with it send "off your day" and "added" texts to two painters and remove and rewrite the Google event. Not a broken line of 9.1b's spec (it says "asks the times with nobody picked"), but a consequence the spec does not name and decision 14 had ruled out. **Suggested fix:** Frank's call. The smallest that keeps decision 3 (no pick offered): on a move with nobody picked, try the booking's own person first when free (findBookingChoices, a `preferPersonId` for `movingBooking`), so "any available" means "whoever is free, keeping yours if you can"; a route test like the probe above. Or keep the swap and say so in the

confirm and the spec ("the business may send someone else"). **Resolution:** Frank decided 2026-10-08: keep her person first (option A). Fixed 2026-10-08: findBookingChoices puts the moving booking's own person first when a move names nobody, so a time-only move keeps who comes while they are free, for both settings (7b decision 14 holds again); someone else only when they are busy. Tests in public-booking-move-routes.test.ts: "a move with nobody picked keeps her own person when free" (business_assigns and customer_picks, Ana with more bookings than Mei still kept) and "a move with nobody picked goes to someone else only when her own person is busy"; proved: dropping the preference fails both cases of the first. Closed 2026-10-08 by re-review of 9.1b's fixes (scope: d21a3e9..0ec47db): the swap while free is gone. find-booking-choices.ts:137-146 puts every choice of `movingBooking.personId` (with each free room, in room order) before the others, only when the move names nobody; the owner's path cannot reach it (`movingBooking?: never` on `byOwner: true`, :37) and bookTime never passes one (book-time.ts:191-202), so a new booking keeps the fewest-bookings order. Proved: forcing the early return (`if (!ownPerson || true)`) fails both cases of "a move with nobody picked keeps her own person when free" (public-booking-move-routes.test.ts:347), the review's own probe in route form (Ana with more bookings than Mei, Ana free at 11:00, kept); file restored, sha256 identical. Scratch probes in the move route tests (removed after): with Room 3 and Ana busier, the move kept Ana and held Ana plus Room 3; with Ana no longer offering the facial, it went to Mei, as decision 14's fallback says; with Ana busy at 11:00 it went to Mei (the saved test at :363). When the person is kept, moveBooking's later steps take the same-person path already used by a picked move: `personChanged` false (move-booking.ts:187), so one "moved" text to her person, the event moved in place, no remove unless no event id was saved. The confirm still names nobody on a business_assigns service, which option A accepts: whoever is free, keeping hers if she can, and the page names who comes after. For customer_picks the explicit "Any available" now also keeps her person when free (the resolution's "for both settings"); no 7b test or decision is broken by it, but it leaves two loose ends recorded as F-266 and F-267. Her own person's calendar unreadable is a separate case, F-265.

9/F-264 [P3] closed - Two of 9.1b's claims are pinned by no test: the page answering business_assigns, and the refusal sitting after "already there"

File: backend/routes/public-booking-page-routes.test.ts:172 (the field: backend/lib/booking/find-booking-page.ts:100; the order: backend/lib/booking/move-booking.ts:76-79,101-105) **Found:** 2026-10-08 by independent review of 9.1b (scope: bd63ab6..d21a3e9; lenses: quality, security, performance, tests) **Why it matters:** The whole frontend change switches on `booking.personChoice`, but the page test only expects `customer_picks`. Scratch mutation, restored after: findBookingPage answering `"customer_picks"` for every booking passed the full backend suite (816). On a business_assigns service that would bring back the Who list and ask the times with her own person, which the times route now refuses with a 400 the client maps to "The free times can't load right now", on every Primo page. Second, the code and the spec say the refusal comes after "already there" so a repeat answers as before; the only repeat test sends nobody, so the order is free. Scratch mutation, restored after: "already there" moved below the refusal passed all 20 move route tests. **Suggested fix:** In the page route tests, one booking on a business_assigns service expecting `personChoice: "business_assigns"`. In the move route tests,

move with a person on a customer_picks service, switch it to business_assigns, send the same move again, expect 200 and one booking_moved entry. **Resolution:** Fixed 2026-10-08 after 9.1b's review: public-booking-page-routes.test.ts "the page says who picks: a service the business assigns says business_assigns" and public-booking-move-routes.test.ts "a picked person on a service the business assigns, already there, answers the same". Proved: the review's two mutations (findBookingPage hardcoded to customer_picks; "already there" moved below the refusal) each fail their test; files restored. Closed 2026-10-08 by re-review of 9.1b's fixes (scope: d21a3e9..0ec47db): both claims are now pinned. findBookingPage answering "customer_picks" as const for every booking fails "the page says who picks: a service the business assigns says business_assigns" (public-booking-page-routes.test.ts:156, 1 failed of 11); the test works on its own business (made customer_picks, restored to customer_picks in finally), never the seeded painting-dev. Moving if (alreadyThere(current)) below the person_not_taken refusal in move-booking.ts fails "a picked person on a service the business assigns, already there, answers the same" (public-booking-move-routes.test.ts:333, 1 failed of 24). That test repeats with her own person at her own time instead of the suggested move-then-switch, which pins the same order (200, sequence 0, no booking_moved entry). Both files restored by git checkout, sha256 identical.

9/F-265 [P3] closed - A move with nobody picked hands the booking to someone else when her own person's calendar cannot be read

File: backend/lib/booking/find-booking-choices.ts:84-92,137-146 (the move: backend/lib/booking/move-booking.ts:122-135,187,238-250) **Found:** 2026-10-08 by re-review of 9.1b's fixes (scope: d21a3e9..0ec47db; lenses: quality, security, performance, tests) **Why it matters:** F-263's repair keeps her person first only when the free check answers "free". When her person's Google cannot be read, the check answers "unreadable", she is left out of the free people as any available always leaves people out, and the move goes to whoever else is free with no word to the customer. Shown with a scratch test in the move routes (removed after): Jane's facial with Ana at 9:00, the service set to business_assigns, Ana's connection needs_reconnect, a move to 11:00 with nobody picked: 200, the booking went to Mei, one booking_moved entry. Before 9.1b the panel asked with her own person and the same move answered 503 "try again" (7b's "a picked person whose calendar cannot be read answers 503 and nothing changes"). A needs_reconnect connection lasts until the person reconnects, so for Primo, where every service is business_assigns, every time-only move of that estimator's bookings in that window can change the estimator, with "off your day" and "added" texts to two painters and the event moved between calendars, which is the hand-over decision 14 and F-263 rule out while the person may well be free. The customer_picks "Any available" case does the same, but there it is her explicit pick (unchanged since 7b). **Suggested fix:** Frank's call, one of: on a move with nobody picked, when her own person still offers the service and their calendar is unreadable, answer unavailable (503, "try again", as a picked person does) instead of handing over; or accept the hand-over as any available's rule and say so in the spec. Either way a route test like the probe above. **Resolution:** Fixed 2026-10-08 under Frank's F-263 decision ("someone else only if her person is busy"; an unreadable calendar is not busy): findBookingChoices answers unavailable (the move's 503, "try again") when a move names nobody

and its own person's calendar cannot be read, before any other person is tried. Test: public-booking-move-routes.test.ts "a move with nobody picked answers try again when her own person's calendar cannot be read" (503, the booking unchanged); proved: removing the check fails it. Re-reviewed 2026-10-08 by independent review of 9.2 (scope: ef70707..d1c13c6, with the repair in 0ec47db..ef70707); stays fixed, for F-269. What holds: the move itself no longer hands the booking over. find-booking-choices.ts:90-92 answers `unavailable` when a move names nobody and its own person is among the candidates with an unreadable calendar; a person who no longer offers the service gives `indexOf` -1, so the move goes on to whoever is free, as decision 14's fallback says; bookTime never passes `movingBooking`, so a new booking is untouched. Removing the two lines fails the new test (1 failed of 25; file restored, sha256 identical). What does not hold: the repair made the move refuse while the move page's times, asked with nobody picked, still list the other people's times, so every time offered then answers "try again" (F-269). Closed 2026-10-08 by the review of step 9.3 (scope: 3de57a5..21b8d6d): the hand-over stays fixed. find-booking-choices.ts:90-91 answers `unavailable` before any other person is tried when a move names nobody and its own person's calendar cannot be read. Removing those two lines fails "a move with nobody picked answers try again when her own person's calendar cannot be read" (public-booking-move-routes.test.ts:363, 1 failed of 25); file restored, sha256 identical. The page side is F-269, closed in the same pass. The finding number the repair left in the comment at :89 is F-274.

9/F-266 [P3] closed - The day's counts no longer need to leave out the moving booking, so 7b/F-142's guard and its test pin nothing

File: backend/lib/booking/find-booking-choices.ts:120-131 (the test: backend/routes/public-booking-move-routes.test.ts:278-286; the comment: :351) **Found:** 2026-10-08 by re-review of 9.1b's fixes (scope: d21a3e9..0ec47db; lenses: quality, security, performance, tests) **Why it matters:** The moving booking's rows belong to her own person, and her own person now goes first whenever free, whatever the counts; a picked move has one candidate, so its order never matters. So `.filter(notTheMovingBooking)` on the day's rows, and "(the moving booking not counted)" in the comment above it, can no longer change any outcome. Shown: removing that filter passes the full backend suite (821); at d21a3e9 the same removal failed "any available does not count the booking being moved". That saved test now passes because of the preference, not the count it names, so 7b/F-142's coverage is gone without a word. Smaller, in the new test: "Ana now has two bookings that day, Mei one" (:351); Mei has none (makeClinic books only Ana), and with Jane's own left out Ana has one. The test still bites (Ana counts higher either way). **Suggested fix:** Drop the day-row filter and the comment's parenthesis, and retitle or fold "any available does not count the booking being moved" into the keep-own tests (what it now shows is that she stays with Ana); or keep the filter as a guard for a future caller and say in the comment that no current caller depends on it. Correct the test comment to "Ana now has two bookings that day, Mei none". **Resolution:** Fixed 2026-10-08: the day counts no longer leave out the moving booking (find-booking-choices.ts, the comment says why: it is its own person's, first when free and not counted when busy); the room check keeps its filter. The 7b test "any available does not count the booking being moved" keeps its name and its outcome, its comment now says the own-person rule decides it; the new test's comment says "Mei none". Re-reviewed 2026-10-08 by independent review of

9.2 (scope: ef70707..d1c13c6, with the repair in 0ec47db..ef70707); stays fixed, for the test's title only. What holds: the day-row filter is gone (find-booking-choices.ts:128-133) and nothing depended on it: the day's counts read only `freePeople`, so the moving booking's rows count only for its own person, who goes first when free (:140-147) and is not in `freePeople` when busy; its room rows are never read there (the select is by person). `notTheMovingBooking` still guards the room check (:111). The comments at :125-127 and in both tests now say what decides. Backend 843 passed three times. What does not hold: the test is still titled "any available does not count the booking being moved" (public-booking-move-routes.test.ts:277), while the code now does count it; the title claims the guard this repair removed. Retitle it to what it shows (she stays with her own person while free), or fold it into the keep-own tests; then this closes. Fixed again 2026-10-08 after 9.2's review: the test is renamed "a move with nobody picked stays with her own person, though counting her own booking would favour another", which is what decides it now. Closed 2026-10-08 by the review of step 9.3 (scope: 3de57a5..21b8d6d): the test is titled "a move with nobody picked stays with her own person, though counting her own booking would favour another" (public-booking-move-routes.test.ts:278), which is what decides it; the day-row filter stays gone (find-booking-choices.ts:128-133) and the room check keeps `notTheMovingBooking`. Full backend suite 865 passed. The "(F-266)" the repair put in the comment at :127 is F-274.

9/F-267 [P3] closed - The spec still says a customer_picks move is unchanged and a business_assigns move is plain any available; Frank's F-263 decision is in no plan file

File: blueprint/context/current-feature.md:169-189 (step 9.1b) **Found:** 2026-10-08 by re-review of 9.1b's fixes (scope: d21a3e9..0ec47db; lenses: quality, security, performance, tests) **Why it matters:** The delta changes what "any available" means on every move, for both settings: her own person first while free. The spec's 9.1b still says "A service the customer picks for is unchanged" and "asks the times with nobody picked (any available)", and Frank's decision (option A, 2026-10-08) lives only in this ledger and a commit message. /complete archives the spec as the feature's history; without a line there, the permanent record says the opposite of the code, and 7b's archive still says the backend's any-available order is unchanged (7b/F-168's closing note), so the next reader of either file is told the old rule. AGENTS.md asks for the spec to be amended when a review shows it wrong. **Suggested fix:** Add to step 9.1b (or the feature's decisions) one line: a move with nobody picked keeps her own person first while free, for both settings, someone else only when that person is busy (Frank, 2026-10-08, F-263), plus whatever F-265 settles for an unreadable calendar. **Resolution:** Fixed 2026-10-08: step 9.1b in current-feature.md records Frank's F-263 decision and F-265 (a move with nobody picked keeps its own person while free under either setting; someone else only when busy; try again when their calendar cannot be read). The 7b archive is history and stays as written. Closed 2026-10-08 by independent review of 9.2 (scope: ef70707..d1c13c6): current-feature.md:180-183, inside step 9.1b, now reads "Amended after the step's review (Frank, 2026-10-08, F-263 option A): a move with nobody picked, under either setting, keeps the booking's own person while they are free; someone else only when they are busy, and 'try again' when their calendar cannot be read (F-265)", which is what find-booking-choices.ts:88-92 and :140-147 do. The older sentences above it ("A service the customer picks for is unchanged", "asks the times with nobody picked") stay, but the amendment follows them in the same step and names the

decision, so the archived spec no longer says the opposite of the code. Leaving 7b's archive as written is right: it records 7b.

9/F-268 [P3] closed - Two of 9.2's rules are pinned by no test: the answers checked after the form's own booking, and the owner's booking needing none

File: backend/lib/booking/book-time.ts:185-191 (the replay it must follow: :161-162) **Found:** 2026-10-08 by independent review of 9.2 (scope: ef70707..d1c13c6; lenses: quality, security, performance, tests) **Why it matters:** Both are deliberate choices of this step, named in the code's comments and the step report. The answer checks sit in bookTime after the request-key replay, so a retried form gets its booking whatever questions changed meanwhile (the F-258 lesson); and `requireAnswers: source !== "manual"` lets the owner book from a phone call without answering. Scratch mutations, each restored after (sha256 identical): moving the whole answer check above `const earlier = await bookedByThisForm()` passed the full backend suite (843); `requireAnswers: true` for every source passed it too (843). checkAnswers' own unit test covers the flag, not bookTime's wiring of it. So a later edit could put the F-258 bug back for questions (a form booked, its answer lost, a question made required or removed meanwhile: the retry is told "Answer: ..." or "not one of this business's questions" though it is booked, and may book again), or make feature 11's owner booking demand answers, and nothing would say so. The spec's Testing section asks that each new rule's test be shown able to fail. **Suggested fix:** A route test: book with answers, then make the optional question required (or delete one), resend the same form, expect 200 with `alreadyBooked` and the same booking id. A bookTime test: source `manual` in a business with a required question, no answers, expect booked and `lead.answers []`. Show each fails under the mutation above. **Resolution:** Fixed 2026-10-08 after 9.2's review: public-bookings-routes.test.ts "a booked form sent again after a required question was added still gets its booking" and book-time.test.ts "a customer must answer a required question; the owner need not" (the owner's lead keeps []). Proved: moving the answer check above the replay fails the first only; requiring answers for every source fails the second only. Closed 2026-10-08 by the review of step 9.3 (scope: 3de57a5..21b8d6d): both rules are pinned. A copy of the answer check inserted above `const earlier = await bookedByThisForm()` (book-time.ts:161) fails only "a booked form sent again after a required question was added still gets its booking" (public-bookings-routes.test.ts:352); `requireAnswers: true` for every source fails only "a customer must answer a required question; the owner need not" (book-time.test.ts); each run 1 failed of 54, file restored, sha256 identical.

9/F-269 [P3] closed - With her own person's calendar unreadable, the move page still lists other people's times, and every one of them answers "try again"

File: backend/lib/booking/find-booking-move-times.ts:64-77 (the move's refusal: backend/lib/booking/find-booking-choices.ts:88-92; any available leaving an unreadable person out: backend/lib/scheduling/find-free-times.ts:145-155) **Found:** 2026-10-08 by independent review of 9.2 (scope: ef70707..d1c13c6, F-265's repair in it; lenses: quality, security, performance, tests) **Why it matters:** F-265's repair answers 503 to a move with nobody picked when the booking's own person's calendar cannot be read. The times the page offers for that same move are asked with nobody picked too,

and any available simply leaves an unreadable person out, so the list shows everyone else's free times. Shown with a scratch test in the move routes (removed after, sha256 identical): Jane's facial with Ana, the service set to business_assigns, Ana's connection `needs_reconnect`: the move times route answered 200 with 11:00 offered (people `[]`), and the move to 11:00 answered 503. For Primo, where every service is business_assigns, each customer of that estimator sees a full list in which every pick ends in "The move didn't go through. Please try again, or call the business.", for as long as the connection waits to be reconnected. Before 9.1b the panel asked with her own person, so the same case showed "can't load" at the times step, with the phone, before any pick. The customer_picks "Any available" choice does the same. **Suggested fix:** The same rule on the times: in `findBookingMoveTimes`, when nobody is picked and the booking's own person still offers the service, read that person's times first and let `CalendarUnavailableError` through (the route already maps it to 503, and the panel shows its "can't load" state with the phone). A route test like the probe above, expecting 503 from the times. This only makes Frank's F-263/F-265 decision hold on the page as well as on the move. **Resolution:** Fixed 2026-10-08 after 9.2's review, as suggested: `findBookingMoveTimes`, with nobody picked, also reads her own person's times beside the any-available ones, so an unreadable calendar throws as any unreadable read and the route answers 503 "try again", the same as the move. Test: `public-booking-move-times-routes.test.ts` "with nobody picked, her own person's unreadable calendar answers 503, though another is free"; proved: dropping the second read fails it only. Closed 2026-10-08 by the review of step 9.3 (scope: 3de57a5..21b8d6d): `find-booking-move-times.ts:82-85` asks her own person's times beside the any-available ones when nobody is picked, so an unreadable calendar throws `CalendarUnavailableError` and the route answers 503, as the move does. A person who no longer offers the service makes `findFreeTimes` answer null (`find-free-times.ts:76`), which the destructuring ignores, so decision 14's fallback still lists the others. Replacing the second read with `null` fails "with nobody picked, her own person's unreadable calendar answers 503, though another is free" (`public-booking-move-times-routes.test.ts:274`, 1 failed of 17); file restored, sha256 identical.

9/F-270 [P3] closed - `BookingQuestionType` lives in `check-answers.ts`, not in `find-booking-questions.ts` which produces it

File: `backend/lib/booking/check-answers.ts:7` (its producer imports it back: `backend/lib/booking/find-booking-questions.ts:9,11`) **Found:** 2026-10-08 by independent review of 9.2 (scope: ef70707..d1c13c6; lenses: quality, security, performance, tests) **Why it matters:** `coding-standards.md`: "A type sits in the file of the function that produces it", because Frank navigates by file name. `findBookingQuestions` returns `BookingQuestionType[]` but has to import the type from the checker, so someone looking for the shape of a question opens `find-booking-questions.ts` and finds only an import. **Suggested fix:** Move `BookingQuestionType` into `find-booking-questions.ts` and import it in `check-answers.ts` (type-only, no cycle). **Resolution:** Fixed 2026-10-08 after 9.2's review: `BookingQuestionType` moved to `find-booking-questions.ts`, which returns it; `check-answers.ts` imports it. Closed 2026-10-08 by the review of step 9.3 (scope: 3de57a5..21b8d6d): `BookingQuestionType` is declared in `find-booking-questions.ts:10` beside `findBookingQuestions`, which returns it, and `check-answers.ts:7` imports it type-only; `git grep` finds no other declaration. Backend build passes.

9/F-271 [P2] closed - A booked form sent again counts against its contact, and at the limit it is refused 429 instead of getting its booking

File: backend/routes/public-bookings-routes.ts:63-69,89-90 (the replay it runs ahead of: backend/lib/booking/book-time.ts:151-162; the rules: current-feature.md decisions 8 and 9) **Found:** 2026-10-08 by /audit independent (scope: step 9.3; lens: correctness) **Why it matters:** The contact is counted before bookTime and given back only when `booked` is false. A form already booked comes back from bookTime's replay as `booked: true, alreadyBooked: true`, so each resend is counted as a new booking, and once the contact is at 4 the resend never reaches the replay: the route answers 429. Decision 8, amended in this step, says "only bookings made count against the contact", and decision 9 says a retry after a lost answer gets the booking that won, the order F-258 settled for the person check and F-268 for the answers. Shown with scratch tests in the bookings route tests (removed after, sha256 identical): one form sent four times answered 201 each time (the one booking, as "the same form sent twice" shows), then a new booking for the same email answered 429 with one booking made; four bookings for one email, then the fourth's form sent again, answered 429 `too_many_tries` though it is booked. A customer whose answer was lost is told "Too many tries" for a booking that exists, and may call or book again. **Suggested fix:** Give the count back when `result.alreadyBooked` is true; and let a form's own booking answer before the contact's refusal (when the count refuses and the form carries a `requestKey`, look up its booking as bookTime's replay does, and answer it if it exists). Route tests like the two probes above, each shown able to fail. **Resolution:** Fixed 2026-10-08 in 9.3's review fixes: the contact is counted inside bookTime through `admitNewBooking`, asked just after the form's own booking is looked up (book-time.ts), so a form sent again is answered its booking before the limit is asked and is never counted; the route hands the count back when the booking is refused, crashes, or comes back `alreadyBooked` (copies of one form racing). Tests: "a form sent again gets its booking even at the limit, and resends never count" (fails with the limit asked before the look-up) and "copies of one form arriving together count as the one booking they make" (four at once, then three more book and the next is 429; fails 3 of 3 runs without the alreadyBooked hand-back, passes 3 of 3 with it). Spec decision 8 says so. Closed 2026-10-08 by re-review of 9.3's fixes (scope: 21b8d6d..8e8a864): book-time.ts:166-168 asks `admitNewBooking` once, just after the form's own booking is looked up, so a booked form sent again is answered before the limit and never counted. Every way out after it hands the count back once: a refusal (`!result.booked`) or the form's booking found later (`alreadyBooked`, from `refuseUnlessBooked` or the request-key constraint) at public-bookings-routes.ts:99, and a throw in the `.catch` at :94, which rethrows so :99 is not reached; a refusal or crash before it finds `taken` null and hands back nothing. Only the public route passes it (`git grep admitNewBooking`), so the owner's bookings through bookTime are not limited. Asking the limit before the look-up fails "a form sent again gets its booking even at the limit, and resends never count" (1 failed of 35); dropping `|| result.alreadyBooked` fails "copies of one form arriving together count as the one booking they make" 3 runs of 3. Files restored, sha256 identical. Copies of one form arriving together while the contact is at its last place are a narrower case the fix leaves: F-275.

9/F-272 [P3] closed - giveBack takes from whatever window is open when it runs, so a try that straddles a window's end frees a place in the next one

File: backend/lib/rate-limit/create-rate-limiter.ts:51-57 (its caller: backend/routes/public-bookings-routes.ts:85,90) **Found:** 2026-10-08 by /audit independent (scope: step 9.3; lens: correctness) **Why it matters:** take counts a try in the key's open window; giveBack takes one off whichever window is open when it runs. When a booking is counted near the end of a contact's 10 minute window and refused after it ends, while another booking for the same contact has opened a new window, the refusal hands back a place it never took there. Shown with a scratch unit test (removed after, restored byte for byte): `most: 1`, a take, the clock moved one window, a second take allowed (window 2, count 1), giveBack for the first, then a third take in window 2 answered `{ allowed: true }`: two tries made in a window of one. Each straddling refusal lets one more booking through, so it needs a slow bookTime (a Google read) across the boundary with a second booking racing it. Small, but it is the one way the all-or-none count lets a contact past 4. **Suggested fix:** Let take report the window it counted each key in (its `endsAt`) and giveBack take off only a window with that same end; a unit test like the probe above. **Resolution:** Fixed 2026-10-08 in 9.3's review fixes: `take` answers where each key was counted (`taken`, its window's end) and `giveBack` takes off only a window with that same end. Test: "a try counted in a window that has ended frees nothing in the next one", the probe above; fails with the end check removed. Closed 2026-10-08 by re-review of 9.3's fixes (scope: 21b8d6d..8e8a864): create-rate-limiter.ts:44-49 returns each key's window end in `taken`, and giveBack at :55-59 takes off only a window whose `endsAt` is that end. A window opened after the old one ended starts at or after that end, so its own end is later and never equal. Removing `window.endsAt === windowEndsAt &&` fails "a try counted in a window that has ended frees nothing in the next one" (1 failed of 7); file restored, sha256 identical. The route keeps `taken` from its one take and hands back only that.

9/F-273 [P3] closed - Two of 9.3's route rules are pinned by no test: each business counted apart, and the give-back on a crash

File: backend/routes/public-bookings-routes.ts:64,84-86 (the tests: backend/routes/public-bookings-routes.test.ts:546-623, backend/lib/rate-limit/booking-contact-keys.test.ts:24-30) **Found:** 2026-10-08 by /audit independent (scope: step 9.3; lens: tests) **Why it matters:** The Done when lists "two businesses counted apart". It is shown only by bookingContactKeys' unit test; the route's wiring is free. Scratch mutation, restored after (sha256 identical): `bookingContactKeys("one-for-all", body.customer)` in the route passed every route and rate-limit test (242). With it, a contact's four bookings at Primo would refuse her at Face and Body. Second, removing the `.catch` give-back ("a crash made no booking either") passed every route test (231), so a later edit could drop it and a contact would lose a place for each 500. **Suggested fix:** A route test booking one email four times in this file's clinic and once more in a second business it makes, expecting 201; and one where bookTime throws once (a spy on one of its reads) followed by four bookings for that contact that all pass. Show each fails under the mutations above. **Resolution:** Fixed 2026-10-08 in 9.3's review fixes: the bookings route tests make two more businesses, a shop and one with no pipeline stage. "two businesses count the same email apart" (four at the clinic,

then 201 at the shop) fails with one key for every business; "a booking that crashes does not count" (five crashes at the stageless business, each 500, never 429) fails with the crash hand-back removed. Closed 2026-10-08 by re-review of 9.3's fixes (scope: 21b8d6d..8e8a864): `bookingContactKeys("one-for-all", body.customer)` in the route fails "two businesses count the same email apart" (1 failed of 35); deleting the `.catch`'s `handBack()` fails "a booking that crashes does not count" (1 failed of 35). The stageless business crashes at `book-time.ts:233`, after the count is taken, so the test reaches the hand-back. Both extra businesses go in afterAll with the clinic; after a full run no `test-bookings-%` business is left and no row in any table with an `organization_id` points at a missing business. File restored, sha256 identical.

9/F-274 [P3] closed - Code comments carry finding numbers and who agreed when, which the standards keep in the build log

File: `backend/lib/booking/find-booking-choices.ts:88-89,127`; `backend/lib/booking/find-booking-move-times.ts:80-81`; `backend/lib/rate-limit/create-rate-limiter.ts:3,12`; `backend/lib/rate-limit/public-rate-limiters.ts:1` (and eight test comments: `book-time.test.ts:567`, `public-booking-move-routes.test.ts:281,332,346`, `public-booking-move-times-routes.test.ts:272`, `public-booking-page-routes.test.ts:155`, `public-bookings-routes.test.ts:351,386`) **Found:** 2026-10-08 by /audit independent (scope: step 9.3, with the repairs of F-265 to F-269 it re-checked; lens: quality) **Why it matters:** `coding-standards.md`, Comments: "No history in code comments (step numbers, finding numbers, ...): that lives in the build log." Feature 9's repairs put bare IDs (F-258, F-263 to F-269) into five source comment lines and eight test comments, and this step adds "(Frank, Oct 8)" and "agreed by Frank on Oct 8". The IDs also go stale: /complete archives this ledger as 9/F-265 and the next ledger starts again at F-01, so a later F-265 would be a different finding from the one the code names. The decision references beside them ("7b decision 14", "feature 9, decision 8") are spec links and fine. Smaller, same lens: `RateLimiterType` (`create-rate-limiter.ts:12`) is exported and used nowhere (`git grep` finds only its declaration).

Suggested fix: Drop the finding IDs and the who and when, keeping the sentence each one ends (the rule stands without its number); delete `RateLimiterType`, or use it where the limiters are typed. **Resolution:** Fixed 2026-10-08 in 9.3's review fixes: the finding numbers are gone from the five source and eight test comments, each sentence kept; the who and when are gone from `create-rate-limiter.ts` and `public-rate-limiters.ts`; `RateLimiterType` is deleted. `git grep -E "F-[0-9]{2,3}|Oct 8" -- backend` finds nothing outside dist. The migration's F-06 stays: migrations are not edited. Closed 2026-10-08 by re-review of 9.3's fixes (scope: 21b8d6d..8e8a864): `git grep -n -E "F-[0-9]{2,3}|Oct 8" 8e8a864 -- backend '!:backend/dist'` finds nothing, and `git grep RateLimiterType` finds nothing in backend, packages or frontend. Each edited comment in `find-booking-choices.ts:88-89,127`, `find-booking-move-times.ts:79-81` and the eight tests keeps its sentence with only the IDs gone; the decision references stay. `npx tsc --noEmit -p backend` passes.

9/F-275 [P3] closed - Two copies of one form at the contact's last place: one books, the other is refused 429 for that same booking

File: backend/lib/booking/book-time.ts:166-168 (the hand-back: backend/routes/public-bookings-routes.ts:68-77,99; the rules: current-feature.md decisions 8 and 9) **Found:** 2026-10-08 by /audit independent (scope: 9.3 review fixes; lens: correctness) **Why it matters:** F-271's fix looks up the form's booking before the limit is asked, which answers a resend sent after the first copy finished. A copy that arrives while the first is still being saved finds no booking yet, so both copies take a place; at the contact's last place the second is refused. Shown with a scratch test in the bookings route tests (removed after, sha256 identical), three runs of three: three bookings for one email, then one form for a fourth sent twice at once answered 201 with the booking and 429 too_many_tries; the same form sent once more afterwards answered 201. Decision 8 says a form sent again after a lost answer "gets its booking and never too many tries", and decision 9 that a retry gets "the booking that won". A retry made while the first request is still running (a slow Google read in the free check) at a contact's fourth booking is told "Too many tries" for a booking that exists. Below the last place the copies are counted once, as "copies of one form arriving together" shows. **Suggested fix:** Count a form once while it is in flight: keep the request keys being booked (per business), and let a copy whose key is already in flight through without a take of its own, so it ends at the constraint or the replay as now and hands back nothing. A route test like the probe above. **Resolution:** Fixed 2026-10-08 in 9.3's second review fixes: the contact limiter remembers which forms (request keys) it counted in each window, and a copy of a form already counted for every key passes with no count of its own (take(keys, formKey) , create-rate-limiter.ts); the route passes the form's request key. Because copies now share one count, a copy answered the form's booking (alreadyBooked) no longer hands it back, or the losing copy would free the place the winning one used; only a refusal or a crash hands it back, and a form handed back is no longer remembered. This replaces the alreadyBooked hand-back of F-271's fix. Tests: "copies of one form at the contact's last place all get the booking" (three bookings, then one form sent twice at once: both 201, one booking, the next form 429) and the unit tests "count once, and a different form is still refused at the limit" and "a form handed back is no longer counted". Mutations: no form key fails both copies route tests; handing back on alreadyBooked fails them in 2 of 3 runs (which copy wins the race decides it); keeping a handed-back form fails its unit test. Spec decision 8 says so. Re-reviewed 2026-10-08 by re-review of 9.3's second fixes (scope: 8e8a864..f99ee0f), left fixed : the defect itself is gone. create-rate-limiter.ts:41-42 lets a copy of a form already counted in every one of the contact's windows pass with taken: [] , and public-bookings-routes.ts:71 passes body.requestKey , so the copy at the last place no longer takes a place of its own. Dropping the form key at :71 fails both copies route tests 3 runs of 3; handing back on alreadyBooked at :101 fails one or both in 2 of 3 runs; deleting window.formKeys.delete(formKey) at create-rate-limiter.ts:65 fails "a form handed back is no longer counted" (1 failed of 9). Files restored, sha256 identical. Not closed because the repair opens a new way past the limit (F-276): it closes with F-276's repair. Fixed again 2026-10-08, after its re-review found F-276: the shared count is gone (create-rate-limiter.ts is back to its 8e8a864 form, no form keys). Instead the route books the copies of one form one after the other (one-copy-of-a-form-at-a-time.ts, keyed by business and request key, in memory like the counts): a copy sent while the first is still saving waits, then is answered that booking by bookTime's look-up before the limit is asked, and the alreadyBooked hand-back of F-271's fix is back. "copies of one form at the contact's last place all get the booking" fails 3 of 3 runs with the turn-taking bypassed. Closed 2026-10-08 by re-review of 9.3's third fixes (scope: f99ee0f..011c991): create-rate-limiter.ts is

its 8e8a864 form again (`git diff 8e8a864 011c991` on it and its test is empty), so no copy passes on another's count. public-bookings-routes.ts:82-83 runs `bookTime` through `oneCopyOfAFormAtATime` keyed `${organizationId}:${requestKey}` (a request key matches `^[A-Za-z0-9_-]{1,64}$`, so no colon lets two businesses' keys meet); a copy that starts after its sibling saved is answered at `book-time.ts:166-167`, before `admitNewBooking` at :168, so it takes no place. Each request keeps its own `taken` and hands back only that (public-bookings-routes.ts:68-78, :99-105). Passing `null` as the form key at :83 fails "copies of one form at the contact's last place all get the booking" 3 runs of 3 (1 failed of 37). Bounded: a slow first copy holds the next only while its `bookTime` runs, and its Google reads abort after 10 s (google-calendar-provider.ts:15, google-oauth-client.ts:13). Only the public route calls `bookTime` (`git grep bookTime`). Full backend suite 875 passed. File restored, sha256 identical.

9/F-276 [P2] closed - A refused copy of a form hands back the one count its sibling copy books on, so the contact gets a fifth booking

File: backend/routes/public-bookings-routes.ts:101 (with backend/lib/rate-limit/create-rate-limiter.ts:41-42,63-65; the rules: current-feature.md decision 8) **Found:** 2026-10-08 by /audit independent (scope: 9.3 second review fixes; lens: correctness, security) **Why it matters:** Copies of one form now share one count, held by whichever copy took it first. When that copy is refused (a booking link that does not exist, a missing answer, a taken time) it hands the count back at :101 and `giveBack` forgets the form (:65), while the other copy, which passed with `taken: []`, goes on to book. A booking is made and nothing is counted for it. The request key is the client's own, so two requests with one key and different contents do it on purpose. Shown with a scratch test in the bookings route tests (removed after, sha256 identical), 3 runs of 3: eight rounds, each a fresh key sent twice at once for one email, one copy with an unknown `bookingLinkId`, the other a valid form, answered `[404,201]` five times then `[429,429]`: five bookings in a window of four. With the valid copy sent 1ms later the same; 3ms later it is counted (four). Before f99ee0f each copy took its own place, so this could not happen; decision 8 says "many sent at once cannot all pass" and only bookings made count. **Suggested fix:** Hand back a shared count only when no copy of the form booked: when the counting copy is refused, look up the form's booking (`bookedByThisForm`, or a hand-back that `bookTime` makes after its own `refuseUnlessBooked`) and keep the count if it exists; or hold the form's count until every copy in flight has finished (a per-form in-flight tally in the limiter), handing back only when the last ends unbooked. A route test like the probe above, shown able to fail. **Resolution:** Fixed 2026-10-08 by replacing F-275's shared count: copies of one form no longer pass on another copy's count; they are booked one after the other, each taking its own count unless the form's booking already exists, so a refused copy hands back only its own. Test: "a broken copy of a form never lets its valid copy book uncounted" (the probe above: eight rounds of a broken and a valid copy of one form sent at once for one email; exactly four bookings made), 3 of 3 runs. Unit tests for one-copy-of-a-form-at-a-time.ts: copies wait for each other, different forms and the owner's bookings do not, a failed copy does not stop the next. Closed 2026-10-08 by re-review of 9.3's third fixes (scope: f99ee0f.011c991): the shared count is gone, so a refused copy can hand back only the place it took itself (`taken` is per request, public-bookings-routes.ts:68-78). Whichever copy runs first: a broken one takes a place, is refused `not_found` (book-time.ts:186) and hands it back before its sibling is admitted (the

sibling starts only after the first's turn ends, and reads the database twice before :168); a valid one books, and the broken one is then answered `request_key_used` at :166 with no count. With f99ee0f's route and create-rate-limiter.ts put back under the current tests, "a broken copy of a form never lets its valid copy book uncounted" fails `expected 5 to be 4` 3 runs of 3 (1 failed of 37); with the current code it passes. Dropping `.catch(() => {})` at one-copy-of-a-form-at-a-time.ts:14 fails "a copy that fails does not stop the next one" (1 failed of 3). F-271 still holds: its two route tests pass, and a resend after the booking is answered before the limit. Its `|| result.alreadyBooked` hand-back (public-bookings-routes.ts:105) no longer bites any test (dropped, 37 of 37 pass 3 runs of 3), since copies in one API now always meet the booking at book-time.ts:166; it stays for a copy that finds the booking later, as decision 8 says. Files restored, sha256 identical.

9/F-277 [P3] closed - Nothing pins that a form's turn is cleared from memory, so a later edit could keep every booked form in the API forever

File: backend/lib/booking/one-copy-of-a-form-at-a-time.ts:19 (its tests: backend/lib/booking/one-copy-of-a-form-at-a-time.test.ts) **Found:** 2026-10-08 by /audit independent (scope: 9.3 third review fixes; lens: tests) **Why it matters:** The `finally` at :19 removes a form's entry from `formsInHand` once its last copy ends; it is the only thing that empties the map, and today it does, on success and on a throw alike. But deleting that line passed every test of the file and of the bookings route (40 of 40, restored after, sha256 identical). Without it each public booking that carries a request key, which the widget always sends (decision 9), leaves its key and its settled answer (the booking and contact ids) in the API's memory until the next restart. The rate limiter's own clean-up is pinned for the same reason ("windows that are over are dropped", through `size()`). **Suggested fix:** Give the module a small read of how many forms are in hand, as the limiter's `size()` does, and a unit test that it is 0 after a copy that books, after one that throws, and after two copies of one form; shown to fail with the line at :19 removed. **Resolution:** Fixed 2026-10-08 in 9.3's fourth review fixes: one-copy-of-a-form-at-a-time.ts exports one object, `oneCopyOfAFormAtATime`, with `book` (the route calls `oneCopyOfAFormAtATime.book`) and `formsInHand()`, the count of forms in hand, as the limiter's `size()`. Test: "every form is cleared from memory once its copies end: booked, failed, or two at once" (0 after a copy that books, 0 after one that throws, 1 while two copies of one form run, 0 after); fails with the clearing line removed (restored after). Backend 77 files, 876 tests pass. Closed 2026-10-08 by re-review of 9.3's fourth fix (scope: 011c991..16a02b3): the `finally` at one-copy-of-a-form-at-a-time.ts:20 still clears a form once its last copy ends, and `formsInHand()` (:25-27) reads the module's map size, as the limiter's `size()` does. Deleting the clearing line fails "every form is cleared from memory once its copies end" at its first check (`formsInHand()` 1, expected 0; 1 failed of 4); file restored, sha256 identical. `git diff -w 011c991 16a02b3` on public-bookings-routes.ts shows only `oneCopyOfAFormAtATime(` becoming `oneCopyOfAFormAtATime.book(` and the `.catch` moved to its own line; the `bookTime` arguments and the hand-back are unchanged, and `git grep oneCopyOfAFormAtATime` finds no other caller. Names and the one-line comment on `formsInHand()` match coding-standards.md. Full backend suite 77 files, 876 passed; `npx tsc --noEmit -p backend` passes.

9/F-278 [P2] closed - A Book now with an empty bookingId calls the service list's route, reads its answer as a service and crashes the host page

File: packages/booking-component/api-client/fetch-booking-service.ts:17-24 (the call: booking-modal.tsx:103-104; the crash: booking-modal/service-screen.tsx:18; the cause: hono's client, node_modules/hono/dist/client/utils.js:12) **Found:** 2026-10-09 by /audit independent (scope: step 9.4, f341569..90a6a1e; lens: all) **Why it matters:** `BookNowTrigger` takes `bookingId?: string`, and the modal treats anything but `undefined` as a named service. Hono's client fills a path parameter without encoding it and drops the segment when the value is empty (`v ? "/" + v : ""`), so `bookingId=""` asks `/public/<slug>/booking-links`, the list route, which answers 200. `fetchBookingService` trusts every 200 as one service and returns `{ state: "ok", service: undefined }`. Shown with a scratch test in the package (removed after) against the running API: the URL built was `http://localhost:3401/public/clinic-dev/booking-links` and the result `{"state":"ok"}` with no service. The modal then renders the service screen, which reads `props.service.layout` on `undefined`, a `TypeError` during render; with no error boundary in the package that takes down the host's page, not only the window. A host passing a blank id from its own content (a `bookingId` field left empty instead of null, the shape `face-and-body's Service.bookingId: string | null` invites) is the realistic way in; an id holding a `/` or `?` likewise reaches another route, though those answer a refusal today. The spec says a service id that is not offered shows "Nothing can be booked online right now." **Suggested fix:** Treat a blank `bookingId` as "nothing to book" (or as no id, opening the list) before any call, encode the path parameters (`encodeURIComponent`) where the package calls the client, and have `fetchBookingService` return "cannot-load" when a 200 carries no `bookingLink`. A unit test of `fetchBookingService` through `hc<PublicAppType>` with a fake fetch: a blank id never reaches the list route, and a 200 without `bookingLink` is a problem, shown able to fail. **Resolution:** Fixed 2026-10-09 in 9.4's review fixes (Frank's yes): `fetch-one-service.ts` (`fetch-booking-service.ts` before the rename in 859b64d) answers "nothing to book" for a blank slug or id before any call, encodes both path values, and reads a 200 with no `bookingLink` as "cannot load"; `fetch-service-list.ts` does the same for a blank slug and a 200 without the business or its list. New tests through `hc<PublicAppType>` over a fake fetch (`fetch-one-service.test.ts`, `fetch-service-list.test.ts`): 5 of them fail on the code before the fix and pass after (restored, sha256 identical); 18 of 18 pass. Live on the booking preview, painting-dev: a blank id shows "Nothing can be booked online right now." with "Call 403 555 0100", and the page stays up. Closed 2026-10-09 by re-review of 9.4's review fixes (scope: 90a6a1e..bc34f31): with 859b64d's `fetch-one-service.ts` and `fetch-service-list.ts` put back under the new tests, 5 of 18 fail (blank id and blank slug ask nothing, a 200 without the expected shape is a problem, "a/b?c" stays one segment); with bc34f31's code 18 of 18 pass; files restored, sha256 identical. A scratch Hono app behind `hc<PublicAppType>` shows the server decodes each value once, so `c.req.param()` gets exactly what the host passed ("a/b?c", "50%", and a literal "a%2Fb" sent as `a%252Fb`), no double encoding; against the running API `clinic-dev` lists 10 services and a real uuid answers its service, while odd ids and slugs ("a/b?c", "..", "clinic-dev/booking-links/x", "clinic-dev?x=1") all come back "nothing to book". The rename leaves no old name anywhere in the repo outside history and findings (`git grep` for each old file and function name), the package's dist holds only the new paths, and `npm run build --workspace=frontend` passes.

9/F-279 [P1] closed - The calendar reads the API's times with the visitor's own time-zone rules, so an older browser shows every Alberta time from Nov 1 an hour early

File: packages/booking-component/booking-window/month-calendar/format-time-of-day.ts:4-7 (also group-times-by-day.ts:9; shown at screens/month-layout/day-times-view.tsx:40 and month-details-screen.tsx:35,50; the same pattern is older in frontend/components/customer-booking/change-time-panel.tsx:44-54) **Found:** 2026-10-09 by /audit independent (scope: step 9.5, c67bc85..2001dc7; lens: all) **Why it matters:** The API works out each free time with its own time-zone rules and sends only the instant (`2026-11-02T15:00:00.000Z`). The component turns that back into a day and a clock time with whatever rules the visitor's browser carries. The two disagree for America/Edmonton, the zone of every dev business and of the four tenants: the API's Node (tz data 2026c) keeps Edmonton on UTC-6 after Nov 1 2026, while Chromium 150 (VS Code's Electron 43 on this laptop, tz data 2025c) falls back to UTC-7. Shown against the running API: clinic-dev's Chemical Peel on Monday Nov 2 (the clinic opens at 9:00) answers 39 times from 15:00Z to 00:30Z; formatted the component's way (`Intl.DateTimeFormat("en-CA", { timeZone, hour, minute })`) Node says "9:00 a.m." to "6:30 p.m." and Chromium 150 says "8:00 a.m." to "5:30 p.m.". Oct 12 reads "9:00 a.m." in both. So from Nov 1, three weeks away, a visitor whose browser predates the change picks "8:00 a.m." and is booked at 9:00, which is what the API, the dashboard and the server-written emails say. The zone line reads "Mountain Time" either way, so nothing on screen shows the mismatch. Decision 11 asks for the business's clock, and the business's clock is the one the API computed with. **Suggested fix:** Let the API name its own clock: the times route sends, beside each instant, the business's date and time of day it computed it from (for example `{ startsAt, date: "2026-11-02", time: "9:00 a.m." }`), or the UTC offset at that instant), and the component groups by that date and shows that label instead of re-deriving both with the browser's rules; the details rail does the same with the chosen time. A unit test that a start whose label the API sent is shown with that label, not re-derived. The 7b change-time panel has the same pattern (only a note, it predates this feature).

Resolution: Fixed 2026-10-09 (Frank: "fix F-279 now"). The times route now names its own clock: beside `startTimes` (kept, every other caller reads it) it sends `localStartTimes: [{ startsAt, date, time }]` , worked out by backend/lib/scheduling/local-start-times.ts with the API's time-zone rules, the same Node the emails are written by. The component groups by the sent `date` and shows the sent `time` (group-times-by-day.ts, day-times-view.tsx), the chosen time carries both to screen two (ChosenTimeType, month-details-screen.tsx), and format-time-of-day.ts is gone, so nothing in the window re-derives a time with the browser's rules. fetch-free-times.ts reads an answer without `localStartTimes` as a problem. Tests: local-start-times.test.ts (3), the route test checks every time comes back with the business's date and a clock label, group-times-by-day.test.ts keeps a Nov 2 9:00 label as sent. Backend 879 of 879, package 41 of 41, frontend build passes. Live: the browser pane is itself Chrome 152 with the old rules (its own Intl says "8:00 a.m." for 2026-11-02T15:00Z in Edmonton); the booking preview for clinic-dev's Chemical Peel shows Monday, November 2 starting at "9:00 a.m.", and screen two reads "9:00 a.m., Monday, November 2". The 7b change-time panel is recorded on its own as F-281. Closed 2026-10-09 by re-review of 9.5's review fixes (scope: 3f204e0..cf87ebe): a grep of packages/booking-component (outside tests and dist) for Intl, toLocale, localDate, new Date and Date.parse finds no start time turned into a date or clock label in the browser; the only browser-zone

reads left are `today` in `month-service-screen.tsx` (a date, never a start time) and the generic zone name ("Mountain Time"), and day names and month names are formatted in UTC from date strings. Screen one groups by the sent `date` (`group-times-by-day.ts`) and shows the sent `time` (`day-times-view.tsx`); screen two shows `chosen.date` and `chosen.time` as carried from screen one (`month-details-screen.tsx`), and `format-time-of-day.ts` is gone with no remaining import. The API's Node here is 26.7.0 with tz data 2026c, and the running API answers clinic-dev's Chemical Peel on 2026-11-02 with 39 times, the first `{ startsAt: "2026-11-02T15:00:00.000Z", date: "2026-11-02", time: "9:00 a.m." }` and the last 2026-11-03T00:30Z grouped under "2026-11-02" at "6:30 p.m.", so the date a UTC-day split would get wrong is also sent right. `fetch-free-times.ts` refuses an answer without `localStartTimes`. Package 43 of 43, backend 880 of 880, `npm tsc -p packages/booking-component/tsconfig.json` clean.

9/F-280 [P3] closed - The guard that drops a slow answer for a month or person no longer shown is pinned by no test

File: `packages/booking-component/booking-window/screens/month-layout/use-month-times.ts:45` (and the first-day choice at `month-service-screen.tsx:84-86`) **Found:** 2026-10-09 by /audit independent (scope: step 9.5, c67bc85..2001dc7; lens: tests) **Why it matters:** `if (!live) return;` is the only thing that stops a late answer for the previous month or person landing on the calendar: a customer who switches from one person to another while the first answer is on its way would see the first person's times under the second one's name, and Next would carry the second person's id with a time they may not have free. The guard is right today, but deleting the line leaves the package's suite green (41 of 41; file restored, sha256 identical), because the hook and the screen's own choices (the first day with a time when a month loads, the picked time cleared on a change of month or person) have no test; the package has no DOM test environment. The project's test gate says a step that adds logic adds its tests, and the spec names stale answers among this step's rules ("Changing the person reloads the month"). **Suggested fix:** Move the two choices into plain functions beside the others in `month-calendar/` (which day is shown for a month's days and the picked date; whether an answer belongs to the question now asked, keyed by from, to and person), use them from the hook and the screen, and test them, each shown able to fail. **Resolution:** Fixed 2026-10-09 the standard way, on Frank's yes to the install: `@testing-library/react` and `jsdom` are dev dependencies of the booking component. `use-month-times.test.ts` (`jsdom`) runs the real hook over the typed client with held answers: Ana asked, Mei picked, Mei's answer lands, then Ana's late one, and the month still shows Mei's times; and it draws `MonthServiceScreen` on Oct 9 with times on Oct 14 and 16, and the shown day is Wednesday, October 14 with its 10:00 a.m., not Oct 16's 9:00 a.m. Proved able to fail: with `if (!live) return;` removed the first fails, with the first day changed to the last the second fails; files restored, sha256 the same. The code itself did not change. Package 43 of 43, `tsc` over the package and its tests clean. Closed 2026-10-09 by re-review of 9.5's review fixes (scope: 3f204e0..cf87ebe): with `if (!live) return;` commented out in `use-month-times.ts`, `npm run test --workspace=@scheduleads-app/booking-component` fails 1 of 43 ("drops a late answer for a person no longer shown"); with the first-day pick in `month-service-screen.tsx` changed to the last day, it fails 1 of 43 ("shows the first day with a time when a month loads"), and changed to today it fails the same test; each file restored from a copy, sha256 identical before and after (`use-month-`

times.ts fa96707a..., month-service-screen.tsx 0430944e...), and the suite is back to 43 of 43. The two new dev dependencies sit only in packages/booking-component's devDependencies; comparing the lockfile's package entries between 3f204e0 and cf87ebe, the only changed existing entry is packages/booking-component (its two new devDependencies), the only removed one is frontend/node_modules/punycode 2.3.1, hoisted to node_modules/punycode at the same 2.3.1 for jsdom, and every added entry is @testing-library/react, jsdom or one of their own dependencies. No root, frontend, backend or shared package.json changed.

9/F-281 [P1] closed - The customer's change-time panel reads the move times with the visitor's own time-zone rules, so an older browser shows every Alberta time from Nov 1 an hour early

File: frontend/components/customer-booking/change-time-panel.tsx:44-54 (the grouping and the labels; the route: GET /public/bookings/:token/times) **Found:** 2026-10-09 while fixing F-279 (the same pattern, noted in F-279's own text; feature 7b, not step 9.5) **Why it matters:** The same mismatch as F-279 on the customer's own booking page: the panel groups and labels the move times with the browser's rules, so from Nov 1 a browser older than tz data 2026c offers "8:00 a.m." for what the API, the dashboard and the emails call 9:00. **Suggested fix:** The move times route sends `localStartTimes` the same way (local-start-times.ts), and the panel shows those. **Resolution:** Fixed 2026-10-09 (Frank: "keep going"). Wider than the panel: the customer's page itself wrote Jane's own time with the browser's rules (customer-booking-screen.tsx:124), and so did the panel's "Move to ...?" question. Now the API writes every one: `localStartTimes` gains `when` (the whole moment as the emails say it, `formatBookingTime` on the server), the move times route sends `localStartTimes` beside `startTimes`, and the booking page answer carries `when`. The panel groups by the sent date, shows the sent clock time and asks "Move to {when}?" ; the page shows `booking.when`. Nothing on the customer's page formats a time any more; only `today` (which week the panel opens on) still reads the browser's date, at most an hour off around midnight. Tests: local-start-times.test.ts checks `when`; the booking page route test expects `when`; a new move-times route test checks every time comes back as `localStartTimes` writes it. Backend 880 of 880, package 41, frontend build and lint clean. Live, in the browser pane with the old rules (its own clock: "8:00 a.m. MST" for 2026-11-02T15:00Z): Riverbend's booking page reads "Friday, October 16 at 9:00 a.m. MDT", the panel's Monday, November 2 starts at "9:00 a.m.", and the question reads "Move to Monday, November 2 at 9:00 a.m. MDT with Ana?". Closed 2026-10-09 by re-review of 9.5's review fixes (scope: 3f204e0..cf87ebe): every route that feeds the page writes its times on the server. GET /public/bookings/:token, POST /public/bookings/:token/move and POST /public/bookings/:token/cancel all answer `{ booking }` from `findBookingPage`, which sets `when` with `formatBookingTime` after the move or cancel has been written, so the page that replaces itself after a move reads the new time's `when`; GET /public/bookings/:token/times adds `localStartTimes` from local-start-times.ts, and its new route test checks the answer equals `localStartTimes(startTimes, timezone)`. In frontend/components/customer-booking/ the panel groups by the sent `date`, shows the sent `time` and asks "Move to {when}?", and the screen shows `booking.when` for both "When" and "It's now ..."; a grep of frontend/app, components and lib for Intl, toLocale, `formatBookingTime` and `localDate` finds on that page only `today` (a date) and day

and week names formatted in UTC from date strings. The booking page route test expects `when` exactly. Backend 880 of 880, `npm run build --workspace=frontend` and the frontend lint pass. Left as a note only: the move answer's `when` following the new time is pinned by no test of its own, but it comes from the same `findBookingPage` the page route test pins.

9/F-282 [P2] closed - After a lost answer the form stays live: Back and another time get "already used, reload and book again" while she is in fact booked, so following the words books her twice

File: `packages/booking-component/booking-window/screens/booking-form-view.tsx:103-104` (the no-answer case) and `booking-window/booking-window.tsx:213-216` (Back keeps the form and its key); the words come from `backend/routes/public-bookings-routes.ts:136-142` **Found:** 2026-10-09 by /audit independent (scope: step 9.6, a946115..00bde08; lens: correctness) **Why it matters:** A lost answer means the booking may have been made, and decision 9's guard only holds if the next send is the same form. But after "We couldn't reach the booking just now" the window still offers Back to the times and every field stays editable, and the key is kept across Back. Shown live on the booking preview with clinic-dev's Chemical Peel: the first Book was let through to the API and its 201 dropped in the page (booking 970826ae, Saturday, October 10 at 2:30 p.m., Ana, now in the dev database); the window showed the lost-answer words with Back, Try again and Book; Back, 3:30 p.m., Next, Book answered 409 and the window showed "This booking form was already used. Please reload the page and book again." with no Try again. Nothing tells her she is booked at 2:30; doing what the words say (reload, or close and open again, which makes a new key) books a second time, and the contact limit lets up to four through. The same with curl: one key at 15:00 answers 201, again at 15:00 answers the same booking, the same key at 16:00 answers 409 `request_key_used`. A smaller form of the same gap: an email edited after a lost answer is sent with Try again, the API answers the earlier booking (the key matches on the time only), and the done screen says "A confirmation is on its way to" the new address while the confirmation went to the one first saved. **Suggested fix:** Treat a form whose last send got no answer as in doubt until a send gets one: while it is, offer only Try again (no Back, fields read only), or have the window resend the earlier choice before anything else. And make "already used" useful: the route's `request_key_used` answer carries the booking that form made (its `when`, service and person, never the customer's details), and the window shows it as "You're already booked" instead of telling her to book again. A test for each: a lost answer leaves no way to send a different time with the same key, and a `request_key_used` answer shows the booking made. **Resolution:** Fixed 2026-10-09 in 9.6's review fixes, as the plan's "a lost connection (the choice kept, Try again with the same key)" means. After no answer the form freezes until an answer comes: the fields sit in one fieldset, disabled; Book is hidden; screen two hides its Back (`month-details-screen.tsx` holds `unsure` from the form's `onUnsureChange`); only Try again and the phone remain, and Try again sends the frozen form with its key, so a booking made unseen is answered with that booking. The words now say it: "We couldn't hear back about your booking. It may have gone through: press Try again to find out. It never books twice." The done screen names the address the sent booking carried. Test: `booking-form-view.test.ts` "after a lost answer changes nothing until one comes" (fields disabled, no Book, Back hidden, Try again sends an identical body, then the booking); fails with the fieldset left live.

Live on the preview page: the first Book reached the API and its 201 was dropped in the page; the form froze with no Back and no Book; Try again showed "You're booked" for Wednesday, October 21 at 9:00 a.m. MDT, and the database holds one booking for that email. Closing the window still starts a new form, as the plan says. Closed 2026-10-09 by re-review of 9.6's review fixes (scope: 134b281..4664807): after a send with no answer, booking-form-view.tsx sets `unsure`, the fieldset is disabled (so no field takes Enter and no box can change), the Book button is not drawn (so the form has no submit path), MonthRail gets `back: null`, and the only control left is Try again (type="button"), which re-reads the unchanged form and sends the identical body with the same key. Every answer the route itself writes to that resend is right to unfreeze on, because book-time.ts looks the key up before any refusal: the same key and time answer the earlier booking (201, alreadyBooked); time_taken and the route's 503 go through refuseUnlessBooked, so they mean this key booked nothing; request_key_used cannot come back for the same time; the 400s and the contact limit's 429 come after the look-up; another lost answer keeps it frozen. Breaking it fails the suite: the fieldset left live, Book drawn while unsure, and `onUnsureChange(false)` each fail "after a lost answer changes nothing until one comes" (files restored, sha256 identical). Two gaps remain and are recorded as their own findings: answers not written by the route (the per-visitor 429, a gateway 5xx) also unfreeze it (F-286), and the screen shows nothing while the resend is on its way (F-287); the hidden Back itself is pinned by no test (F-288).

9/F-283 [P3] closed - The key's life in the window and the done screen's server-written time are pinned by no test

File: packages/booking-component/booking-window/booking-window.tsx:41,210,221-225; booking-window/screens/month-layout/month-done-screen.tsx:45; booking-window/booking-form/read-booking-form.ts:73-86,100 **Found:** 2026-10-09 by /audit independent (scope: step 9.6, a946115..00bde08; lens: tests) **Why it matters:** The spec's Testing names "the request key's lifetime" for the package, and the step's claims are one key per opening, kept across a taken time, and the done screen never formatting a time in the browser. The new tests pin the key only inside BookingFormView with its own holder, never the window that owns it. Each of these breaks left the package suite green, 61 of 61 (files restored, sha256 identical): onTimeTaken also calling `setForm(newBookingForm())` (a new key after a taken time); passing `form={{ ...form, requestKey: crypto.randomUUID() }}` to DetailsScreen (a new key on every render, so a Try again after a lost answer could book twice); the done screen showing `new Date(booking.startsAt).toLocaleString()` instead of `booking.when`. In read-booking-form.ts, dropping the sort into form order, and mapping every answer error to the first answered question, also left it green: the focus goes to the first wrong field only because of that sort (a 501-character answer to question two with required question one empty is the case it decides). **Suggested fix:** A jsdom test of BookingWindow over the held-answers client: open, pick, Book answered time_taken, pick again, Book, and the two sends carry one key; close, open again, and the next send's key differs; the done screen shows the `when` it was sent. Two cases for readBookingForm: the order above, and a too-long answer to the second answered question tied to that question. Each shown able to fail. **Resolution:** Fixed 2026-10-09: booking-window.test.ts (jsdom, with open and close standing in for <dialog>) drives the whole window: 10:00 picked, details typed, Book answered 409 time_taken, back on her day without

10:00, 11:00 picked, her name still there, Book answered 201; both sends carry the same key and the done screen shows the API's `when` text as sent. `read-booking-form.test.ts` adds "ties each answer's error to its own question, in the business's order". Proved able to fail: a new form on a taken time, the done screen formatting `startsAt` in the browser, the errors reversed, and an answer's error tied by index into all questions each failed a test; files restored, sha256 the same. Re-review 2026-10-09 of 9.6's review fixes (scope: 134b281..4664807), left open: most of it holds, one named break still passes. Holding, each break run with `npm run test --workspace=@scheduleads-app/booking-component` and each file restored from a copy with sha256 identical: `onTimeTaken` also calling `setForm(newBookingForm())`, `form={{ ...form, requestKey: crypto.randomUUID() }}` given to `DetailsScreen`, and the done screen showing `new Date(booking.startsAt).toLocaleString()` each fail "keeps the form and its key across a time taken" (`booking-window.tsx` 97ba7800..., `month-done-screen.tsx` 2affb87e...); dropping the sort into form order, and `fieldOf(issue.path, questions)` (an answer tied by index into all questions), each fail "ties each answer's error to its own question" (`read-booking-form.ts` 4aa84208...). Not holding: the finding's own named break, every answer error mapped to the first answered question (`answered[second]` changed to `answered[0]`, and also to the last answered), leaves the suite green, 65 of 65, because the new tie test answers only one question (q-colour is empty, so the too-long q-pets is both `answered[0]` and `answered[second]`). With two answers, "Blue" to question one and 501 characters to question two, the error would show under question one and focus would land there. Needed: a case with both questions answered and the second too long, its error under the second. Also not pinned, which the suggested fix asked for: a new key for each opening (`useState(newBookingForm)` changed to one form kept across openings leaves 65 of 65 green); the window is mounted only while open, so today it holds by structure. Second fix 2026-10-09: the re-review's gap is pinned. `read-booking-form.test.ts` "ties a too-long answer to its own question when every question is answered" fails with every answer error on `answered[0]`; `booking-window.test.ts` "gives each opening of the window its own form key" fails with one key shared by every opening (a module-level form). Files restored, sha256 the same. Second re-review 2026-10-09 of 9.6's review fixes (scope: 8f28143..379d886), left open: both gaps the second fix names are pinned, one tie break the first re-review also named still passes. Holding, each run with `npm run test --workspace=@scheduleads-app/booking-component` and the file restored from a copy with sha256 the same: `answered[second]` changed to `answered[0]` fails "ties a too-long answer to its own question when every question is answered" (`read-booking-form.ts` 4aa84208...); `useState(newBookingForm)` changed to `useState(sharedForm)` with one `sharedForm = newBookingForm()` at module level fails "gives each opening of the window its own form key" (`booking-window.tsx` 97ba7800...). Not holding: `answered[second]` changed to `answered[answered.length - 1]` (every answer error on the last answered question, the "and also to the last answered" of the first re-review) leaves the suite green, 70 of 70, because in both tie tests the too-long answer is to q-pets, which is always the last one answered. With 501 characters to q-colour and an answer to q-pets, the error would show under q-pets and focus would land there. Needed: one case with both questions answered and the first one too long, its error under the first. Third fix 2026-10-09: the tie test now also has the first of two answered questions too long, so neither end of the list can stand in; it fails with the last answered question used for every error. Closed 2026-10-09 by the third re-review of 9.6's review fixes (scope: 0db1191..70097a3): "ties a too-long answer to its own question when every question is answered" now

has both cases, 501 characters to q-pets with q-colour answered, and 501 characters to q-colour with q-pets answered. Both ends of the list fail it, each run with `npm run test --workspace=@scheduleleads-app/booking-component` and `read-booking-form.ts` restored from a copy, sha256 4aa84208... before and after: `answered[second]` changed to `answered[answered.length - 1]` gives 1 failed, 69 passed, and changed to `answered[0]` gives 1 failed, 69 passed, both on that test. The earlier holds (the key kept across a taken time, a key per opening, the done screen's `when`, the sort) were proved in the two re-reviews before and this commit does not touch them.

9/F-284 [P3] closed - The browser check lets a form through when the schema fails on a part no field shows, such as more than 20 answers

File: packages/booking-component/booking-window/booking-form/read-booking-form.ts:56-72 **Found:** 2026-10-09 by /audit independent (scope: step 9.6, a946115..00bde08; lens: correctness) **Why it matters:** `readBookingForm` returns `ok` whenever no issue maps to a field, even when `createBookingValidationSchema.safeParse` failed. Probed with a temporary test (removed): a business with 21 optional questions, all answered, gives `state: "ok"`, and the route refuses it 400 "That is too many answers." with no Try again and nothing she can change on the form, since the box that caused it is not named. Nothing caps a business's questions (`booking_question` has no count rule, and feature 12 will let owners add them). The same path would send a bad `startsAt` or `id`, though those come from the API today. **Suggested fix:** When the parse fails and no field took the issue, do not send: show the issue's words in the form's problem line (with the phone). Separately, cap a business's questions at the schema's 20 where they are written, so the form can always be sent. **Resolution:** Fixed 2026-10-09: a schema issue no field shows is now the form's own error (`field: "form"`, last in the order), shown with the business's phone and no Try again, and nothing is sent. Tests: `read-booking-form.test.ts` (21 answered questions give "That is too many answers.") and `booking-form-view.test.ts` (the alert shows and no send happens); both fail when such an issue is dropped. How many questions a business may have is item 12's (the plan already says up to about 20). Closed 2026-10-09 by re-review of 9.6's review fixes (scope: 134b281..4664807): `read-booking-form.ts` now maps every schema issue with no field to `"form"` (`fieldOf(...) ?? "form"`), last in the order, so `state: "ok"` is returned only when the shared schema passes; `book()` in `booking-form-view.tsx` returns before `send()` on any error and shows the form error through `ProblemMessage` with `retry: false`, which draws the business's phone when it has one. The customer's own schema refine (email or phone) carries `path: ["email"]`, so it still lands under Email and not on the form. Breaking it fails the suite: putting back `if (field && ...)` fails both new tests, and `setProblem(null)` in place of the form error fails the view test (files restored, sha256 identical).

9/F-285 [P3] closed - Two new code comments carry finding numbers again

File: backend/routes/public-bookings-routes.ts:185; packages/booking-component/api-client/send-booking.ts:34 **Found:** 2026-10-09 by /audit independent (scope: step 9.6, a946115..00bde08; lens: quality) **Why it matters:** `coding-standards.md`, Comments: "No history in code comments (step numbers, finding numbers ...): that lives in the build log." F-274 removed them once; this step adds "(F-279)" and "

(F-278)". The ledger is archived at /complete and the next one starts again from a low number, so the IDs go stale. The same pattern sits in six lines from 9.5's review fixes (find-booking-page.ts:26, local-start-times.ts:4, public-booking-links-routes.ts:112, change-time-panel.tsx:45, customer-booking-screen.tsx:123, and the three fetch-*.ts files of the package), outside this step's scope. **Suggested fix:** Drop the IDs and keep each sentence. **Resolution:** Fixed 2026-10-09: the finding numbers are gone from all ten comments the finding lists (the two new ones and the eight from 9.5's fixes); each sentence kept. `git grep` for an F-number in a code comment outside tests now finds nothing. Closed 2026-10-09 by re-review of 9.6's review fixes (scope: 134b281..4664807): the commit's diff in backend, frontend and the package changes exactly ten comment lines (find-booking-page.ts, local-start-times.ts, public-booking-links-routes.ts, public-bookings-routes.ts, change-time-panel.tsx, customer-booking-screen.tsx, fetch-free-times.ts, fetch-one-service.ts, fetch-service-list.ts, send-booking.ts), each only losing its "(F-27x)" with the sentence kept. `git grep -n -E "\bF-[0-9]+"` over code files outside tests, blueprint and the skills finds nothing, and a looser case-blind grep for F followed by two or three digits in frontend, backend and packages (tests, JSON, SQL and Markdown left out) finds nothing either.

9/F-286 [P3] closed - After a lost answer, a 429 from the visitor limit or a gateway's 5xx unfreezes the form, though neither says whether she is booked

File: packages/booking-component/booking-window/screens/booking-form-view.tsx:100-102 (any answer but no-answer clears `unsure`); backend/middleware/public-middleware/public-rate-limit-middleware.ts:12-17 and backend/app.ts:35 (the visitor limit answers before the route); packages/booking-component/api-client/send-booking.ts:72-75 **Found:** 2026-10-09 by re-review of 9.6's review fixes (scope: 134b281..4664807; lens: correctness) **Why it matters:** F-282's freeze ends on any HTTP answer. Every answer the route writes is safe to end it on, because book-time.ts asks the key first, but two kinds of answer never reach that look-up: the per-visitor write limit (10 in 10 minutes, a middleware mounted on /public/* before the routes, which counts every Try again that reached the server even when its answer was lost, and is shared by everyone behind one address), and a 500, 502 or 504 not written by the route. Shown with a temporary jsdom test of BookingFormView (removed): Book, no answer, Try again answered 429 gives `unsure` [true, false], the Email box enabled, Book drawn and "Too many tries. Wait a few minutes."; answered 502 or 504 gives the same with "Booking couldn't load right now.". The month screen then shows Back again, so Back, another time and Book gets 409 request_key_used, "This booking form was already used. Please reload the page and book again.", the exact path F-282 closed; and an email edited then sent with Try again gets the earlier booking with the done screen naming the new address. A 500 thrown after the booking was saved (the names look-up in public-bookings-routes.ts) never freezes the form at all. Rare: it needs a lost answer and then one of these. **Suggested fix:** Keep the form in doubt unless the answer settles the key: a 201, a 409, or a 400 or 503 the route wrote (its JSON `error.code`). A 429, a 5xx, or a body that is not the route's keeps it frozen with Try again and the phone, in words that say she may already be booked. A test for the 429 and for a 502 after a lost answer. **Resolution:** Fixed 2026-10-09: only an answer that settles what became of this form unfreezes it: a booking, a taken time, or the route's own refusal (400, 409 request_key_used, the route's 503 "unavailable"). A server fault, a proxy's page, a 201 without its booking (all read as cannot-

load), and "too many tries" freeze the form, or keep it frozen, with the lost-answer words and Try again. send-booking.ts reads a 503 as the route's only when it carries code "unavailable". Tests: booking-form-view.test.ts "stays frozen on an answer that does not settle the form" (a 500 freezes, a 429 keeps it frozen, the route's 400 frees it), booking-window.test.ts "hides Back to the times while a lost answer is unknown" (a 502 freezes, the route's 409 time_taken frees it), send-booking.test.ts "reads a 503 the route did not write as unclear". Breaking the rule, or the 503 check, fails them. Second re-review 2026-10-09 of 9.6's review fixes (scope: 8f28143..379d886), left open: the freeze rule holds, the words do not. Holding: every answer that frees the form is one the route writes after the key's look-up, or one the identical resend would also have got the first time. The 201 with its booking comes from the look-up in book-time.ts or a new booking; 409 time_taken and the 503 "unavailable" both go through refuseUnlessBooked; 409 request_key_used is the look-up itself; the 400s person_not_taken, unknown_question, answered_twice and answer_needed come after it; the validator's, the JSON parse's and the slug's 400s come before it but are fixed by the body, which Try again sends unchanged (the shared schema has no rule that reads the clock). The visitor limit's and the contact limit's 429 (one body, so the form cannot tell them apart), the 404s (two of them are written before the look-up: findBookableOrganizationId and resolveBookableHours), a 413, and any 5xx or proxy page keep it frozen. No dead end: while unsure every problem keeps Try again and the phone, and both 429 windows end within 10 minutes. Breaking it fails the suite (files restored from a copy, sha256 the same): the old rule (`nowUnsure = answer.state === "no-answer"`) and `false` in place of the cannot-load check each fail "stays frozen on an answer that does not settle the form" and "hides Back to the times while a lost answer is unknown"; `nowUnsure = unclear` and dropping "refused" from `settled` each fail the first (booking-form-view.tsx f840f9fa...); dropping the 503's code check fails "reads a 503 the route did not write as unclear" (send-booking.ts b339afb5...). Not holding: the resolution says "too many tries" keeps the lost-answer words, but booking-form-view.tsx:124 gives them only to cannot-load. Shown with a temporary jsdom test (removed): Book, no answer, Try again answered 429 shows "Too many tries. Wait a few minutes." with the fields still disabled; Try again then answered 404 shows "Nothing can be booked online right now." beside Try again. Both drop "It may have gone through", which the suggested fix asked to keep, so a customer who gives up and opens the window again gets a new key and can be booked twice once the limit's window ends; and the 404 words say nothing can be booked while offering Try again. The new view test never checks the words after the 429, which is how the claim passed. Needed: while unsure, every problem's words say she may already be booked (a reason after it is fine), and a test that checks them after the 429. Third fix 2026-10-09: while unsure every problem keeps the lost-answer words; "too many tries" says "Too many tries. Wait a few minutes, then press Try again to find out whether your booking went through. It never books twice." The test now checks those words after the 429; it fails with the plain limit words. Closed 2026-10-09 by the third re-review of 9.6's review fixes (scope: 0db1191..70097a3): in booking-form-view.tsx a problem shown while `nowUnsure` takes `tooManyWhileUnsureWords` for "too many tries" and the lost-answer words for the other two kinds ("cannot-load" and "nothing-to-book", the only three in problemWords), always with Try again and the phone; a problem shown when not unsure keeps its plain words. Shown with a temporary jsdom test (removed): Book, no answer, Try again answered 429 shows "...to find out whether your booking went through. It never books twice."; Try again answered 404 now shows "...It may have gone through: press

Try again to find out. It never books twice." beside Try again, not "Nothing can be booked online right now."; a 429 on a first Book, not unsure, still shows "Too many tries. Wait a few minutes." Breaking it fails "stays frozen on an answer that does not settle the form, and says it is checking" (1 failed, 69 passed), file restored from a copy, sha256 f5180dd4... before and after: the old rule (`answer.problem === "cannot-load" ? noAnswerWords : problemWords[answer.problem]`) and `tooManyWhileUnsureWords` swapped for the plain limit words each fail it. Not pinned by a test, only noted: the 404 while unsure, which the code gives the lost-answer words.

9/F-287 [P3] closed - While Try again resends after a lost answer, screen two shows nothing at all: no button, no words, no sign it is sending

File: packages/booking-component/booking-window/screens/booking-form-view.tsx:96

(`setProblem(null)` at the start of each send) and :260 (Book drawn only when not `unsure`) **Found:** 2026-10-09 by re-review of 9.6's review fixes (scope: 134b281..4664807; lens: correctness) **Why it matters:** Pressing Try again clears the problem line, and Book, which carries "Booking..." and `aria-busy` , stays hidden because `unsure` changes only when the answer comes. Shown with a temporary jsdom test (removed): with the resend held, the form has no button, no alert and no `aria-busy` element, only the disabled fields; the month screen has no Back either. That can last until the 10-second time limit. The focus was on Try again, which is removed, so it falls out of the form, and a screen reader hears nothing. A customer who has just been told her booking may have gone through sees a frozen screen; the one thing left to press is Close, and opening the window again makes a new form and key, the second booking F-282 was about. **Suggested fix:** While a resend is on its way, keep a line that says so (for example "Checking your booking..." with `role="status"` or the problem area's own busy state) and keep the focus inside the form; a test that the held resend shows it. **Resolution:** Fixed 2026-10-09: while Try again resends, "Checking your booking..." shows as a status in place of the button. Pinned by booking-form-view.test.ts "stays frozen on an answer that does not settle the form, and says it is checking"; fails with the line removed. Second re-review 2026-10-09 of 9.6's review fixes (scope: 8f28143..379d886), left open: the status line holds, the focus does not. Holding: while the resend is held, "Checking your booking..." shows with `role="status"` ; making its condition always false fails "stays frozen on an answer that does not settle the form, and says it is checking" (booking-form-view.tsx f840f9fa..., restored from a copy, sha256 the same). Not holding: the suggested fix's other half, keep the focus inside the form. Shown with a temporary jsdom test (removed): with focus on Try again, pressing it leaves `document.activeElement` on BODY while the resend is held, and still on BODY once a 429 brings Try again back. A keyboard user has to tab from the top of the window to reach Try again each time, and the focus point is lost the moment she presses. Needed: focus kept in the form while it checks, for example Try again left in place with `aria-disabled` and `aria-busy` so focus never leaves it, or moved to the status line and back to Try again when it returns; a test that the focus stays inside the form. Third fix 2026-10-09: focus stays in the form while unsure, on the checking line while Try again waits and on the words once an answer keeps it unsure (both focusable, never outlined). The test checks focus is on the status while resending and inside the alert after the 429; it fails with the focus moves removed. Third re-review 2026-10-09 of 9.6's review fixes (scope: 0db1191..70097a3), left open: the code holds, half of it

is pinned by no test. Holding, shown with a temporary jsdom test (removed): after a lost answer the focus is on the `.sa-focus-place` div around the words, while Try again resends it is on the "Checking your booking..." status, and after a 429 that keeps it unsure it is back on that div, never on BODY. Nothing steals the focus when the form is not unsure: a first Book answered with a refusal or a 429 leaves it on Book, and a field error still puts it on the first wrong field (the effect returns while not `unsure`). The new `.sa-focus-place:focus { outline: none; }` follows `.sa-title:focus`, and no wider focus rule in booking-component.css outlines it. The focus on the status is pinned: dropping `checkingRef.current?.focus()`, or the whole effect, fails "stays frozen on an answer that does not settle the form, and says it is checking". Not holding: dropping `problemRef.current?.focus()` (the move onto the words once an answer keeps it unsure) leaves the suite green, 70 of 70 (booking-form-view.tsx restored from a copy, sha256 f5180dd4... before and after). The test's check, `alert.contains(document.activeElement) || document.activeElement?.contains(alert)`, also passes with the focus on BODY, because the page's body contains the alert. Needed: a check that fails on BODY, for example that `document.activeElement` is the `.sa-focus-place` wrapping the alert (or is not `document.body`), after the 429. Fourth fix 2026-10-09: the focus test now asks for the words' own place (`.sa-focus-place` around the alert), which the page never is; it fails with the words' focus removed. Closed 2026-10-09 by the fourth re-review of 9.6's review fixes (scope: 27d2e69..a17840e): booking-form-view.test.ts now checks `document.activeElement` is the `.sa-focus-place` wrapping the alert after the 429 (line 173) and after the route's 400 (line 189), a strict identity check that BODY can never pass. Breaking it fails the suite (booking-form-view.tsx copied aside and restored, sha256 409bcd61... before and after every break): dropping `problemRef.current?.focus()` fails "stays frozen on an answer that does not settle the form, and says it is checking" at line 173, focus on BODY (1 failed, 69 passed); dropping `checkingRef.current?.focus()` fails it at line 163. Both halves of the suggested fix are now pinned: the status while Try again waits, and the words once an answer keeps the form unsure.

9/F-288 [P3] closed - Screen two's hidden Back during a lost answer is pinned by no test

File: packages/booking-component/booking-window/screens/month-layout/month-details-screen.tsx:53,60 **Found:** 2026-10-09 by re-review of 9.6's review fixes (scope: 134b281..4664807; lens: tests) **Why it matters:** F-282's resolution says its test covers "Back hidden", but booking-form-view.test.ts only records the `onUnsureChange` calls. With the rail's `back` given in both cases (`unsure ? { label, onBack } : { label, onBack }`), the package suite stays green, 65 of 65 (file restored, sha256 37a96fa4... identical). Back is the path F-282 closed; the project's test gate says logic a step adds gets its tests. **Suggested fix:** In booking-window.test.ts (which already drives the whole window), a Book answered with no answer, then no "Back to the times" button until Try again is answered; shown able to fail with the line above. **Resolution:** Fixed 2026-10-09: booking-window.test.ts "hides Back to the times while a lost answer is unknown, and shows it again once settled" checks the rail's Back is drawn, gone after a 502, and the window back on screen one once the route settles it; fails with Back always drawn in month-details-screen.tsx. Closed 2026-10-09 by the second re-review of 9.6's review fixes (scope: 8f28143..379d886): booking-window.test.ts "hides Back to the times while a lost answer is unknown, and shows it again once settled" drives the real window: Back drawn on screen two, gone after a 502, and the route's 409 time_taken brings screen one back. The finding's own break, the rail's `back` given in both

cases in month-details-screen.tsx, fails it (1 failed, 69 passed; sha256 37a96fa4... before and after the restore, identical). details-screen.tsx only renders MonthDetailsScreen, so this is the one Back on screen two.

9/F-289 [P3] closed - After a lost answer, a refusal that settles the form drops the focus to the page

File: packages/booking-component/booking-window/screens/booking-form-view.tsx:71-75 (the focus moves run only while `unsure`) and :290-294 (the checking line that holds the focus goes when the answer comes) **Found:** 2026-10-09 by the third re-review of 9.6's review fixes (scope: 0db1191..70097a3; lens: correctness) **Why it matters:** F-287's fix holds the focus on "Checking your booking..." while Try again resends. When that resend is answered with the route's own refusal (a 400 such as "Answer: Which colour?", 409 request_key_used, or the route's 503), the form unfreezes, the checking line is removed and the effect returns because `unsure` is now false, so the focus falls to BODY. Shown with a temporary jsdom test (removed): Book, no answer, Try again answered 429, Try again answered 400 "Answer: Which colour?" leaves `document.activeElement` on BODY, with Book drawn and the fields enabled. The words are read out as an alert, but a keyboard user who now has to change a field or press Book again starts from the top of the window. The other two settling answers are fine: a booking and a taken time change the screen, and booking-window.tsx moves the focus to the new screen's heading. Rare: it needs a lost answer and then a refusal. **Suggested fix:** When an answer settles a form that was unsure and leaves it on screen two with a problem, move the focus onto the problem's words (the same `sa-focus-place` wrapper), so it never lands on the page; a test that checks the focus after that refusal is not BODY. **Resolution:** Fixed 2026-10-09: focus goes to the words whenever a send ends with words on screen, a refusal that settles the form included (a counter bumped with each set of words drives it), and to the checking line while Try again waits. booking-form-view.test.ts checks, after a lost answer, Try again and the route's 400 "Answer: Which colour?", that the words hold the focus; with the old rule (focus only while unsure) it fails there, focus on the page. Closed 2026-10-09 by the fourth re-review of 9.6's review fixes (scope: 27d2e69..a17840e): in booking-form-view.tsx every send that ends with words goes through `showWords`, which bumps `wordsShown`, and that effect focuses the `sa-focus-place` div around the alert; the checking line's effect now runs only while unsure and sending. Shown with temporary jsdom tests (removed): Book, no answer, Try again answered 409 request_key_used leaves the focus on the words, not BODY; a first Book refused, or answered 429, moves the focus from Book onto the words; while Try again waits after a lost answer the focus is on "Checking your booking...". Nothing steals the focus wrongly: a field error still focuses the first wrong field (Name), also after earlier words had the focus, because it never bumps `wordsShown`; the "too many answers" form error, which sends nothing, shows its words as an alert and leaves the focus on Book, never the page; in the whole window a booking or a taken time after a lost answer moves the focus to the new screen's heading ("You're booked", "Select a date & time"). Breaking it fails the suite (booking-form-view.tsx copied aside and restored, sha256 409bcd61... before and after): the old single effect (`if (!unsure) return;` then the checking line or the words) fails "stays frozen on an answer that does not settle the form, and says it is checking" at line 189, focus on BODY; the refusal shown with `setProblem`

in place of `showWords` fails it at line 189 too. Only a note: the comment over `.sa-focus-place:focus` in `booking-component.css` still says "while a lost answer is unknown", though the place now takes the focus for any words after a send. One gap left outside this finding is F-290.

9/F-290 [P3] closed - Try again outside a lost answer drops the focus to the page while it sends

File: `packages/booking-component/booking-window/screens/booking-form-view.tsx:73-75` (the checking line, and its focus, only while `unsure`) and `:114 ( setProblem(null)` removes Try again, which had the focus) **Found:** 2026-10-09 by the fourth re-review of 9.6's review fixes (scope: 27d2e69..a17840e; lens: correctness) **Why it matters:** The new comment says focus never falls to the page, but Try again is also shown when the form is not unsure: after a first Book answered "too many tries", or the route's 503 "unavailable" (`canRetry` true). Pressing it clears the problem, so Try again and its focus go, and the checking line is not drawn because `unsure` is false. Shown with a temporary jsdom test (removed): a first Book answered 429 puts the focus on the words; with focus on Try again, pressing it leaves `document.activeElement` on BODY while the resend is held, with only "Booking..." (disabled, not focused) on screen. The focus comes back once the answer arrives (the words, or the next screen's heading), so it is the same gap F-287 closed for a lost answer, for up to the 10-second time limit: a keyboard user's place is lost the moment she presses, and a screen reader hears nothing while it sends. Rare: it needs a limit reached or the route's 503 first. **Suggested fix:** Keep the focus in the form whenever Try again resends, not only while unsure: for example move it to the "Booking..." button, or show the checking line (or a plain "Booking..." status) and focus it while any resend from Try again waits; a test that the focus is not BODY while a Try again after a first 429 is held. Then the comment's "never falls to the page" holds as written. **Resolution:** Fixed 2026-10-09: any send from Try again draws a waiting line in place of the Book button and moves the focus onto it until the answer ("Checking your booking..." while unsure, "Booking..." otherwise), so Try again removing itself never drops the focus to the page. The comment over `.sa-focus-place:focus` now says it holds the focus while Try again sends and on the words a send ends with. `booking-form-view.test.ts` checks that after a first Book answered 429, Try again shows "Booking..." as a status holding the focus, with no second "Booking..." button; with the old form view it fails there, no status on screen. Closed 2026-10-09 by the fifth re-review of 9.6's review fixes (scope: 9f82c69..bbb6880): in `booking-form-view.tsx` Try again calls `book(true)`, `send` sets `retrying` for that send only (cleared when the answer comes), and `waiting` (sending and unsure or retrying) draws the status line in place of Book and moves the focus onto it. Shown with temporary jsdom tests (removed), each pressing Try again with the focus on it: after a first Book answered 429, and after the route's 503 "unavailable", the focus is on the "Booking..." status while the resend is held, never BODY, with no Book or "Booking..." button beside it, and the same key sent; a second 503 puts the focus back on the words with Book drawn, and a third try books. Nothing else broke: a first Book, and a Book after a retried 429, still show the disabled "Booking..." button and no status (`retrying` does not stick); Try again after Address was emptied sends nothing and focuses Address, Book drawn; a retry from a 429 that loses its answer turns unsure, and the next Try again shows "Checking your booking..." (one status only), with a 400 refusal then focusing the words; a time taken after such a retry is handed on; in the whole window a

booking or a taken time after Try again from a 429 sends the same key and moves the focus to the new screen's heading. Breaking it fails the suite (booking-form-view.tsx copied aside and restored, sha256 59bfe7d7... before and after): the 9f82c69 form view fails "keeps the focus in the form while Try again resends after a limit reached" (1 failed, 70 passed, no status on screen), and the focus move limited to unsure (`if (waiting && unsure)`) fails it at line 208. Package suite 71 of 71.

9/F-291 [P3] closed - A tick from anyone who types her email moves her yes to their number

File: backend/lib/booking/book-time.ts:300-306 (the contact's yes and number are set from this form's phone whatever was there) and backend/lib/crm/find-or-create-contact.ts:56-80 (a contact is matched by email alone) **Found:** 2026-10-09 by the review of step 9.7 (scope: cfc3dbc; lens: data integrity) **Why it matters:** The contact holds one yes and one number. A later booking with her email and a different phone, ticked, overwrites both, so her yes for her own number is gone from the contact. Since the public form has no login, anyone who types her email can do it. Shown with a temporary route test (removed) on a business that asks: Jane books with her email and (403) 555-0148, ticked, and the contact holds +14035550148; a second booking with the same email, name "Someone Else" and (587) 555-0177, ticked, leaves the contact with `laterTextsYesPhone` +15875550177 while its `phone` is still (403) 555-0148. Decision 13 says an unticked box never takes back an earlier yes; a tick for another number does, and the same happens to the real customer when she books once from a second phone. Nobody is texted without a yes (the new number was typed by whoever ticked), and the timeline still holds her first entry with its number, so nothing is lost for good, but whatever later reads the contact to choose who may get later texts (feature 12 onward) sees her as not having said yes for her own phone. **Suggested fix:** Keep the yes per number rather than one per contact (for example a small table of contact, number and date, one row per number, its date moved on each new tick), or, keeping the two columns, move them only when the ticked number is the contact's own phone or the contact has no yes yet, and otherwise record only the timeline entry. Add a route test: a second ticked booking with her email and another phone leaves her yes for her own number in place. **Resolution:** Fixed 2026-10-09, Frank chose one yes per number: the contact's two columns became the `later_texts_yes` table, one row per business, contact and number (as Twilio texts it), dated by the latest yes for that number, its contact kept by a foreign key to the same business. A tick upserts its own number's row and never touches another number's; no tick changes nothing. Migration 0024_later_texts_yes regenerated (never deployed). `public-bookings-routes.test.ts` checks that a tick with her email and another phone adds a second yes and leaves hers, and that her number ticked again stays one row with a later date; with the old move (a tick clearing her other yeses first) it fails. Closed 2026-10-09 by the re-review of 9.7's review fix (scope: 06d4195..08e1a3e): `book-time.ts` no longer writes the contact; a tick upserts its own row in `later_texts_yes` (primary key business, contact and number; on conflict only `yesAt` moves), inside the booking's transaction, so a tick for another number can never touch hers. The contact's two columns and their checks are gone from the schema, the code and the dev database, and nothing else read them (`git grep`). Shown with temporary route tests (removed): a form sent again after its booking answers the same booking and leaves one yes and one timeline entry; three copies of one ticked form at once give one booking, one yes and one entry;

two different ticked forms at once with the same email and phone both book and leave one row; two ticked forms racing for one time (four rounds) leave the winner one yes and the loser no contact and no yes; a ticked form for a time already taken is a 409 with no contact and no yes; a failure thrown after the yes inside the transaction (a temporary throw in a copy of book-time.ts, restored) answers 500 and leaves no yes and no contact; a yes for the painter shows nothing under another business's same email. The suite's own tests still hold an unticked booking keeping the earlier yes, a tick with no phone refused with nothing saved, and a business that does not ask saving no yes. Tenant scope, on a throwaway database: a row naming business A with business B's contact, and the reverse, are refused by `later_texts_yes_contact_fk`; a number not as Twilio texts it by `later_texts_yes_phone_check`; deleting the contact removes its yeses. Migration: a fresh `*_dev` database migrated to 0023 with an organization and contacts in it, then the real folder, applies 0024 cleanly (25 ledger rows, rows kept, the contact without the old columns, `askLaterTextsYes` default false), and `db:seed` runs on it twice; the dev database's ledger holds the regenerated 0024's hash; `drizzle-kit generate` against a copy of the migrations says no schema changes. Breaking it fails the suite (book-time.ts copied aside and restored, sha256 c526ac0d... before and after): deleting the contact's other yeses before the upsert (the old move) fails "a tick with another number adds its own yes and never moves hers" at line 447, one number left. One gap left outside this finding is F-292. Suites: shared 167 of 167, backend 887 of 887, booking component 75 of 75.

9/F-292 [P3] closed - The test does not catch a repeat tick that leaves the old date

File: backend/routes/public-bookings-routes.test.ts:457 (`toBeGreaterThanOrEqual` on her number's date after a second tick) **Found:** 2026-10-09 by the re-review of 9.7's review fix (scope: 06d4195..08e1a3e; lens: tests) **Why it matters:** The table is meant to date each number by her latest yes, and F-291's Resolution says the test checks that her number ticked again stays one row "with a later date". It allows the same date, so the date moving is not guarded: in a copy of book-time.ts with `.onConflictDoNothing()` in place of the update (restored after, sha256 c526ac0d... before and after), the whole yes block still passes, 6 of 6. Nothing reads the date yet, so no customer is affected today; whatever later picks who may get later texts by how recent the yes is (feature 12 onward) would read her first yes's date. **Suggested fix:** Expect the date to be strictly later (`toBeGreaterThan`). The route takes the real clock and two bookings in a row are milliseconds apart: with that one change the test passed three runs in a row on the fix, and failed on the `onConflictDoNothing` copy at line 457 with the two dates equal. **Resolution:** Fixed 2026-10-09: the re-tick check is `toBeGreaterThan`, so her number ticked again must carry a later date. It passed three runs on the fix, and with the upsert replaced by `onConflictDoNothing()` it fails. Closed 2026-10-09 by the re-review of 9.7's second fix (scope: 1cf4830..b89aaee): the one line changed is the re-tick check at public-bookings-routes.test.ts:457, now `toBeGreaterThan`, and it guards the date moving. Breaking it fails the file (book-time.ts copied aside and restored, sha256 c526ac0d... before and after): with `.onConflictDoNothing()` in place of the upsert, "a tick with another number adds its own yes and never moves hers" fails at line 457, "expected 1791586271749 to be greater than 1791586271749", 1 failed and 42 passed. Not flaky: the route takes `new Date()` per request and her two ticks have the other number's whole booking between them; measured in a temporary copy of the test (removed), the gap was 99 to 144 ms over 12 runs, and the file

itself passed 8 runs of 8, 43 of 43 each. Nothing else in the scope changed. Suites: backend 887 of 887 (78 files).

9/F-293 [P2] closed - Back to the times stays live while Book is sending, so a lost answer after Back loses the freeze and "already used" books her twice

File: packages/booking-component/booking-window/screens/month-layout/month-details-screen.tsx:53,60 (Back is hidden only once `unsure` is set) and packages/booking-component/booking-window/screens/booking-form-view.tsx:115-133 (`onUnsureChange` is called only after the answer), :213 (the fieldset is disabled only while `unsure`); booking-window/booking-window.tsx:41,213-216 (the form and its key survive Back); the words come from backend/routes/public-bookings-routes.ts:137-142

Found: 2026-10-09 by /audit independent (scope: current, ece1aa2..012c200; lens: quality, tests) **Why it matters:** F-282's fix freezes the form after a lost answer, but only from the moment the answer is known to be lost. While the first send is on its way (up to the 10-second limit in create-booking-api-client.ts:11), the rail still draws "Back to the times" and every field is editable. A customer who presses Back during "Booking..." unmounts the form, so when that send then ends with no answer, its `setUnsure(true)` and `onUnsureChange(true)` land on unmounted components and nothing is frozen. She is on screen one with the same form and key (kept by the window), picks another time, and Book sends that key with a different start. If the lost send was saved, book-time.ts finds the key's booking, `isSameRequest` fails on the start, and the route answers 409 request_key_used, "This booking form was already used. Please reload the page and book again.", which send-booking reads as a settled refusal. Doing what it says makes a second booking, the outcome F-282 (P2) was about, reached by pressing Back a few seconds earlier. The same live fieldset lets her edit the email during the send; if it is then lost, Try again sends the edited address, the API answers the first booking, and the done screen names an address the confirmation never went to (F-282's smaller case). Traced by reading; no test drives Back or an edit while a send is held (booking-window.test.ts:165 covers Back only after the answer is lost). **Suggested fix:** Treat the form as in doubt from the press, not from the lost answer: hide (or disable) Back and disable the fieldset while `sending` as well as while `unsure` , for example by calling `onUnsureChange(true)` as the send starts and settling it when the answer comes. A booking-window.test.ts case that holds the first send, checks Back is not drawn and the fields are disabled while it is held, then drops the answer and checks the form is frozen; shown able to fail with Back drawn during the send. **Resolution:** Fixed 2026-10-09 in the final review's fixes: booking-form-view.tsx freezes the form from the press, not only after a lost answer (`frozen = sending || unsure`): the fieldset is disabled and Book gives way to the role=status line, which takes the focus, for every send, and `onFrozenChange(true)` is called as the send starts, so month-details-screen.tsx hides Back to the times for the whole send (the prop and state renamed from unsure to frozen). The now unused `.sa-submit:disabled` rule and the `retrying` state are gone. booking-window.test.ts "hides Back to the times and locks the fields from the press, so a lost answer stays frozen" holds the first send, checks Back is not drawn, the email field is disabled and the line says Booking..., then drops the answer and checks it stays frozen with Try again; shown to fail on the old code (Back drawn). booking-form-view.test.ts's first-send case now checks the line, the focus and the disabled fields. Closed 2026-10-10 by the final independent review (scope: current, ece1aa2..28a2f1c): in

booking-form-view.tsx `send` calls `onFrozenChange(true)` before `sendBooking` is awaited and `frozen = sending || unsure` disables the fieldset (every field and the later-texts box) and removes Book from the press, so month-details-screen.tsx draws no "Back to the times" for the whole send; after the answer `onFrozenChange(nowUnsure)` keeps Back hidden when the answer is lost or unclear and gives it back once the route settles the form. `sendBooking` never rejects (the call and the body read are both caught), so a send always reaches that second call and nothing stays frozen for good. The window's other ways off screen two are the Close button and Escape, which start a new form as the plan says. Focus: the "Booking..." status takes it as the send starts (effect on `sending`), the words take it when a send ends with words, and a booking or a taken time moves it to the next screen's heading. booking-window.test.ts "hides Back to the times and locks the fields from the press, so a lost answer stays frozen" holds the first send and asserts no Back and a disabled Email while it is held, then still frozen with Try again after the drop; on the earlier code Back was drawn while `unsure` was false, so the first assertion fails by reading. Not re-run against a broken copy: this reviewer may not edit product code. Package suite 76 of 76.

9/F-294 [P3] closed - The coding standards still say browser or proxy is an open question, which decision 1 answered

File: blueprint/context/coding-standards.md:229-231 ("Whether the booking widget calls the API from the browser or proxies through the host site's Server Action is open until Phase 3 (`project-plan.md` , open question 5)") **Found:** 2026-10-09 by /audit independent (scope: current, ece1aa2..012c200; lens: quality) **Why it matters:** This feature answered it: the spec's decision 1 and project-plan.md decision 31 (open question 15, "Answered 2026-10-08: browser to API"), and step 9.1's plan lines refreshed the project plan, build plan and overview but not this file. coding-standards.md is the file read before changing code, so feature 10, which wires the component into a host site, meets a rule saying the question is still open and pointing at a question number that no longer holds it. The workspace rule is that the levels never contradict each other. **Suggested fix:** Replace the line with the decision: the booking component calls the public routes from the visitor's browser (project-plan.md decision 31), with no Server Action proxy, because booking holds no secret and the rate limits must see each visitor. **Resolution:** Fixed 2026-10-09: coding-standards.md's open question is replaced by the decision: the booking component calls the public routes straight from the visitor's browser, never through a Server Action proxy (project-plan.md decision 31), because booking holds no secret and the rate limits must see each visitor. Closed 2026-10-10 by the final independent review (scope: current, ece1aa2..28a2f1c): coding-standards.md:229-232 now states the decision (browser to the public routes, never a Server Action proxy, project-plan.md decision 31, with its reason), and project-plan.md:541 records open question 15 as answered by decision 31. No other line in coding-standards.md calls the question open.

9/F-295 [P3] closed - History is back in comments: who agreed the problem words and when, and finding numbers in eight test lines

File: packages/booking-component/booking-window/screens/problem-words.ts:1-2; backend/routes/public-booking-links-routes.test.ts:366; backend/routes/public-booking-move-times-routes.test.ts:326; backend/routes/public-booking-page-routes.test.ts:185; packages/booking-

component/api-client/fetch-one-service.test.ts:23,36; packages/booking-component/booking-window/month-calendar/group-times-by-day.test.ts:25; packages/booking-component/booking-window/screens/month-layout/use-month-times.test.ts:65,123 **Found:** 2026-10-09 by /audit independent (scope: current, ece1aa2..012c200; lens: quality) **Why it matters:** coding-standards.md, Comments: "No history in code comments (step numbers, finding numbers ...): that lives in the build log." problem-words.ts, product code, opens with "The words Frank agreed at step 9.4's plan (2026-10-09)", a step number and who agreed when, the pattern F-274 removed. F-285's repair cleared code outside tests only; eight test names and comments still carry F-278 to F-281. `/complete` archives this ledger as `9/F-...` and the next one restarts the numbering, so a bare F-279 in a test later points at an unrelated finding. **Suggested fix:** Keep each sentence and drop the history: "what the window says for each problem" in problem-words.ts, and the test names and comments without their "(F-27x)" or "F-279:" (the reason, such as an older browser's time-zone rules, already reads on its own). **Resolution:** Fixed 2026-10-09: problem-words.ts opens with "what the window says for each problem" only; the eight test names and comments lose their "(F-27x)" and "F-279:" with each sentence kept; AGENTS.md's "since F-280" reads "since feature 9". `git grep` finds no F-number left in backend, frontend or packages code. Closed 2026-10-10 by the final independent review (scope: current, ece1aa2..28a2f1c): problem-words.ts opens with "what the window says for each problem" only; the eight named test lines lose only their finding numbers, each sentence kept; AGENTS.md says "since feature 9". `git grep -n -E "\bF-[0-9]{2,3}\b" -- backend frontend packages` (migrations aside) finds nothing, tests included. One step number left in a CSS heading is recorded on its own as F-297, since it is not one of the lines this finding named.

9/F-296 [P3] closed - The seed sets painting-dev to ask for a yes only when it first makes its text settings, so a database seeded before 0024 never shows the box

File: packages/shared/scripts/seed-dev.ts:417-429 (text settings inserted with `onConflictDoNothing`, never updated) with packages/shared/migrations/0024_later_texts_yes.sql:11 (`askLaterTextsYes` added DEFAULT false) **Found:** 2026-10-09 by /audit independent (scope: current, ece1aa2..012c200; lens: quality) **Why it matters:** Step 9.7.1 says the seed sets painting-dev to ask, and its Done when has the box seen on the painting-dev preview. painting-dev has had a text_settings row since feature 8b, so on any dev database seeded before this step, `db:migrate` gives it `askLaterTextsYes = false` and `db:seed` leaves it there: the preview shows no box and nothing says why. The seed's own rule for the same situation in this feature (who picks the person, seed-dev.ts:584-623, from F-257) is to bring an earlier database up "so no machine needs a rebuild". No test depends on it (the route and job tests make their own settings), so only the dev preview is affected. **Suggested fix:** As with who picks: set `askLaterTextsYes` from the seed on an existing row while it still holds the migration's false and nothing else in the row was changed by hand, or note in the seed and AGENTS.md that a database seeded before 0024 is rebuilt. **Resolution:** Fixed 2026-10-09 the way F-257 was: seed-dev.ts parses the seeded text settings once, and on an existing row that still holds 0024's false while every other field is still the seed's, sets `askLaterTextsYes` and prints its own line. Shown on the local `scheduleads_dev`: painting-dev set to false, db:seed brought it up with the line; a second db:seed changed nothing; with `confirmationOn` also

changed by hand, db:seed left the row as it was; the row was then restored. Closed 2026-10-10 by the final independent review (scope: current, ece1aa2..28a2f1c): seed-dev.ts parses the seeded text settings once (`seededTexts`), inserts them only while the business has none, and when the row already existed and the seed asks, reads it and sets `askLaterTextsYes` only while it is still false and fromNumber, confirmationOn, reminderMinutesBefore, replyPhone and replyEmail all equal the parsed seed values, inside the seed's transaction, printing its own line. The stored numbers were written from the same parse, so the comparison matches an untouched row (painting-dev's replyEmail is null in both). A second run finds it already true and changes nothing; a row changed by hand in any of those fields is left alone, the same rule F-257 uses for who picks. Read only; this reviewer did not run `db:seed` against the shared dev database.

9/F-297 [P3] closed - Two comments still carry history: a step number in the package's CSS, and a body-limit comment that points at the wrong feature's decision

File: packages/booking-component/booking-component.css:877 ("SCREEN ONE ADDITIONS (step 9.5, decision 12)"); backend/routes/public-bookings-routes.ts:29-30 ("Decision 10: ... (feature 9)") **Found:** 2026-10-10 by /audit independent (scope: current, ece1aa2..28a2f1c; lens: quality) **Why it matters:** coding-standards.md:365-366: "No history in code comments (step numbers, finding numbers ...): that lives in the build log." F-274, F-285 and F-295 cleared the finding numbers; this heading still names step 9.5, the only step number left in backend, frontend or packages code (grep for "step [0-9]" over .ts, .tsx and .css). Separately, the body limit's comment now reads "Decision 10: ... (feature 9)", but the 16 KB limit was 5d's decision 10 (blueprint/history/features/05d-the-booking.md:99) and the growth to 64 KB comes from step 9.2's plan, not a numbered decision; feature 9's own decision 10 is the native dialog, so the reference sends a reader to the wrong rule. No behaviour is affected. **Suggested fix:** Drop "step 9.5, " from the CSS heading (keep "decision 12" or say "screen one"). Reword the route comment to say what it is without the history, for example "A form with every field full, 20 full answers in any script included, is far below this." **Resolution:** Fixed 2026-10-10 as suggested: the CSS heading reads "SCREEN ONE ADDITIONS (decision 12)", and the body-limit comment reads "A form with every field full, 20 full answers in any script included, is far below this." git grep finds no step or finding number left in backend, frontend or packages code, apart from migration 0000's "(F-06)", an applied migration from before feature 9 that is never edited. Closed 2026-10-10 by the final independent re-review (scope: current, ece1aa2..087c090): 087c090 changes only those two comments in code. booking-component.css:877 now reads "SCREEN ONE ADDITIONS (decision 12)", and public-bookings-routes.ts:29 reads "A form with every field full, 20 full answers in any script included, is far below this." with `MOST_BYTES = 64 * 1024` unchanged beneath it. `git grep -i -E "step [0-9]|\bF-[0-9]{2,3}\b"` over backend, frontend and packages .ts, .tsx, .css and .sql finds only migration 0000's "(F-06)" and "Step 1.3", both from before feature 9 in an applied migration. The remaining "(feature N, decision M)" references point at the spec, the pattern the code used before this feature, and are not the history the standard names. No behaviour changed; format:check, the backend build and the booking component build pass.

Independent review

Status: passed **Target commit:** 087c090c13432cc1c3479a4fc0243a1bc9556bbe **Base commit:** ece1aa2b995f5395f9f35fdd568a985659a3495f **Base ref:** main **Spec hash:** f24901865565ed52eaa3463678280f41266c06badbfee83e51d1ed3cf0f30005 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** runtime default (exact model not known until reviewer starts) **Requested execution:** automatic **Requested at:** 2026-10-10T00:44:49.000Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-10T00:48:46.000Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete

`ece1aa2b995f5395f9f35fdd568a985659a3495f..087c090c13432cc1c3479a4fc0243a1bc9556bbe` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

The untracked `blueprint/ai-voice-proposal.md` is the owner's own file, never committed and outside this work item; it is not part of the target and does not make the review stale.

Commands

- `npm run test --workspace=@scheduleads-app/shared` : pass (22 files, 167 tests)
- `npm run test --workspace=backend` : pass (78 files, 887 tests, against the local seeded scheduleads_dev)
- `npm run test --workspace=@scheduleads-app/booking-component` : pass (16 files, 76 tests)
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=@scheduleads-app/booking-component` : pass
- `npm run build --workspace=frontend` : pass (includes /admin/booking-preview/[businessSlug] and /customer-booking/[bookingPageToken])
- `npm run lint --workspace=frontend` : pass
- `npm run format:check` : pass

Evidence

- Freshness: HEAD 087c090c, `git merge-base main HEAD` ece1aa2b, sha256 of current-feature.md matches the spec hash; the only differing paths were review.md (this file) and the owner's untracked ai-voice-proposal.md, before and after the commands.

- Whole delta: 55 commits, 205 files. Read: the public routes (booking links, bookings, booking page), app.ts and PublicAppType, bookTime, checkAnswers, the question, person-choice and later-texts helpers, move and move-times, the rate limiter, its middleware, visitor address and contact keys, oneCopyOfAFormAtATime, migrations 0022 to 0024 and their tables, the notification email, the shared create-booking schema, the whole booking component (provider, trigger, API client, window, screens, form, month calendar, CSS), the preview page, the change-time panel, the frontend api-client split, sign-in and dashboard home, AGENTS.md and standards changes. Migration snapshot JSON and earlier findings text were not read line by line.
- Security: public answers carry only the business's name, logo, phone, questions and the box sentence; the bookings route keeps the body limit before the validator, the organization from the slug only, and every refusal in the `{ error: { code, message } }` shape; user and business text is rendered as text (React, react-email); the package CSS reads only `--sa-*` tokens and every class is `sa-` prefixed; no product name appears in the package's source.
- Tests: no `.only`, `.skip` or `.todo` added in the delta; no skipped tests in any suite run.
- F-297 re-examined against 087c090 and closed (see its Resolution).

Findings

- F-297 [P3] closed by this pass.
- No new findings.

Remaining risk

- Check was not required and was not run; the booking window, preview page and customer page were not exercised in a browser by this reviewer (no browser test harness is declared).
- Per the spec's deploy notes, unverified until the first deploy: Railway's proxy replacing a visitor-sent `X-Real-IP`, and the API running with `NODE_ENV=production` (without it every visitor shares one rate-limit count behind the proxy). The counts and the form queue live in one API copy's memory.
- With nobody picked, the move times route reads the booked person's calendar a second time beside the any-available read (F-269's repair), so one extra Google busy-time read per such request; not measured.
- The package's built types import `backend/app-type`, which resolves only inside this repo; how another repo consumes the package is feature 10's question.